

VĂN THỊ MINH HUYỀN

KHOA HỌC MÁY TÍNH

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



ĐỒ ÁN TỐT NGHIỆP

HỌC TĂNG CƯỜNG ĐỂ TỐI ƯU HÓA
KHẢ NĂNG TƯƠNG TÁC CỦA AI TRONG GAME

Sinh viên: Văn Thị Minh Huyền

Mã số sinh viên: 22010329

Khóa: K16

Ngành: Khoa học máy tính (Tài năng)

Hệ: Đại học chính quy

Giảng viên hướng dẫn: Th.S Nguyễn Văn Sơn

HÀ NỘI, THÁNG 8 NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA

ĐỒ ÁN TỐT NGHIỆP

HỌC TĂNG CƯỜNG ĐỂ TỐI ƯU HÓA
KHẢ NĂNG TƯƠNG TÁC CỦA AI TRONG GAME

Họ và tên: Văn Thị Minh Huyền

Mã sinh viên: 22010329

Khóa: K16

Ngành: Khoa học máy tính (Tài năng) **Hệ:** Đại học chính quy

Giảng viên hướng dẫn: Th.S Nguyễn Văn Sơn

HÀ NỘI, THÁNG 8 NĂM 2026

LỜI CAM ĐOAN

Tên tôi là: Văn Thị Minh Huyền

Mã sinh viên: 22010329

Ngành: Khoa học máy tính

Tôi đã thực hiện đồ án tốt nghiệp với đề tài: Học tăng cường để tối ưu hóa khả năng tương tác của AI trong game

Tôi xin cam đoan đây là đề tài nghiên cứu của riêng tôi và được sự hướng dẫn của Th.S Nguyễn Văn Sơn.

Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa được các tác giả khác công bố dưới bất kỳ hình thức nào. Nếu phát hiện có bất kỳ hình thức gian lận nào tôi xin hoàn toàn chịu trách nhiệm trước pháp luật.

GIẢNG VIÊN HƯỚNG DẪN

Hà Nội, ngày ... tháng ... năm 20...

(Ký, ghi rõ họ tên)

SINH VIÊN

(Ký, ghi rõ họ tên)

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc tới quý thầy cô Trường Công nghệ thông tin Phenikaa. Suốt những năm em học tại trường, thầy cô đã tận tình giảng dạy, truyền đạt kiến thức và tạo mọi điều kiện thuận lợi để em theo đuổi việc học.

Em đặc biệt cảm ơn thầy Nguyễn Văn Sơn, giảng viên hướng dẫn đồ án tốt nghiệp của em. Thầy đã dành rất nhiều thời gian và công sức để định hướng, góp ý và hỗ trợ em qua từng giai đoạn của đề tài. Nếu không có những chỉ dẫn tận tâm ấy, đồ án này khó mà hoàn thành.

Em cũng xin gửi lời biết ơn tới gia đình. Đó là những người đã luôn đồng hành, động viên và lo cho em điều kiện tốt nhất để em toàn tâm với chương trình học.

Kiến thức và kinh nghiệm của em còn hạn chế, nên đồ án chắc chắn vẫn còn thiếu sót. Em rất mong nhận được góp ý từ quý thầy cô và các bạn để đề tài hoàn thiện hơn.

Em xin chân thành cảm ơn!

TÓM TẮT ĐỒ ÁN

Trong một dự án học tăng cường cho game, phần việc nặng nhất hiện không còn nằm ở thuật toán. Các thư viện đã đóng gói sẵn những gì cần thiết. Cái còn lại, và cũng là cái khó, là môi trường: phải tự dựng, tự đặc tả, tự hiệu chỉnh. Đồ án này lấy chính khâu đó làm đối tượng nghiên cứu, cụ thể là bài toán xây dựng môi trường game ba chiều đủ chuẩn cho việc huấn luyện.

Ba môi trường đã được dựng trên nền tảng Unity ML-Agents, mỗi cái ứng với một dạng bài toán khác nhau. Football Table là bài toán đối kháng một-một với không gian hành động liên tục và cơ chế tự chơi. Capture The Flag đòi hỏi hai tác nhân hợp tác, dùng phần thưởng nhóm cùng chương trình học chia theo bậc. Cross The Road thì ngược lại, chỉ một tác nhân điều hướng nhưng phần thưởng rất thưa. Với từng môi trường, đồ án thuật lại quá trình thiết kế bộ ba quan sát - hành động - phần thưởng. Sản phẩm cuối được đóng thành một gói Unity Package Manager, kèm 15 câu hình huấn luyện, bộ công cụ Editor và trình hướng dẫn cài đặt.

Để nghiệm thu, đồ án cho chạy năm thuật toán - PPO, SAC, MA-POCA, MAPPO và DQN - trên cả ba môi trường, giữ nguyên cấu hình lẫn ngân sách bước chạy. Năm thuật toán ở đây đóng vai trò phép thử: một môi trường đạt yêu cầu thì tác nhân phải học được, các hướng tiếp cận phải phân biệt được với nhau, và không được tồn tại lỗi tắt. Cả ba đều qua được phép thử này. Phần thưởng cuối trải rộng, và mọi khoảng cách đều truy được về cơ chế của bài toán; SAC chẳng hạn, dẫn đầu trên Cross The Road với 0,846 nhưng sụp xuống 0,183 ở Capture The Flag. Đáng chú ý hơn là điều này: cùng một thuật toán, khoảng cách giữa hai môi trường luôn lớn hơn khoảng cách giữa các thuật toán trong cùng một môi trường. Nói cách khác, đặc tính bài toán - thứ do thiết kế môi trường quyết định - chi phối kết quả mạnh hơn lựa chọn thuật toán. Kèm theo đó, đồ án xây dựng một ứng dụng trình diễn nạp trực tiếp mười lăm mô hình ONNX đã huấn luyện để quan sát hành vi thật.

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	xi
1 MỞ ĐẦU	1
1.1 Lý do chọn đề tài	1
1.2 Mục tiêu nghiên cứu	3
1.3 Đối tượng và phạm vi nghiên cứu	4
1.4 Tổng quan tài liệu và khoảng trống nghiên cứu	5
1.5 Phương pháp nghiên cứu	7
1.6 Ý nghĩa khoa học và thực tiễn	7
1.7 Bố cục đồ án	8
2 CƠ SỞ LÝ THUYẾT	9
2.1 Trí tuệ nhân tạo trong game	9
2.1.1 AI truyền thống	9
2.1.2 AI học máy	10
2.2 Nền tảng Reinforcement Learning	11
2.2.1 Các khái niệm cơ bản	11
2.2.2 Markov Decision Process	12
2.2.3 Policy và Value Function	13
2.3 Học tăng cường sâu	14
2.3.1 Vai trò của mạng Neural Network trong RL	14
2.3.2 Phân loại: Policy-Based, Value-Based, Actor-Critic	15
2.3.3 Phần thưởng tò mò nội tại	16
2.4 Các thuật toán sử dụng trong đề tài	18
2.4.1 PPO	18
2.4.2 SAC	19
2.4.3 MA-POCA	21
2.4.4 MAPPO	22
2.4.5 DQN	24
2.4.6 Tổng kết và so sánh năm thuật toán	25
2.5 Unity ML-Agents Toolkit	26

2.5.1	Kiến trúc tổng thể	27
2.5.2	Cơ chế Vật lý và Tính xác định trong Unity	28
2.5.3	Hệ thống Cảm biến và Tiền xử lý quan sát	28
2.5.4	Cơ chế Hành vi và Gom nhóm quyết định	29
2.5.5	Quy trình huấn luyện Agent	30
2.5.6	Hỗ trợ thuật toán và giới hạn tích hợp sẵn	30
3	THIẾT KẾ MÔI TRƯỜNG VÀ PHƯƠNG PHÁP THỰC NGHIỆM	31
3.1	Tổng quan thiết kế hệ thống	32
3.1.1	Kiến trúc huấn luyện tổng thể	32
3.1.2	Không gian quan sát	33
3.1.3	Không gian hành động	35
3.1.4	Nguyên tắc thiết kế hàm phần thưởng	36
3.2	Bộ môi trường thực nghiệm	40
3.2.1	Tích hợp với Unity ML-Agents	40
3.3	Môi trường Football Table	43
3.3.1	Mô tả bài toán	43
3.3.2	Không gian quan sát	44
3.3.3	Không gian hành động và cơ chế điều khiển	45
3.3.4	Hàm phần thưởng	45
3.3.5	Điều kiện kết thúc lượt chơi	47
3.3.6	Cấu hình huấn luyện	48
3.4	Môi trường Capture The Flag	48
3.4.1	Mô tả bài toán	48
3.4.2	Cửa ải khối vật cản	49
3.4.3	Không gian quan sát	50
3.4.4	Không gian hành động và cơ chế điều khiển	51
3.4.5	Cơ chế cửa và pha bàn giao	51
3.4.6	Hàm phần thưởng	52
3.4.7	Quá trình hiệu chỉnh hàm phần thưởng	56
3.4.8	Điều kiện kết thúc lượt chơi	58
3.4.9	Cấu hình huấn luyện và học theo chương trình	58

3.5	Môi trường Cross The Road	59
3.5.1	Mô tả bài toán	59
3.5.2	Không gian quan sát	60
3.5.3	Không gian hành động và cơ chế điều khiển	60
3.5.4	Hàm phần thưởng	61
3.5.5	Điều kiện kết thúc lượt chơi	62
3.5.6	Cấu hình huấn luyện	62
3.6	Bảng tổng hợp đặc trưng ba môi trường	63
3.7	Triển khai thuật toán ngoài ML-Agents	65
3.7.1	Hai thuật toán không có sẵn trong ML-Agents	65
3.7.2	Bổ sung thuật toán bằng cơ chế trainer mở rộng	65
3.7.3	Kết nối thư viện ngoài qua Python Low-Level API	66
3.7.4	Cấu trúc mã nguồn bổ sung	67
3.8	Thiết lập thực nghiệm	67
3.8.1	Cấu hình phần cứng và phần mềm	67
3.8.2	Tham số huấn luyện từng thuật toán	68
3.8.3	Tiêu chí đánh giá	69
4	KIỂM CHỨNG MÔI TRƯỜNG QUA THỰC NGHIỆM	71
4.1	Kết quả môi trường Football Table	73
4.2	Kết quả môi trường Cross The Road	76
4.3	Kết quả môi trường Capture The Flag	81
4.4	Đánh giá tổng hợp	86
4.4.1	So sánh tốc độ hội tụ	86
4.4.2	So sánh mức năng lực đạt được	87
4.4.3	So sánh độ ổn định	88
4.4.4	Bảng chứng từ chỉ số huấn luyện nội tại	89
4.4.5	Phân tích ưu nhược điểm từng thuật toán	90
4.5	Thảo luận	92
4.5.1	Khả năng ứng dụng trong phát triển game	92
5	XÂY DỰNG ỨNG DỤNG TRÌNH DIỄN TƯƠNG TÁC	92

5.1	Mục tiêu và yêu cầu chức năng	93
5.2	Kiến trúc hệ thống	94
5.3	Giao diện menu và luồng sử dụng	96
5.4	Nạp mô hình ONNX tại thời gian chạy	98
5.5	Trực quan hóa kết quả huấn luyện trong ứng dụng	100
5.5.1	Đường ống dữ liệu từ TensorBoard vào game	100
5.5.2	Tab THÔNG SỐ	101
5.5.3	Tab SO SÁNH	102
5.6	Ba chế độ chơi trong từng môi trường	103
5.6.1	Cross The Road: trải nghiệm và quan sát chính sách	104
5.6.2	Capture The Flag: phối hợp người - máy	105
5.6.3	Football Table: đối đầu người - máy và máy - máy	106
5.7	Đóng gói ba môi trường thành gói dùng lại được	108
5.7.1	Cấu trúc gói	108
5.7.2	Gỡ phụ thuộc ngược từ môi trường vào ứng dụng	109
5.7.3	Trình hướng dẫn cài đặt	110
5.7.4	Dự án tự dùng chính gói của mình	112
5.8	Các vấn đề kỹ thuật và giải pháp	112
5.9	Kiểm thử ứng dụng và hạn chế	115
6	KẾT LUẬN	117
6.1	Kết quả đạt được	117
6.2	Đóng góp của đề tài	119
6.3	Hạn chế	120
6.4	Hướng phát triển	121
	PHỤ LỤC	xvii
	Phụ lục A. Cấu hình huấn luyện PPO	xvii
	Phụ lục B. Cấu hình huấn luyện SAC	xvii
	Phụ lục C. Cấu hình huấn luyện MA-POCA	xix
	Phụ lục D. Cấu hình huấn luyện DQN	xix
	Phụ lục E. Mã nguồn minh họa	xx

Phụ lục F. Kết quả TensorBoard	xxi
--	-----

DANH SÁCH HÌNH VẼ

2.1	Vòng lặp tương tác giữa agent và môi trường trong học tăng cường	12
2.2	Sơ đồ kiến trúc Intrinsic Curiosity Module (ICM)	17
2.3	So sánh cấu trúc giữa MAPPO và MA-POCA	23
2.4	Kiến trúc tổng thể của Unity ML-Agents Toolkit	28
3.1	Ba môi trường thực nghiệm. Trên trái: Football Table, hai đội Xanh - Đỏ mỗi đội bốn thanh gạt. Trên phải: Capture The Flag, hai tác nhân phối hợp giữ cửa để cùng tới đích. Dưới: Cross The Road, một tác nhân băng qua các làn xe đang chạy	41
3.2	Sơ đồ bố cục sân và cơ chế điều khiển thanh gạt trong Football Table	44
3.3	Sơ đồ bố cục không gian và trình tự bàn giao trong Capture The Flag	49
4.1	Tiến trình chỉ số xếp hạng ELO trong cơ chế tự chơi trên Football Table	74
4.2	Đường cong phần thưởng tích lũy trên môi trường Football Table	75
4.3	Đường cong phần thưởng trên môi trường Cross The Road	77
4.4	Độ dài lượt chơi trên môi trường Cross The Road	79
4.5	Diễn biến entropy của chính sách trên môi trường Cross The Road	80
4.6	Hàm mất mát của mạng phê bình trên môi trường Cross The Road	81
4.7	Đường cong phần thưởng trên môi trường Capture The Flag	82
4.8	Độ dài lượt chơi trên môi trường Capture The Flag: bốn thuật toán rút được lượt chơi xuống mức thấp, riêng SAC nằm ở trần thời gian gần như suốt quá trình	84
4.9	Diễn biến entropy của chính sách trên môi trường Capture The Flag: ba thuật toán họ PPO giảm đều, riêng SAC sụp xuống rồi bật ngược về mức cực đại	85
4.10	Phần thưởng tò mò trên môi trường Capture The Flag (trục tung thang lô-ga-rit)	86
4.11	So sánh phần thưởng trung bình mười phần trăm cuối của năm thuật toán trên ba môi trường	88
5.1	Luồng hoạt động của ứng dụng trình diễn	95

5.2	Tab CHƠI của menu chính: khung xem trước ba chiều (trái) và các lựa chọn chế độ chơi (phải)	96
5.3	Khu vực chọn mô hình khi Football Table ở chế độ Agent tự chơi: hai ô chọn riêng cho đội Xanh và đội Đỏ	97
5.4	Tab THÔNG SỐ: đường cong phần thưởng tích lũy và bảng thống kê của năm thuật toán trên môi trường Cross The Road	102
5.5	Tab SO SÁNH: biểu đồ cột theo từng môi trường và bảng xếp hạng thuật toán	103
5.6	Chế độ Agent tự chơi trong Cross The Road: mô hình ctr_mappo_v1 điều khiển tác nhân, chỉ số hiển thị trực tiếp trong cảnh	105
5.7	Chế độ Chơi với Agent trong Capture The Flag: người chơi phối hợp cùng tác nhân đồng đội dùng mô hình ctf_mappo_v2	106
5.8	Chế độ Người chơi trong Football Table: người chơi cầm đội Xanh, góc trên trái hiển thị thanh gạt đang được chọn	107
5.9	Chế độ Agent tự chơi trong Football Table với mô hình football_dqn_v1: ứng dụng nạp bản scene rời rạc và chỉ giữ lại một bản trong số tám bản của cấu hình huấn luyện	108
5.10	Trình hướng dẫn cài đặt của gói: bảy bước, mỗi bước tự kiểm tra trạng thái và chỉ bật nút khi thực sự còn thiếu	111

DANH SÁCH BẢNG

2.1	Phân loại các thuật toán sử dụng trong đề tài	16
2.2	So sánh các thuật toán học tăng cường	26
3.1	Tốc độ huấn luyện thực đo trên ba môi trường, mỗi môi trường lấy một lần chạy đại diện; số bước là số bước huấn luyện của hành vi tương ứng	43
3.2	Hàm phần thưởng của môi trường Football Table	46
3.3	Hàm phần thưởng của môi trường Capture The Flag	55
3.4	Hàm phần thưởng của môi trường Cross The Road	62
3.5	Tổng hợp đặc trưng ba môi trường thực nghiệm	64
3.6	Cấu hình phần cứng và phần mềm thực nghiệm	68
3.7	Tóm tắt siêu tham số huấn luyện chính	69
4.1	Kết quả năm thuật toán trên môi trường Football Table	75
4.2	Kết quả năm thuật toán trên môi trường Cross The Road	77
4.3	Kết quả năm thuật toán trên môi trường Capture The Flag	82
4.4	Chỉ số huấn luyện nội tại (các môi trường hành động rời rạc)	89
4.5	Xếp hạng thuật toán theo từng môi trường	90
5.1	Các thành phần chính của ứng dụng trình diễn	94
5.2	Mười lăm mô hình ONNX được đóng gói sẵn trong ứng dụng	99
5.3	Ảnh xạ ba chế độ chơi vào ba môi trường	104
5.4	Bộ cục gói com.vanhuyen.rl-game-envs	109
5.5	Kết quả kiểm thử các luồng chức năng của ứng dụng	116
6.1	Kết quả thực đo của 15 lần huấn luyện	xxii

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Diễn giải
AI	Artificial Intelligence	Trí tuệ nhân tạo
RL	Reinforcement Learning	Học tăng cường
Deep RL	Deep Reinforcement Learning	Học tăng cường sâu
MDP	Markov Decision Process	Quá trình quyết định Markov
ONNX	Open Neural Network Exchange	Định dạng trao đổi mô hình mạng neural mở
PPO	Proximal Policy Optimization	Thuật toán tối ưu chính sách vùng lân cận
SAC	Soft Actor-Critic	Thuật toán actor-critic cực đại hóa entropy
MA-POCA	Multi-Agent POsthumous Credit Assignment	Thuật toán gán công lao hậu định cho đa tác nhân
MAPPO	Multi-Agent Proximal Policy Optimization	Tối ưu hóa chính sách cận biên đa tác nhân
TRPO	Trust Region Policy Optimization	Tối ưu chính sách theo vùng tin cậy
DQN	Deep Q-Network	Mạng Q sâu
GAE	Generalized Advantage Estimation	Ước lượng advantage tổng quát
FSM	Finite State Machine	Máy trạng thái hữu hạn
GOAP	Goal-Oriented Action Planning	Lập kế hoạch hành động hướng mục tiêu
NPC	Non-Player Character	Nhân vật phi người chơi
MLP	Multi-Layer Perceptron	Mạng perceptron nhiều lớp
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CTDE	Centralized Training Decentralized Execution	Huấn luyện tập trung, thực thi phân tán
ELO	Elo Rating System	Hệ thống xếp hạng Elo
API	Application Programming Interface	Giao diện lập trình ứng dụng

Từ viết tắt	Tiếng Anh	Diễn giải
gRPC	google Remote Procedure Call	Giao thức gọi thủ tục từ xa của Google
YAML	YAML Ain't Markup Language	Định dạng tệp cấu hình
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
SB3	Stable-Baselines3	Thư viện hiện thực các thuật toán RL

1. MỞ ĐẦU

1.1 Lý do chọn đề tài

Trò chơi điện tử có một vị thế khá lạ trong nghiên cứu trí tuệ nhân tạo. Chúng vừa là phòng thí nghiệm, vừa là sản phẩm. Chạy hàng triệu lần thử trong một môi trường game gần như không tốn gì và cũng chẳng mang rủi ro nào, thứ mà một con robot thật hay một hệ thống giao thông thật không bao giờ cho phép. Mặt khác, ngành công nghiệp game lại có nhu cầu thương mại rất trực tiếp: họ cần những nhân vật máy cừ xử sao cho thuyết phục. Quãng đường từ một kết quả nghiên cứu tới một tính năng bán được, hiếm lĩnh vực nào ngắn như vậy.

Học tăng cường ăn khớp với bối cảnh đó hơn cả. Hướng tiếp cận này không đòi hỏi tập dữ liệu gán nhãn sẵn, mà với câu hỏi “nước đi nào là đúng trong tình huống này” thì một tập như vậy gần như không thể có. Thay vào đó, tác nhân tự rút ra chiến lược từ hậu quả của chính những hành động nó đã thử.

Những gì học tăng cường làm được đã được chứng minh qua một loạt cột mốc. Có hệ thống chơi thành thạo hàng chục trò chơi điện tử cổ điển mà chỉ nhìn vào hình ảnh thô của màn hình [14]. Có hệ thống hạ được kỳ thủ cờ vây hàng đầu thế giới [23]. Có hệ thống điều phối nhiều tác nhân để thắng những trò chiến thuật phức tạp [16, 26]. Không hành vi nào trong số đó được lập trình sẵn cả; tất cả đều nảy sinh từ quá trình thử và sai của bản thân tác nhân.

Có điều, ấn tượng mà những cột mốc ấy để lại rất dễ khiến người ta nhầm về chỗ khó thật sự. Thuật toán học tăng cường ngày nay đều đã được cài đặt sẵn, kiểm chứng kỹ và gói gọn trong các thư viện chín muồi; gọi ra một trình huấn luyện PPO tốn đúng một dòng lệnh. Thứ không ai đưa sẵn cho ta, và cũng là thứ ngốn phần lớn công sức của một dự án học tăng cường thực tế, là *môi trường huấn luyện*. Tác nhân nhìn thấy những gì? Nó hành động ra sao? Thuởng phạt theo tiêu chí nào? Sau mỗi lượt thì tình huống được đặt lại thế nào? Và làm sao để cả hệ thống chạy đủ nhanh mà gom về hàng triệu bước kinh nghiệm? Người xây dựng phải tự trả lời từng câu một. Mỗi câu là một quyết định thiết kế, mà quyết định sai thì thường không báo lỗi: chương trình vẫn chạy bình thường, đường cong vẫn vẽ ra, chỉ có điều tác nhân không bao giờ học được thứ ta muốn.

Các môi trường hai chiều đơn giản hay những bộ dữ liệu chuẩn quen thuộc trong nghiên cứu học thuật rất tiện, nhưng chúng lại che mất đúng phần khó vừa nói: mọi đặc tả đã có người làm sẵn từ trước, người dùng chỉ việc nạp vào. Phải là một môi trường ba chiều - mô phỏng vật lý chân thực, quan sát chỉ một phần qua cảm biến, tình huống thay đổi liên tục - thì các bài toán thiết kế mới lộ ra hết, và bài toán khi đó cũng gần với phát triển game thương mại hơn nhiều. Những môi trường như thế, ở dạng dùng lại được, tức là có tài liệu, có công cụ, có cấu hình huấn luyện đi kèm, hiện gần như vắng bóng. Đó chính là khoảng trống mà đề án này nhắm tới.

Unity kết hợp với bộ công cụ ML-Agents cho một hệ sinh thái đủ mạnh để làm việc đó. Unity hiện thuộc nhóm công cụ phát triển game phổ biến nhất, mô phỏng vật lý và dựng hình ba chiều ở mức chuyên nghiệp; ML-Agents thì nối thẳng một cảnh Unity đang chạy với các thuật toán học tăng cường bên hệ sinh thái Python. Ghép hai thứ lại, ta có một nền tảng rất hợp để thiết kế đủ loại môi trường huấn luyện, từ đối kháng, hợp tác cho tới điều hướng tránh vật cản trong không gian ba chiều.

Từ những lý do đó, tôi chọn đề tài “Học tăng cường để tối ưu hóa khả năng tương tác của AI trong game”. Trọng tâm đặt vào việc **nghiên cứu và xây dựng môi trường huấn luyện** cho tác nhân học tăng cường trên nền tảng Unity ML-Agents: ba môi trường ba chiều thuộc ba dạng bài toán khác nhau, kèm theo toàn bộ quá trình thiết kế quan sát - hành động - phần thưởng, các công cụ hỗ trợ và cách đóng gói để người khác dùng lại được.

Năm thuật toán PPO, SAC, MA-POCA, MAPPO và DQN vẫn được huấn luyện đầy đủ trên cả ba môi trường, nhưng vai trò của chúng ở đây là *kiểm chứng môi trường*, không phải xếp hạng thuật toán. Lý do khá đơn giản: nhìn vào mã nguồn thì không thể nghiệm thu một môi trường huấn luyện. Cách duy nhất để biết đặc tả của nó đúng hay sai là thả nhiều thuật toán khác nhau vào rồi xem chúng cư xử thế nào. Nếu mọi thuật toán đều thất bại, khả năng cao lỗi nằm ở hàm phần thưởng chứ không phải ở thuật toán. Nếu thuật toán nào cũng đạt điểm tối đa, khả năng cao là đang có lỗi tịt ở đâu đó. Chỉ khi các thuật toán tách ra thành thứ hạng giải thích được thì môi trường mới thực sự đo đúng thứ nó định đo. Đích đến của đề tài, vì vậy, là để lại một bộ môi trường đã qua kiểm chứng và đã đóng gói, phục vụ cho

các nghiên cứu cũng như dự án phát triển game về sau.

1.2 Mục tiêu nghiên cứu

Xét tổng quát, đề tài muốn nghiên cứu quy trình thiết kế, xây dựng và kiểm chứng môi trường huấn luyện cho tác nhân học tăng cường trong game ba chiều trên nền tảng Unity ML-Agents, đồng thời để lại một bộ môi trường dùng lại được. Mục tiêu tổng quát ấy được cụ thể hóa thành năm mục tiêu nhỏ hơn, trong đó ba mục tiêu đầu giữ vai trò trọng tâm.

- **Xây dựng bộ môi trường huấn luyện.** Ba môi trường game ba chiều được dựng lên với tính chất khác biệt nhau, phủ ba dạng bài toán học tăng cường phổ biến. Football Table mô phỏng một bàn bi lắc ba chiều, đại diện cho đối kháng một đối một trong không gian hành động liên tục. Capture The Flag bắt nhiều tác nhân phối hợp để cùng hoàn thành một mục tiêu chung. Cross The Road buộc tác nhân điều hướng vượt qua dòng phương tiện chạy liên tục để tới đích. Sở dĩ chọn ba dạng này là vì chúng đặt ra ba nhóm bài toán thiết kế khác hẳn nhau: gán công lao trong tự chơi, phần thưởng nhóm trong hợp tác, và phần thưởng thừa trong điều hướng.
- **Nghiên cứu quy trình thiết kế quan sát, hành động và phần thưởng.** Đây là phần lõi học thuật của đề tài. Với mỗi môi trường, đề án không chỉ đưa ra phương án cuối cùng mà thuật lại cả con đường đi tới nó: những đặc tả đã thử rồi thất bại, dấu hiệu để nhận ra một hàm phần thưởng bị hỏng, và cách sửa. Cái muốn rút ra là những nguyên tắc thiết kế dùng lại được, chứ không phải ba cấu hình tình cờ chạy được.
- **Đóng gói môi trường thành sản phẩm dùng lại được.** Một môi trường chỉ có giá trị với người khác khi họ cài được nó vào dự án của mình. Cả ba môi trường vì thế được gói thành một gói Unity Package Manager, kèm cấu hình huấn luyện, bộ công cụ Editor (nhân khu vực huấn luyện, build headless, sinh biến thể không gian hành động) và một trình hướng dẫn cài đặt biết tự kiểm tra trạng thái từng bước.

- **Kiểm chứng môi trường bằng thực nghiệm đa thuật toán.** Năm thuật toán PPO, SAC, MA-POCA, MAPPO và DQN được huấn luyện trên từng môi trường, cùng điều kiện phần cứng và cùng ngân sách bước, tạo thành một lưới đầy đủ mười lăm lần chạy. Mục đích là xác nhận ba tính chất của một môi trường đạt yêu cầu - tác nhân học được, môi trường phân biệt được các hướng tiếp cận, và không tồn tại lỗi tắt thoái hóa - đồng thời đo đặc tính của từng môi trường qua bốn tiêu chí định lượng: mức năng lực đạt được, tốc độ hội tụ, độ ổn định huấn luyện và độ dài lượt chơi.
- **Xây dựng ứng dụng trình diễn để quan sát hành vi.** Biểu đồ phần thưởng không nói cho ta biết tác nhân đang thực sự làm gì, trong khi rất nhiều lỗi đặc tả chỉ chịu lộ ra khi nhìn hành vi thật. Đề tài do đó dựng thêm một ứng dụng cho phép nạp trực tiếp các mô hình đã huấn luyện dưới định dạng ONNX, tự điều khiển hoặc chơi cùng tác nhân để đối chiếu độ khó, và tra cứu số liệu huấn luyện ngay trên cùng giao diện.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu chính là **môi trường huấn luyện học tăng cường trong game ba chiều**: nó có cấu trúc ra sao, khi xây dựng thì phải đưa ra những quyết định thiết kế nào, và làm thế nào để kiểm chứng xem nó đã được đặc tả đúng hay chưa. Đối tượng ấy được cụ thể hóa thành ba môi trường dựng trên nền tảng Unity: môi trường đối kháng Football Table, môi trường hợp tác Capture The Flag và môi trường điều hướng Cross The Road.

Đối tượng thứ hai là bộ công cụ dùng để xây dựng và vận hành môi trường: nền tảng Unity ML-Agents, cơ chế trainer mở rộng, giao diện lập trình cấp thấp bằng Python, cùng những công cụ do chính đề tài viết ra để nhân khu vực huấn luyện, dựng bản build không đồ họa và đóng gói môi trường thành gói phân phối.

Năm thuật toán - PPO (tối ưu chính sách vùng lân cận), SAC (actor-critic cực đại hóa entropy), MA-POCA (gán công lao cho đa tác nhân), MAPPO (mở rộng PPO cho đa tác nhân với chia sẻ tham số) và DQN (dựa trên giá trị với bộ đệm phát lại) - tham gia đề tài với tư cách *công cụ đo*. Chúng là những phép thử độc lập dùng để dò đặc tính của môi trường. Đề tài không đặt mục tiêu cải tiến hay đề xuất thuật

toán mới.

Về nội dung, đề tài xoay quanh thiết kế môi trường ba chiều, xây dựng hệ thống quan sát, hành động, phần thưởng, quy trình huấn luyện và cách đóng gói môi trường để dùng lại. Những khía cạnh như tối ưu hóa đồ họa, âm thanh hay hoàn thiện một sản phẩm game thương mại đều nằm ngoài phạm vi. Đề tài cũng không nhắm tới hiệu năng cao nhất có thể trên từng môi trường: ngân sách bước chỉ được chọn ở mức đủ để các đường cong bộc lộ đặc tính của bài toán, chứ không nhằm ép mỗi thuật toán tới giới hạn của nó. Ba môi trường, như đã nói, được chọn để phủ ba dạng bài toán học tăng cường điển hình là đối kháng một đôi một, hợp tác đa tác nhân và điều hướng đơn tác nhân.

Về thuật toán, năm cái tên nói trên được chọn dựa trên ba yếu tố: mức độ phổ biến trong nghiên cứu, tính sẵn có trong hệ sinh thái ML-Agents, và sự đa dạng về mặt kiến trúc, khi chúng đại diện lần lượt cho on-policy đơn tác nhân, actor-critic off-policy, đa tác nhân dựa trên attention, đa tác nhân dựa trên chia sẻ tham số và phương pháp dựa trên giá trị. Riêng DQN đòi hỏi không gian hành động rời rạc, nên môi trường Football Table vốn dùng hành động liên tục phải được dựng lại thành một bản riêng có hành động rời rạc dành cho lần chạy này. Toàn bộ thực nghiệm chạy trên một cấu hình máy tính cố định để bảo đảm công bằng khi so sánh; chi tiết cấu hình nằm ở Mục 3.8.1.

1.4 Tổng quan tài liệu và khoảng trống nghiên cứu

Học tăng cường trong game đã đi qua nhiều giai đoạn phát triển. Mnih và cộng sự [14] đề xuất DQN, lần đầu chứng minh học tăng cường sâu có thể đạt trình độ con người ở các trò chơi điện tử cổ điển mà chỉ dựa vào dữ liệu điểm ảnh thô; công trình này mở ra hướng ứng dụng học tăng cường trực tiếp vào môi trường game, và cũng là lý do DQN được giữ lại trong đề tài như một mốc so sánh lịch sử cho nhóm phương pháp dựa trên giá trị. Nối tiếp thành tựu đó, Silver và cộng sự [23, 24] tạo bước đột phá với các hệ thống chơi cờ vây ở trình độ kiện tướng thế giới, nhờ kết hợp học tăng cường với mạng nơ-ron sâu và tìm kiếm cây.

Ở quy mô lớn hơn, nơi môi trường phức tạp và có nhiều tác nhân cùng hoạt động, các hệ thống chơi trò chơi chiến thuật thời gian thực [16, 26] cho thấy học

tăng cường vẫn xử lý được những bài toán với không gian trạng thái khổng lồ, nhiều tác nhân hoạt động đồng thời và đòi hỏi chiến lược dài hạn. Về phía thuật toán, Schulman và cộng sự [22] đề xuất PPO, một phương pháp on-policy chịu đánh đổi hiệu quả mẫu để lấy sự ổn định và tính dễ triển khai. Haarnoja và cộng sự [7] giới thiệu SAC, dùng cơ chế điều hòa entropy để cân bằng giữa khám phá và khai thác, đặc biệt hiệu quả khi không gian hành động liên tục. Sang địa hạt đa tác nhân, Cohen và cộng sự [4] phát triển MA-POCA với cơ chế phê bình tập trung nhằm gỡ bài toán gán công lao trong môi trường hợp tác; còn Yu và cộng sự [29] chứng minh MAPPO, vốn chỉ là một mở rộng khá đơn giản của PPO sang nhiều tác nhân với chia sẻ tham số, lại đạt hiệu năng cạnh tranh và thậm chí vượt trội trên nhiều bộ benchmark hợp tác chuẩn.

Về mặt công cụ, Juliani và cộng sự [9] giới thiệu Unity ML-Agents, một hệ sinh thái mã nguồn mở tích hợp sẵn khả năng xây dựng môi trường ba chiều với mô phỏng vật lý chân thực và hỗ trợ huấn luyện tác nhân trực tiếp; công cụ này hiện được dùng rộng rãi trong cả nghiên cứu lẫn phát triển game.

Có một điểm chung giữa toàn bộ các công trình vừa kể: chúng *tiêu thụ* môi trường chứ không bàn tới việc tạo ra môi trường. Bộ benchmark đã được ai đó đặc tả sẵn - lưới hai chiều, vector trạng thái rút gọn, động lực học đơn giản - và đóng góp khoa học nằm hết ở phía thuật toán. Hệ quả là tài liệu về việc *thiết kế* một môi trường học tăng cường mỏng đến bất ngờ nếu đem so với tài liệu về thuật toán. Nên đưa đại lượng nào vào quan sát? Phần thưởng dày hay thưa? Phạt thời gian bao nhiêu là đủ? Làm sao biết một hàm phần thưởng đang vô tình trừng phạt chính hành vi ta muốn dạy? Những câu hỏi kiểu này hầu như chỉ được truyền lại dưới dạng kinh nghiệm rời rạc trong các diễn đàn kỹ thuật.

Khoảng trống mà đề tài nhắm tới nằm đúng chỗ đó, và nó có hai mặt. Mặt thứ nhất là *tri thức thiết kế*. Quá trình đi từ một ý tưởng trò chơi tới một đặc tả mà tác nhân học được rất hiếm khi được ghi lại đầy đủ, nhất là các phương án đã thất bại, vốn lại là phần dạy được nhiều nhất. Mặt thứ hai là *hiện vật dùng lại được*. Môi trường ba chiều đi kèm tài liệu, cấu hình huấn luyện và công cụ cài đặt, ở dạng một nhóm phát triển khác lấy về là dùng được ngay, gần như không tồn tại; thành ra mỗi dự án lại phải dựng lại từ con số không. Đồ án này lấp cả hai mặt: ghi lại

toàn bộ quá trình thiết kế ba môi trường thuộc ba dạng bài toán khác nhau, rồi đóng gói chúng thành sản phẩm phân phối được. Còn việc đặt PPO, SAC, MA-POCA, MAPPO và DQN lên cùng một bàn cân - điều gần như không làm được nếu chỉ đối chiếu các tài liệu đã công bố - vừa là phương tiện kiểm chứng ba môi trường đó, vừa là một kết quả phụ có ích cho người làm game.

1.5 Phương pháp nghiên cứu

Đề tài dùng ba nhóm phương pháp bổ trợ cho nhau. Nhóm thứ nhất là nghiên cứu lý thuyết: tổng hợp và phân tích tài liệu khoa học về học tăng cường, học tăng cường sâu và năm thuật toán PPO, SAC, MA-POCA, MAPPO, DQN, dựa vào các bài báo gốc cùng tài liệu kỹ thuật của Unity ML-Agents. Đây là cơ sở để thiết kế môi trường và chọn tham số huấn luyện.

Nhóm thứ hai là thực nghiệm. Ba môi trường game ba chiều được xây dựng trong Unity với đầy đủ hệ thống vật lý, quan sát, hành động và phần thưởng; sau đó từng thuật toán được triển khai và huấn luyện trên mỗi môi trường, giữ nguyên điều kiện phần cứng và số bước huấn luyện. Dữ liệu huấn luyện trong quá trình này được thu thập và trực quan hóa theo thời gian thực bằng công cụ TensorBoard.

Nhóm cuối là so sánh và đánh giá. Hiệu năng của các thuật toán được đối chiếu qua bốn tiêu chí định lượng - mức năng lực đạt được, tốc độ hội tụ, độ ổn định huấn luyện và độ dài lượt chơi - kết hợp thêm phân tích định tính về mức độ phù hợp giữa từng thuật toán và từng dạng môi trường.

1.6 Ý nghĩa khoa học và thực tiễn

Xét về mặt khoa học, đề tài đề xuất và hiện thực hóa một quy trình xây dựng môi trường game ba chiều phục vụ huấn luyện học tăng cường trên nền tảng Unity, kèm theo cách nghiệm thu môi trường bằng thực nghiệm đa thuật toán dựa trên ba tiêu chí học được - phân biệt được - không có lỗi tắt. Quy trình ấy áp dụng được cho bất kỳ môi trường nào khác, không riêng gì ba môi trường của đề tài. Phần ghi chép về những phương án đặc tả đã thất bại cũng mang giá trị riêng: đây là loại tri thức thường bị bỏ lại ngoài các công bố khoa học, dù nó chiếm phần lớn thời gian thực hiện. Bên cạnh đó, đồ án trình bày chi tiết quy trình bổ sung thuật toán mới vào

ML-Agents dưới dạng trainer mở rộng, cùng cách nối một thuật toán off-policy với Unity thông qua giao diện lập trình cấp thấp bằng Python. Đây là một hướng đi có thể tái sử dụng cho những nghiên cứu cần thuật toán nằm ngoài bộ tích hợp sẵn của Unity.

Xét về mặt thực tiễn, ba môi trường ba chiều của đề tài được đóng gói ở dạng cài thẳng vào một dự án Unity bất kỳ, nên dùng ngay được làm nền tảng thử nghiệm trí tuệ nhân tạo mà khỏi phải dựng lại từ đầu - vốn là phần tốn kém nhất của một dự án học tăng cường. Quy trình thiết kế hàm phần thưởng và không gian quan sát trong môi trường ba chiều được trình bày ở đây cũng có giá trị tham khảo cho các nhóm phát triển game muốn tích hợp trí tuệ nhân tạo. Kết quả so sánh năm thuật toán, dù trong đề tài chỉ đóng vai trò kiểm chứng, vẫn giúp nhà phát triển có cơ sở chọn thuật toán hợp với từng cơ chế game mà không phải làm lại toàn bộ thực nghiệm.

1.7 Bố cục đồ án

Ngoài phần Mở đầu và Kết luận, nội dung chính của đồ án chia thành bốn chương. Chương 2 lo phần cơ sở lý thuyết: tổng quan trí tuệ nhân tạo trong game, nền tảng lý thuyết của học tăng cường với quá trình quyết định Markov, chính sách và hàm giá trị, các kiến trúc học tăng cường sâu, rồi đặc điểm của năm thuật toán PPO, SAC, MA-POCA, MAPPO và DQN. Chương này cũng giới thiệu nền tảng Unity ML-Agents cùng những giới hạn tích hợp liên quan tới việc triển khai thuật toán tùy chỉnh.

Chương 3 là chương trọng tâm của đồ án, đi vào chi tiết quá trình thiết kế ba môi trường game ba chiều trong Unity: không gian quan sát, không gian hành động và hàm phần thưởng của từng môi trường. Chương này cũng mô tả cách đưa các thuật toán nằm ngoài bộ huấn luyện tích hợp sẵn của ML-Agents vào thực nghiệm, cùng thiết lập thực nghiệm và các tiêu chí đánh giá được sử dụng. Sang Chương 4, mười lăm lần huấn luyện được dùng để nghiệm thu ba môi trường đó: kết quả của từng thuật toán trên từng môi trường được trình bày qua biểu đồ và bảng số liệu, rồi đối chiếu với nhau để xác nhận môi trường học được, phân biệt được và không có lỗi tắt; từ đó rút ra nhận xét về mức độ phù hợp giữa thuật toán và dạng bài toán.

Chương 5 khép lại phần nội dung chính với quá trình xây dựng ứng dụng trình diễn tương tác: kiến trúc hệ thống lựa chọn chế độ chơi, giao diện menu, cơ chế nạp mô hình ONNX tại thời gian chạy, đường ống đưa số liệu huấn luyện từ TensorBoard vào giao diện trong game, cách ánh xạ ba chế độ chơi (người chơi, tác nhân tự chơi, người chơi cùng tác nhân) vào từng môi trường, cùng các vấn đề kỹ thuật phát sinh và kết quả kiểm thử ứng dụng.

2. CƠ SỞ LÝ THUYẾT

Chương này dựng nền lý thuyết cho toàn bộ đồ án. Mạch trình bày đi từ bối cảnh lịch sử của trí tuệ nhân tạo trong game, qua bộ khái niệm hình thức của học tăng cường, rồi tới năm thuật toán mà đề tài đem ra so sánh. Phần cuối chương thuộc về Unity ML-Agents, trong đó dung lượng lớn nhất được dành cho những giới hạn tích hợp của nền tảng này. Lý do là chính các giới hạn ấy - chứ không phải một lựa chọn học thuật nào - đã quy định cách đề tài phải bố trí thực nghiệm ở Chương 3.

2.1 Trí tuệ nhân tạo trong game

Trí tuệ nhân tạo trong game lớn lên cùng chính ngành công nghiệp game, tính từ những thập niên 1970-1980. Nhưng nó khác AI trong nghiên cứu học thuật ở một điểm căn bản: nó không nhất thiết phải tối ưu hay thông minh tuyệt đối. Mục tiêu cốt lõi chỉ là tạo ra những đối thủ hoặc nhân vật phi người chơi (NPC) có hành vi đủ thuyết phục, đủ để người chơi thấy thú vị và thấy có thử thách [12]. Nhìn theo chiều thời gian, lĩnh vực này đã đi qua hai giai đoạn khá rõ: giai đoạn lập trình hành vi cứng nhắc dựa trên quy tắc, rồi tới giai đoạn của các phương pháp học máy biết thích nghi và tự cải thiện.

2.1.1 AI truyền thống

Ở giai đoạn đầu, AI game gần như hoàn toàn dựa vào các hệ thống quy tắc cứng (rule-based systems). Lập trình viên định nghĩa sẵn toàn bộ hành vi của NPC dưới dạng tập hợp điều kiện kèm phản ứng tương ứng. Ưu điểm là dễ kiểm soát, dễ dự đoán; nhược điểm lộ ra ngay khi NPC gặp một tình huống nằm ngoài dự kiến.

Máy trạng thái hữu hạn (Finite State Machine - FSM) ra đời để khắc phục hạn

chế đó, và được ứng dụng rộng rãi suốt những thập niên 1990-2000. FSM mô hình hóa hành vi NPC bằng một tập hữu hạn các trạng thái - tuần tra, truy đuổi, tấn công, rút lui - cùng các điều kiện chuyển giữa chúng. Đơn giản, rẻ về mặt tính toán, nên đến tận bây giờ FSM vẫn phổ biến trong nhiều thể loại game [12]. Vấn đề chỉ xuất hiện khi số trạng thái và số điều kiện chuyển tiếp phình lên: hệ thống nhanh chóng trở nên rối rắm và khó bảo trì, hiện tượng vẫn hay được gọi là “state explosion”.

Với những bài toán lập kế hoạch hành động phức tạp hơn, Goal-Oriented Action Planning (GOAP) là câu trả lời tiếp theo; tựa game F.E.A.R. (2005) đã đưa kỹ thuật này tới đông đảo người biết [12]. GOAP không định nghĩa cứng các chuyển trạng thái mà để NPC tự lập kế hoạch hành động dựa trên mục tiêu hiện tại và trạng thái thế giới, nhờ đó hành vi linh hoạt và tự nhiên hơn hẳn. Điểm yếu thì vẫn còn nguyên: toàn bộ tri thức trong GOAP đều do con người lập trình thủ công, nên game càng phức tạp thì chi phí xây dựng và mở rộng hệ thống càng đội lên.

2.1.2 AI học máy

Học máy bùng nổ trong thập niên 2010 và mở ra một hướng đi hoàn toàn khác cho AI trong game. Không còn phải lập trình hành vi thủ công nữa; các kỹ thuật học máy để agent tự học từ dữ liệu hoặc từ chính tương tác với môi trường. Trong số đó, học tăng cường (Reinforcement Learning - RL) hợp với bản chất của game hơn cả: agent học cách hành động qua thử và sai, nhận thưởng hoặc phạt từ môi trường, rồi dần tối ưu chiến lược của mình để đạt điểm số cao nhất [25].

Bước ngoặt đến vào năm 2013, khi DeepMind công bố thuật toán DQN (Deep Q-Network) [14]. Đây là lần đầu tiên một agent RL học chơi thành thạo 49 trò chơi Atari khác nhau chỉ từ dữ liệu pixel thô, không hề có tri thức nào được lập trình sẵn. Thành tựu này khai sinh Deep Reinforcement Learning, tức là kết hợp sức mạnh biểu diễn của mạng nơ-ron sâu với khả năng học từ tương tác của RL.

Những năm sau đó là một chuỗi thành tựu liên tiếp. AlphaGo rồi AlphaGo Zero [23, 24] chinh phục cờ vây ở trình độ kiện tướng thế giới. OpenAI Five [16] đánh bại các đội chuyên nghiệp trong Dota 2. AlphaStar [26] đạt danh hiệu Grandmaster trong StarCraft II. Giá trị của những kết quả này không dừng ở học thuật: chúng mở ra khả năng dùng RL ngay trong phát triển game thương mại, từ việc tạo NPC

thông minh cho tới sinh nội dung tự động và hỗ trợ kiểm thử game [28].

2.2 Nền tảng Reinforcement Learning

Học tăng cường (Reinforcement Learning - RL) là nhánh học máy trong đó một tác nhân, hay agent, học cách hành động bằng tương tác trực tiếp với môi trường. Điểm phân biệt nó với học có giám sát nằm ở dữ liệu: RL không cần nhãn cho trước. Agent tự khám phá không gian hành động, nhận phản hồi dưới dạng phần thưởng hoặc hình phạt, và dần cải thiện chiến lược của mình [25]. Chính đặc tính này khiến RL hợp với các bài toán game, nơi hành vi tối ưu rất khó định nghĩa tường minh nhưng lại dễ đánh giá qua kết quả.

2.2.1 Các khái niệm cơ bản

Vòng lặp tương tác trong RL dựng trên năm khái niệm nền tảng. Trước hết là agent, thực thể ra quyết định; trong bối cảnh game, nó có thể là nhân vật người chơi, NPC hay một robot mô phỏng.

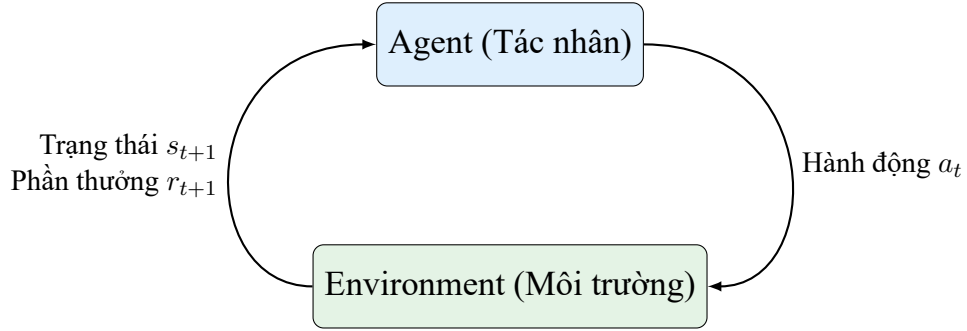
Environment, tức môi trường, là thế giới mà agent tương tác cùng: toàn bộ đối tượng, luật chơi và cơ chế phản hồi đều thuộc về nó. Ở đề tài này, môi trường chính là các scene 3D dựng trong Unity.

State, tức trạng thái $s \in \mathcal{S}$, là biểu diễn của môi trường tại một thời điểm mà agent quan sát được. Với môi trường 3D, state thường gồm vị trí và vận tốc của các đối tượng, khoảng cách tới mục tiêu cùng những thông tin không gian liên quan. Đi kèm là action, hành động $a \in \mathcal{A}$ - tập lựa chọn mà agent có thể thực hiện ở mỗi bước. Hành động có thể rời rạc, chẳng hạn di chuyển lên, xuống, trái, phải; cũng có thể liên tục, chẳng hạn lực đẩy theo trục x, y, z .

Reward, phần thưởng $r_t \in \mathbb{R}$, là tín hiệu môi trường trả về sau mỗi hành động. Đây là thước đo duy nhất để agent biết quyết định của mình tốt hay dở. Thiết kế được một hàm phần thưởng hợp lý vì thế vừa quan trọng vừa là khâu khó nhất khi xây dựng hệ thống RL: hàm phần thưởng đặt sai sẽ dẫn tới hành vi ngoài ý muốn ngay cả khi thuật toán hoạt động hoàn toàn đúng về mặt lý thuyết [25].

Ở mỗi bước thời gian t , agent quan sát trạng thái s_t , thực hiện hành động a_t , nhận phần thưởng r_t rồi chuyển sang trạng thái mới s_{t+1} . Chu trình ấy lặp đi lặp

lại cho tới khi episode kết thúc - agent hoàn thành nhiệm vụ, thất bại, hoặc vượt quá số bước tối đa - và để lại một quỹ đạo tương tác $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$. Hình 2.1 minh họa toàn bộ vòng lặp này. Có thể xem đây là sơ đồ khái quát nhất của mọi hệ thống học tăng cường: agent nhìn trạng thái hiện tại của môi trường, chọn một hành động theo chính sách của mình, môi trường đáp lại bằng một trạng thái mới kèm một phần thưởng, và cứ thế agent dần cải thiện chính sách.



Hình 2.1: Vòng lặp tương tác giữa agent và môi trường trong học tăng cường

2.2.2 Markov Decision Process

Phần lớn bài toán RL được hình thức hóa thành Quá trình Quyết định Markov (Markov Decision Process - MDP). Đây là khung toán học cho phép mô hình hóa chặt chẽ những bài toán ra quyết định tuần tự trong môi trường ngẫu nhiên [18]. Một MDP được định nghĩa bởi bộ năm thành phần $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, trong đó $\mathcal{S}$ là không gian trạng thái, $\mathcal{A}$ là không gian hành động, $\mathcal{P}$ là hàm chuyển trạng thái, $\mathcal{R}$ là hàm phần thưởng và $\gamma \in [0, 1)$ là hệ số chiết khấu.

Hàm chuyển trạng thái $\mathcal{P}(s' | s, a)$ cho biết xác suất đi từ trạng thái s sang trạng thái s' khi thực hiện hành động a . Tính chất Markov, hay tính không nhớ, đòi hỏi trạng thái tiếp theo s_{t+1} chỉ phụ thuộc vào trạng thái hiện tại s_t cùng hành động a_t , hoàn toàn độc lập với lịch sử trước đó:

$$\mathcal{P}(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots) = \mathcal{P}(s_{t+1} | s_t, a_t) \quad (2.1)$$

Giả thiết Markov là thứ giữ cho bài toán còn giải được. Nhờ nó, agent chỉ cần nhìn trạng thái hiện tại là ra được quyết định tối ưu, thay vì phải lưu và xử lý toàn bộ lịch sử tương tác. Trên thực tế, khá nhiều môi trường game thỏa mãn gần đúng

tính chất này, miễn là state được thiết kế đủ phong phú để mang hết thông tin cần thiết.

Hệ số chiết khấu γ nói lên agent coi trọng phần thưởng tương lai đến mức nào so với phần thưởng hiện tại. γ càng gần 1, agent càng thiên về kế hoạch dài hạn; γ nhỏ thì agent chỉ chăm chăm vào phần thưởng tức thời. Tổng phần thưởng tích lũy có chiết khấu mà agent muốn tối đa hóa được định nghĩa như sau:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.2)$$

2.2.3 Policy và Value Function

Policy, tức chính sách π , là hàm ánh xạ từ trạng thái sang hành động; nó chính là chiến lược hành động của agent. Policy có hai dạng: tất định $\pi : \mathcal{S} \rightarrow \mathcal{A}$, hoặc ngẫu nhiên $\pi(a | s) = P(A_t = a | S_t = s)$ cho biết xác suất chọn hành động a khi đang ở trạng thái s . Cái mà RL đi tìm là policy tối ưu π^* , tức policy cho tổng phần thưởng tích lũy kỳ vọng lớn nhất.

Value Function, hàm giá trị, đo xem một trạng thái hay một cặp trạng thái-hành động tốt đến đâu khi agent đi theo một policy nhất định. Hàm giá trị trạng thái $V^\pi(s)$ là tổng phần thưởng tích lũy kỳ vọng nếu bắt đầu từ trạng thái s và tuân theo policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | S_t = s \right] \quad (2.3)$$

Hàm giá trị hành động $Q^\pi(s, a)$, hay Q-function, mở rộng khái niệm trên: nó đánh giá giá trị của việc thực hiện hành động a tại trạng thái s , sau đó mới tuân theo policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t | S_t = s, A_t = a] \quad (2.4)$$

Mối quan hệ giữa V^π và Q^π được thể hiện qua phương trình Bellman [2], nền tảng lý thuyết của hầu hết các thuật toán RL hiện đại:

$$V^\pi(s) = \sum_a \pi(a | s) Q^\pi(s, a) \quad (2.5)$$

Policy tối ưu π^* và hàm giá trị tối ưu $V^*(s)$, $Q^*(s, a)$ thỏa mãn phương trình tối ưu Bellman:

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid S_t = s, A_t = a \right] \quad (2.6)$$

Trên thực tế, với không gian trạng thái lớn và liên tục như ở các môi trường game 3D, chẳng cách nào biểu diễn chính xác V^π hay Q^π dưới dạng bảng tra cứu. Đó chính là động lực khai sinh Deep Reinforcement Learning, hướng tiếp cận xấp xỉ hàm giá trị và policy bằng mạng nơ-ron sâu; Mục 2.3 trình bày chi tiết.

2.3 Học tăng cường sâu

Học tăng cường sâu (Deep Reinforcement Learning) ghép học tăng cường với mạng nơ-ron sâu (deep neural network). Nó ra đời để gỡ đúng cái nút thắt của RL truyền thống: những bài toán có không gian trạng thái và hành động vừa lớn vừa liên tục. Thay vì lưu giá trị của từng cặp trạng thái-hành động vào bảng tra cứu, Deep RL dùng mạng nơ-ron xấp xỉ hàm giá trị hoặc policy, nhờ đó tổng quát hóa được sang cả những trạng thái chưa từng gặp khi huấn luyện [10]. Đây là đặc tính then chốt giúp Deep RL xử lý nổi các môi trường game 3D phức tạp như trong đề tài này.

2.3.1 Vai trò của mạng Neural Network trong RL

RL truyền thống biểu diễn hàm giá trị $Q(s, a)$ hoặc policy $\pi(a \mid s)$ dưới dạng bảng tra cứu (lookup table). Muốn ước lượng chính xác thì mỗi cặp (s, a) phải được thăm viếng đủ nhiều lần, nên cách này chỉ khả thi khi không gian trạng thái hữu hạn và nhỏ. Môi trường game 3D thì ngược lại: không gian trạng thái liên tục và số chiều rất lớn. Lấy ngay môi trường Football Table của đề tài làm ví dụ - vector quan sát của agent chứa vị trí và vận tốc của bóng cùng các thanh gạt trong không gian ba chiều. Bảng tra cứu không có cửa nào ở đây.

Lời giải là mạng nơ-ron sâu, đóng vai trò bộ xấp xỉ hàm tổng quát (universal function approximator). Mạng không lưu giá trị cho từng trạng thái mà học một ánh xạ tham số hóa: $f_\theta : \mathcal{S} \rightarrow \mathbb{R}$ nếu là value function, hoặc $f_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ nếu là policy, với θ là tập tham số được tối ưu hóa qua huấn luyện. Khả năng tổng quát hóa cho phép mạng ước lượng giá trị hay đưa ra hành động cho những trạng thái

chưa từng gặp, dựa trên mức tương đồng với các trạng thái đã học [25].

Kiến trúc mạng nào thì tùy dạng dữ liệu quan sát. Dữ liệu là vector số thực - vị trí, vận tốc, khoảng cách - thì mạng fully-connected nhiều lớp (MLP) vừa phổ biến vừa hiệu quả. Dữ liệu dạng ảnh hoặc dữ liệu không gian thì mạng tích chập (CNN) thường được ưu tiên hơn. Unity ML-Agents hỗ trợ cả hai, và cho phép chỉnh số lớp ẩn lẫn số đơn vị mỗi lớp ngay trong tệp cấu hình YAML.

2.3.2 Phân loại: *Policy-Based, Value-Based, Actor-Critic*

Người ta phân loại các thuật toán Deep RL hiện đại thành ba nhóm, căn cứ vào thứ mà mạng nơ-ron học để xấp xỉ. Mỗi nhóm mạnh yếu khác nhau và hợp với những dạng bài toán khác nhau.

Ở nhóm Value-Based, mạng nơ-ron học xấp xỉ hàm giá trị $Q(s, a)$, còn policy tối ưu thì suy ra gián tiếp bằng cách chọn hành động có Q cao nhất. DQN [14] là đại diện tiêu biểu. Nhóm này chạy tốt trên không gian hành động rời rạc, nhưng bí khi hành động liên tục, đơn giản vì không thể tối ưu $\max_a Q(s, a)$ một cách hiệu quả khi $\mathcal{A}$ liên tục.

Nhóm Policy-Based đi thẳng vào policy: tham số hóa $\pi_\theta(a | s)$ rồi tối ưu nó bằng cách tính gradient của hàm mục tiêu theo θ , tức phương pháp policy gradient [27]. Cả không gian hành động rời rạc lẫn liên tục đều xử lý được. Cái giá là phương sai trong ước lượng gradient thường cao, kéo theo huấn luyện kém ổn định. PPO [22] thuộc nhóm này và khắc phục điểm yếu ấy bằng cơ chế clipping, giới hạn độ lớn mỗi bước cập nhật để ổn định hóa quá trình huấn luyện.

Nhóm Actor-Critic gộp ưu điểm của hai nhóm trên, bằng cách nuôi song song hai mạng nơ-ron: actor học policy $\pi_\theta(a | s)$, critic học hàm giá trị $V_\phi(s)$ hoặc $Q_\phi(s, a)$. Critic đánh giá chất lượng hành động của actor và nhờ vậy cấp một tín hiệu học ổn định hơn so với việc chỉ bám vào phần thưởng thực tế [13]. SAC [7] và MA-POCA [4] đều nằm ở nhóm này. SAC thêm entropy regularization để đẩy agent đi khám phá; MA-POCA thì kéo kiến trúc critic sang bài toán đa tác nhân, dùng cơ chế centralized critic để mỗi agent học được dựa trên thông tin toàn cục trong lúc huấn luyện.

Bảng 2.1 tóm tắt vị trí của năm thuật toán được sử dụng trong đề tài theo cách

phân loại trên.

Bảng 2.1: Phân loại các thuật toán sử dụng trong đề tài

Thuật toán	Nhóm	On/Off-policy	Phù hợp với
PPO	Policy-Based	On-policy	Đối kháng, điều hướng
SAC	Actor-Critic	Off-policy	Hành động liên tục
MA-POCA	Actor-Critic	On-policy	Hợp tác đa tác nhân
MAPPO	Policy-Based	On-policy	Hợp tác, tác nhân đồng nhất
DQN	Value-Based	Off-policy	Hành động rời rạc

2.3.3 Phần thưởng tò mò nội tại

Trong các bài toán học tăng cường thực tế, tín hiệu phần thưởng ngoại sinh thường rất thưa hoặc đến rất muộn; những môi trường game 3D phức tạp như Capture The Flag hay Cross The Road là ví dụ điển hình. Nếu chỉ trông vào phần thưởng từ môi trường, tác nhân rất dễ tiêu hết giai đoạn đầu vào khám phá ngẫu nhiên mà chẳng nhận về tín hiệu hữu ích nào, và rốt cuộc không học nổi một chính sách hiệu quả. Cơ chế phần thưởng tò mò nội tại (Intrinsic Curiosity) được đề xuất chính để gỡ chỗ này, bằng cách thúc tác nhân chủ động đi tìm những trạng thái mới mẻ của môi trường.

Đồ án tích hợp cơ chế tò mò qua mô hình Intrinsic Curiosity Module (ICM) của Pathak và cộng sự [17]. Ý tưởng cốt lõi nằm ở việc lấy sai số dự đoán (prediction error) làm tín hiệu phần thưởng nội tại: tác nhân được thưởng cao khi nó đi tới một trạng thái mà mô hình nội tại của chính nó không dự đoán nổi.

ICM gồm hai mạng nơ-ron con chạy song song, mô hình ngược (Inverse Model) và mô hình thuận (Forward Model). Có một cạm bẫy cần tránh ở đây: tác nhân có thể bị hút vào những yếu tố ngẫu nhiên vốn không thể dự đoán được trong môi trường, tức hiện tượng tivi nhiễu hạt (noisy TV problem). Vì vậy ICM không dự đoán trạng thái thô s_t mà dự đoán đặc trưng ẩn $\phi(s_t)$, thứ được trích xuất qua một mạng mã hóa (Encoder).

Mô hình ngược (Inverse Model): Mạng này nhận vào đặc trưng của trạng thái hiện tại $\phi(s_t)$ cùng đặc trưng của trạng thái tiếp theo $\phi(s_{t+1})$, rồi dự đoán hành

động $\hat{a}_t$ nào đã gây ra bước chuyển tiếp ấy. Hàm mất mát của nó là:

$$L_{\text{inv}} = \mathbb{E} [-\log P(\hat{a}_t = a_t \mid \phi(s_t), \phi(s_{t+1}))] \quad (2.7)$$

Huấn luyện Inverse Model có tác dụng ép mạng mã hóa ϕ chỉ giữ lại phần thông tin trạng thái liên quan trực tiếp tới hành động của tác nhân, gạt bỏ mọi chi tiết nhiễu vô ích.

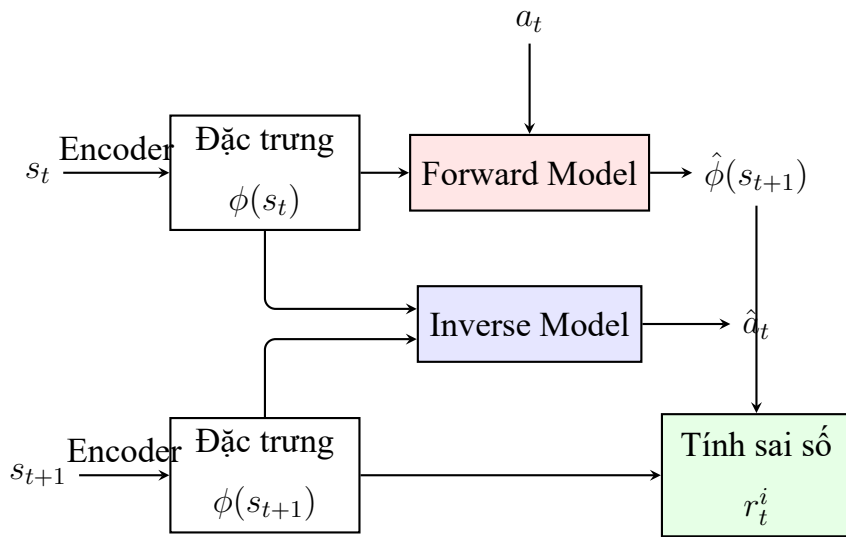
Mô hình thuận (Forward Model): Mạng này nhận vào đặc trưng trạng thái hiện tại $\phi(s_t)$ và hành động a_t , rồi dự đoán đặc trưng của trạng thái tiếp theo $\hat{\phi}(s_{t+1})$. Hàm mất mát ở đây thường là sai số bình phương trung bình:

$$L_{\text{fwd}} = \frac{1}{2} \left\| \hat{\phi}(s_{t+1}) - \phi(s_{t+1}) \right\|_2^2 \quad (2.8)$$

Phần thưởng tò mò: Sai số của Forward Model đo đúng mức độ “ngạc nhiên” của tác nhân, nên phần thưởng tò mò r_t^i được đặt tỷ lệ thuận với sai số ấy:

$$r_t^i = \eta \frac{1}{2} \left\| \hat{\phi}(s_{t+1}) - \phi(s_{t+1}) \right\|_2^2 \quad (2.9)$$

với η là hệ số tỷ lệ. Tổng phần thưởng tác nhân nhận ở mỗi bước là kết hợp có trọng số giữa phần thưởng ngoại sinh và phần thưởng nội tại: $r_t = r_t^e + \beta r_t^i$. Các thực nghiệm của đề tài thường đặt β ở mức 0,02, đủ nhỏ để tác nhân không quên mất mục tiêu chính của trò chơi.



Hình 2.2: Sơ đồ kiến trúc Intrinsic Curiosity Module (ICM)

2.4 Các thuật toán sử dụng trong đề tài

Năm thuật toán mà đề tài sử dụng đại diện cho năm hướng tiếp cận khác nhau. PPO là policy-based on-policy đơn tác nhân. SAC là actor-critic off-policy. MA-POCA chuyên cho bài toán đa tác nhân, dựa trên cơ chế attention. MAPPO cũng là đa tác nhân nhưng đi đường chia sẻ tham số, mở rộng trực tiếp từ PPO. DQN đại diện cho nhóm phương pháp dựa trên giá trị. Phần dưới đây đi qua nguyên lý hoạt động và đặc điểm của từng cái, làm cơ sở cho việc lựa chọn và phân tích kết quả thực nghiệm ở các chương sau.

2.4.1 PPO

PPO (Proximal Policy Optimization) được Schulman và cộng sự đề xuất năm 2017 [22], như một cải tiến của TRPO (Trust Region Policy Optimization) [21]. Mục đích là giữ lại tính ổn định khi cập nhật policy nhưng đơn giản hơn hẳn về mặt triển khai. Hiện PPO nằm trong nhóm thuật toán RL phổ biến nhất, cả trong nghiên cứu lẫn ứng dụng thực tế, và là thuật toán on-policy mặc định của Unity ML-Agents.

Là thuật toán on-policy, PPO thu thập dữ liệu huấn luyện trực tiếp từ policy hiện tại rồi vứt bỏ sau mỗi lần cập nhật. Ý tưởng cốt lõi của nó là chặn độ lớn thay đổi của policy trong từng bước cập nhật. Mục đích là tránh tình trạng policy nhảy quá đột ngột rồi suy giảm hiệu năng không cách nào phục hồi, vốn là vấn đề rất hay gặp ở các thuật toán policy gradient đơn giản.

Cơ chế giúp PPO làm được điều đó là hàm mục tiêu clipped surrogate:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (2.10)$$

trong đó $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ là tỷ lệ xác suất giữa policy mới và policy cũ, $\hat{A}_t$ là ước lượng advantage - tức mức độ một hành động tốt hơn hay kém hơn kỳ vọng trung bình - còn ε là siêu tham số clipping, thường đặt trong khoảng $[0.1, 0.2]$. Toán tử clip ép $r_t(\theta)$ nằm trong $[1 - \varepsilon, 1 + \varepsilon]$, nhờ đó policy mới không lệch quá xa policy cũ chỉ trong một bước cập nhật. Còn hàm min bảo đảm mục tiêu luôn là cận dưới thận trọng (pessimistic lower bound) của mục tiêu chưa clipped.

Giá trị advantage $\hat{A}_t$ trong biểu thức trên thường được ước lượng bằng

Generalized Advantage Estimation (GAE), phương pháp dung hòa độ lệch với phương sai của ước lượng thông qua tham số λ :

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (2.11)$$

trong đó $\lambda \in [0, 1]$ là tham số cân bằng giữa bias và phương sai của ước lượng advantage, $V(s_t)$ là giá trị trạng thái được ước lượng bởi mạng critic.

Mỗi iteration, PPO dùng policy hiện tại để gom một batch dữ liệu từ môi trường, rồi chạy nhiều lần cập nhật gradient trên chính batch đó - thường 3 đến 10 epochs - trước khi đi lấy batch mới. Cơ chế clipping là thứ giữ cho các lần cập nhật lặp lại này không đẩy policy đi quá xa điểm xuất phát, tức là có được tính ổn định mà không phải trả cái giá tính toán đắt đỏ như TRPO.

Ưu điểm: Thế mạnh thật sự của PPO không nằm ở chất lượng chính sách cuối cùng, mà ở chỗ nó hiếm khi hỏng. Clipping chặn trên độ lớn mỗi bước cập nhật, nên đặt sai tốc độ học thường chỉ làm quá trình học chậm lại, chứ không phá sập chính sách như ở các phương pháp policy gradient thuần túy. Thuật toán chạy được trên cả không gian hành động rời rạc lẫn liên tục, và trong ML-Agents thì đã có sẵn cơ chế self-play đi kèm. Chính đặc tính bền bỉ ấy khiến đề tài chọn PPO làm mốc so sánh; mọi thuật toán khác đều được đối chiếu với nó.

Nhược điểm: Cái giá phải trả là hiệu quả mẫu. Vì là thuật toán on-policy, PPO vứt sạch dữ liệu sau mỗi lần cập nhật, nên muốn đạt cùng một mức năng lực thì nó cần nhiều lần tương tác với môi trường hơn hẳn. Đặt trong một môi trường Unity có mô phỏng vật lý, nơi một bước tương tác đắt hơn nhiều so với một bước lan truyền ngược, đây là hạn chế nặng nhất của PPO. Đó cũng chính là lý do đề tài đưa SAC vào so sánh.

2.4.2 SAC

SAC (Soft Actor-Critic) do Haarnoja và cộng sự đề xuất năm 2018 [7, 8]. Đây là thuật toán actor-critic off-policy dựng trên nguyên lý maximum entropy, nhằm cùng lúc vào hai thách thức lớn của RL: hiệu quả mẫu thấp và độ nhạy cảm với siêu tham số.

Chỗ SAC khác các thuật toán RL thông thường nằm ngay ở hàm mục tiêu. Nó

không chỉ tối đa hóa tổng phần thưởng kỳ vọng, mà tối đa hóa đồng thời cả tổng phần thưởng lẫn entropy của policy:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t \left(r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right) \right] \quad (2.12)$$

trong đó $\mathcal{H}(\pi(\cdot | s_t)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s_t)]$ là entropy của policy tại trạng thái s_t , còn $\alpha > 0$ là hệ số nhiệt độ (temperature) dùng để cân giữa phần thưởng và entropy. Entropy càng cao thì policy càng ngẫu nhiên, và agent càng chịu thử nhiều hành động khác nhau thay vì chốt sớm vào một lựa chọn tham lam.

SAC nuôi ba mạng nơ-ron cùng lúc. Mạng actor $\pi_\theta(a | s)$ sinh phân phối xác suất trên không gian hành động; hai mạng critic $Q_{\phi_1}(s, a)$ và $Q_{\phi_2}(s, a)$ được huấn luyện độc lập để ước lượng hàm giá trị hành động. Chạy hai critic song song rồi lấy giá trị nhỏ hơn - kỹ thuật clipped double-Q - là cách SAC hạn chế hiện tượng overestimation vốn rất phổ biến ở các thuật toán Q-learning:

$$Q_{\text{target}}(s_t, a_t) = r_t + \gamma \mathbb{E}_{a' \sim \pi} \left[\min_{i=1,2} Q_{\phi_i}(s_{t+1}, a') - \alpha \log \pi_\theta(a' | s_{t+1}) \right] \quad (2.13)$$

Là thuật toán off-policy, SAC dồn toàn bộ kinh nghiệm tương tác vào replay buffer rồi lấy mẫu ngẫu nhiên theo mini-batch từ đó để cập nhật mạng. Dữ liệu cũ nhờ vậy được dùng đi dùng lại, và hiệu quả mẫu cao hơn PPO rất nhiều. Riêng hệ số nhiệt độ α thì có thể để cố định, hoặc để nó tự điều chỉnh theo mục tiêu entropy mong muốn $\mathcal{H}_{\text{target}}$, qua đó bớt được phần nào công tính chỉnh siêu tham số thủ công.

Ưu điểm: Nhờ replay buffer, mỗi mẫu kinh nghiệm được dùng lại nhiều lần, nên SAC cần ít lần tương tác với môi trường hơn hẳn các thuật toán on-policy; riêng trên môi trường điều hướng của đề tài, khoảng cách ấy lên tới một bậc độ lớn. Cơ chế cực đại hóa entropy giữ cho chính sách vẫn còn ngẫu nhiên ngay cả lúc phần thưởng đã bắt đầu tăng, qua đó chặn được việc hội tụ sớm vào một cực trị cục bộ. SAC cũng chịu đựng tốc độ học đặt sai tốt hơn PPO [8].

Nhược điểm: Đổi lại, SAC phải huấn luyện ba mạng nơ-ron cùng lúc và giữ một bộ đệm hàng trăm nghìn phần tử trong bộ nhớ, nên tính theo thời gian đồng hồ thì lợi thế về số mẫu bị thu hẹp đáng kể. Với đề tài này còn có một điểm nghiêm

trọng hơn nhiều: SAC không đi cùng được với self-play. Khi đối thủ liên tục được thay mới, kinh nghiệm cũ nằm trong buffer lại phản ánh những đối thủ đã không còn tồn tại, và trở thành nhiễu chứ không còn là dữ liệu học. Ràng buộc này chi phối toàn bộ cách đề tài bố trí thực nghiệm trên môi trường đối kháng; Mục 3.7 phân tích kỹ.

2.4.3 MA-POCA

MA-POCA được Cohen và cộng sự [4] đề xuất năm 2021, chuyên cho bài toán học tăng cường đa tác nhân hợp tác (cooperative multi-agent), và hiện là thuật toán đa tác nhân mặc định trong Unity ML-Agents. Nó nhắm vào hai thách thức đặc thù của môi trường đa tác nhân mà PPO hay SAC nguyên bản khó xử lý tốt. Thứ nhất là gán công lao (credit assignment): xác định từng agent đã đóng góp bao nhiêu vào phần thưởng chung của cả nhóm. Thứ hai là tác nhân biến động (posthumous credit): gán công lao cho đúng những agent đã ngừng hoạt động - bị loại hoặc đã hoàn thành nhiệm vụ - từ trước khi phần thưởng nhóm được trao.

Về nguyên lý, MA-POCA kéo kiến trúc actor-critic sang mô hình huấn luyện tập trung, thực thi phân tán (centralized training with decentralized execution). Lúc huấn luyện, một mạng critic tập trung (centralized critic) được phép nhìn trạng thái và hành động của toàn bộ agent trong nhóm để ước lượng giá trị chung, nên đánh giá đóng góp của từng agent chính xác hơn. Nhưng khi triển khai thực tế, mỗi agent lại chỉ dựa vào quan sát cục bộ của riêng mình mà ra quyết định - đó là điều giữ cho hệ thống khả thi và mở rộng được.

Cải tiến cốt lõi của MA-POCA nằm ở cơ chế attention, dùng để xử lý số lượng agent thay đổi động trong một episode. Các phương pháp trước đó cố định số agent [6, 11]; MA-POCA thì cho phép agent gia nhập hoặc rời nhóm giữa chừng mà việc gán công lao vẫn chính xác. Đây đúng là tình huống hay gặp trong game hợp tác, nơi agent có thể “chết” hoặc hoàn thành nhiệm vụ vào những thời điểm rất khác nhau. Hàm giá trị nhóm được học từ tổng phần thưởng nhóm (group reward) mà môi trường cung cấp qua cơ chế SimpleMultiAgentGroup của ML-Agents.

Phân cập nhật policy thì MA-POCA kế thừa gần như nguyên vẹn từ PPO: cùng hàm mục tiêu clipped surrogate, cùng cách ước lượng advantage, và cả hai đều là

thuật toán on-policy. Khác biệt nằm ở chỗ hàm giá trị trạng thái đơn tác nhân $V(s)$ được thay bằng hàm giá trị nhóm dựa trên baseline tập trung, để mỗi agent nhận về tín hiệu học phản ánh đúng phần đóng góp của mình vào kết quả chung.

Ưu điểm: Có một câu hỏi mà PPO chạy độc lập cho từng agent không trả lời nổi, còn MA-POCA thì trả lời được: khi cả nhóm cùng nhận một phần thưởng, agent nào mới thật sự làm nên kết quả đó. Cơ chế attention giữ cho việc gán công lao vẫn đứng vững kể cả khi nhóm đổi quân số giữa chừng - tình huống rất thường gặp trong game hợp tác, nơi nhân vật có thể bị loại hoặc xong phần việc của mình sớm. Thêm nữa, trong ML-Agents thuật toán này ăn khớp sẵn với cơ chế group reward, nên môi trường hợp tác gần như chẳng phải sửa gì.

Nhược điểm: Là thuật toán on-policy, MA-POCA thừa hưởng trọn vẹn vấn đề hiệu quả mẫu của PPO, lại còn gánh thêm chi phí của mạng phê bình tập trung khi quân số đông lên. Đáng bàn hơn là sự linh hoạt kia không hề miễn phí. Attention phải học cách phân bổ trọng số giữa các agent, mà trong một đội hình chỉ có hai tác nhân giống hệt nhau thì đó là công sức đổ vào một bài toán vốn không tồn tại. Thực nghiệm ở Chương 4 cho thấy chính tình huống này đã khiến MA-POCA thua MAPPO. Còn với bài toán đơn tác nhân, thuật toán suy biến gần về PPO mà vẫn phải kéo theo toàn bộ phần kiến trúc thừa.

2.4.4 MAPPO

MAPPO (Multi-Agent Proximal Policy Optimization) do Yu và cộng sự đề xuất năm 2022 [29], giải bài toán đa tác nhân bằng cách mở rộng trực tiếp từ PPO. MA-POCA chọn con đường phức tạp là mạng critic tập trung cộng cơ chế attention; MAPPO đi lối đơn giản hơn nhiều. Nó đưa PPO sang môi trường đa tác nhân qua framework CTDE (Centralized Training with Decentralized Execution), kết hợp chia sẻ tham số và một hàm giá trị tập trung.

Kiến trúc CTDE trong MAPPO: Khi thực thi, mỗi agent i ra hành động theo policy cá nhân $\pi_{\theta_i}(a_i|o_i)$, chỉ dựa vào quan sát cục bộ o_i của nó. Còn khi huấn luyện (centralized training), hàm giá trị tập trung $V_{\phi}(s)$ mới được đem ra để ước lượng giá trị của trạng thái toàn cục s ; nhờ đó việc đánh giá một trạng thái chung chính xác hơn và phương sai khi tính policy gradient cũng giảm xuống. Hàm giá trị tập

trung trong MAPPO có dạng:

$$V_\phi(s) = V_\phi(o_1, o_2, \dots, o_n, s_{\text{global}})$$

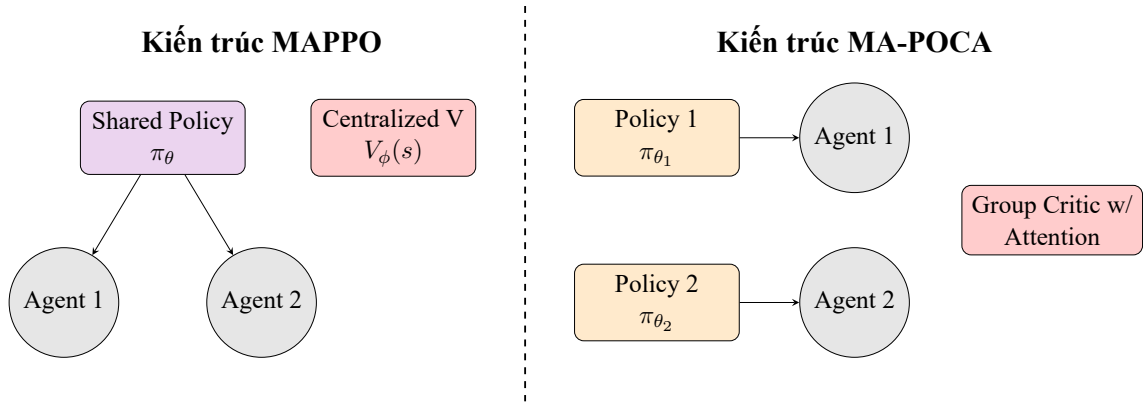
với $o_1, \dots, o_n$ là quan sát của các agent và s_{global} là các thông tin toàn cục khác của môi trường.

Với mỗi agent i , hàm mục tiêu của policy trong MAPPO được định nghĩa là:

$$L_i^{\text{MAPPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t^i, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t^i \right) \right] \quad (2.14)$$

trong đó $\hat{A}_t^i$ là hàm lợi thế (advantage function) cho agent i , được tính toán dựa trên hàm giá trị tập trung $V_\phi(s)$, và $r_t(\theta) = \frac{\pi_\theta(a_t^i|o_t^i)}{\pi_{\theta_{\text{old}}}(a_t^i|o_t^i)}$ là tỷ số xác suất.

Cơ chế chia sẻ tham số (Parameter sharing): Về nguyên tắc mỗi agent có thể giữ một policy riêng, nhưng MAPPO thường cho tất cả các agent đồng nhất (homogeneous agents) dùng chung tham số mạng nơ-ron, tức $\theta_1 = \theta_2 = \dots = \theta_n = \theta$. Cách này vừa cắt được chi phí tính toán và bộ nhớ, vừa giúp các agent học nhanh hơn nhờ chia sẻ kinh nghiệm (shared experience) trong lúc huấn luyện. Nếu cần mạng phân biệt các agent để chúng cư xử khác nhau, danh tính (identity) của từng agent có thể mã hóa thành vector one-hot rồi ghép vào quan sát (observation).



Hình 2.3: So sánh cấu trúc giữa MAPPO và MA-POCA

Ưu điểm: Nghịch lý là sức mạnh của MAPPO đến từ chính chỗ nó chẳng thêm gì mới. Mọi agent dùng chung một bộ tham số, nên mỗi lượt chơi sinh ra n luồng kinh nghiệm cùng đổ về một mạng duy nhất; với n tác nhân, cùng một ngân sách mô phỏng cho ra gấp n lần dữ liệu huấn luyện hữu ích. Thêm vào đó, thuật toán chạy được trên nguyên bộ hạ tầng vốn đã tối ưu sẵn cho PPO. Yu và cộng sự [29]

ghi nhận MAPPO bám sát, có khi vượt, các thuật toán đa tác nhân nặng ký hơn như MADDPG và QMIX trên nhiều bộ benchmark hợp tác.

Nhược điểm: Chia sẻ tham số chỉ hợp lý chừng nào các agent thật sự giống nhau về quan sát lẫn hành động. Gặp đội hình có vai trò bất đối xứng, một mạng duy nhất buộc phải học lẫn lộn nhiều hành vi mâu thuẫn nhau. Hàm giá trị tập trung $V_\phi(s)$ cũng phình lên theo quân số, vì không gian trạng thái toàn cục cứ thế lớn dần. Và khác MA-POCA, MAPPO không sẵn có cách nào gọn gàng để xử lý trường hợp số agent thay đổi giữa lượt chơi.

2.4.5 DQN

DQN (Deep Q-Network) do Mnih và cộng sự đề xuất năm 2015 [14]. Đây là thuật toán đầu tiên cho thấy một mạng nơ-ron sâu có thể học chơi game ở trình độ con người chỉ từ điểm ảnh thô. Nó khác bốn thuật toán phía trên ở một điểm cơ bản: DQN không tham số hóa chính sách. Mạng nơ-ron học xấp xỉ hàm giá trị hành động $Q_\theta(s, a)$, còn chính sách thì suy ra gián tiếp bằng cách chọn hành động có giá trị lớn nhất. Đề tài đưa DQN vào lưới so sánh để có một đại diện của nhóm value-based đứng cạnh bốn thuật toán dựa trên chính sách, qua đó xem hướng tiếp cận cổ điển này còn cạnh tranh nổi đến đâu trong môi trường ba chiều.

Nguyên lý hoạt động: DQN học hàm Q bằng cách cực tiểu hóa sai số thời gian khác biệt (temporal difference) giữa giá trị dự đoán và giá trị mục tiêu suy ra từ phương trình Bellman:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a) \right)^2 \right] \quad (2.15)$$

với $\mathcal{D}$ là bộ đệm phát lại chứa các bộ chuyển tiếp đã thu thập, còn θ^- là tham số của mạng mục tiêu. Đúng hai thành phần này làm nên đóng góp then chốt của DQN. Bộ đệm phát lại vừa cho phép dùng lại mỗi mẫu kinh nghiệm nhiều lần, vừa phá vỡ tương quan thời gian giữa các mẫu liên tiếp trong cùng một lô cập nhật. Mạng mục tiêu thì là bản sao của mạng chính nhưng cập nhật chậm hơn, giữ giá trị mục tiêu tương đối cố định trong từng giai đoạn ngắn. Thiếu nó, mạng sẽ đuổi theo chính cái mục tiêu do mình sinh ra, và quá trình học rất dễ phân kỳ.

Khám phá trong DQN không đến từ entropy của chính sách như ở PPO hay

SAC, mà đến từ quy tắc ε -tham lam: với xác suất ε thì tác nhân chọn hành động ngẫu nhiên, phần xác suất còn lại thì chọn hành động có Q lớn nhất, và ε giảm dần theo số bước huấn luyện. Điều này kéo theo một hệ quả trực tiếp cho đề tài: DQN không sinh ra chỉ số entropy của chính sách, nên nó vắng mặt trong mọi so sánh dựa trên đại lượng này ở Chương 4.

Ưu điểm: DQN là thuật toán off-policy nên tận dụng lại được toàn bộ kinh nghiệm đã lưu, về lý thuyết cho hiệu quả mẫu cao. Kiến trúc chỉ có một mạng giá trị, nhẹ hơn hẳn các thuật toán actor-critic vốn phải nuôi nhiều mạng cùng lúc. Và khi không gian hành động rời rạc lại nhỏ, chọn hành động bằng $\max_a Q(s, a)$ vừa chính xác vừa chẳng tốn thêm chi phí tối ưu nào.

Nhược điểm: Cũng chính phép lấy $\max_a Q(s, a)$ khiến DQN không dùng thẳng được cho không gian hành động liên tục - đây là rào cản mà đề tài gặp phải trên môi trường Football Table. Toán tử cực đại còn gây thiên lệch ước lượng vượt mức (overestimation bias), bởi nhiều trong ước lượng Q luôn bị khuếch đại về phía trên. Cuối cùng, do học một chính sách xác định, DQN chẳng có cơ chế nội tại nào giữ lại tính ngẫu nhiên sau khi ε đã giảm, nên rất dễ chốt sớm vào một phương án dưới tối ưu.

2.4.6 Tổng kết và so sánh năm thuật toán

Năm thuật toán vừa trình bày không xếp được thành một thang bậc từ yếu tới mạnh. Chúng phân hóa theo bốn trục độc lập nhau: học cái gì (hàm giá trị hay chính sách), lấy dữ liệu ra sao (on-policy hay off-policy), xử lý nhiều tác nhân thế nào (không có, attention, hay chia sẻ tham số), và mỗi bước cập nhật tốn bao nhiêu tính toán. Cái quyết định thuật toán nào hợp bài toán nào chính là bốn trục ấy, chứ không phải một thứ hạng chung.

Bảng 2.2: So sánh các thuật toán học tăng cường

Thuật toán	Loại policy	On/Off-policy	Đa tác nhân	Replay buffer	Entropy	Chia sẻ tham số	Độ phức tạp
PPO	Policy gradient	On-policy	Không native	Không	Không	Không	Thấp
SAC	Actor-Critic	Off-policy	Không native	Có	Có (max entropy)	Không	Trung bình
MA-POCA	Actor-Critic	On-policy	Có (attention)	Không	Không	Có (chia sẻ)	Cao
MAPPO	Policy gradient	On-policy	Có (CTDE)	Không	Không	Có (bắt buộc)	Trung bình
DQN	Hàm giá trị	Off-policy	Không native	Có	Không (ϵ -tham lam)	Không	Thấp

Từ bảng trên có thể rút ra một khuyến nghị đủ cụ thể để dùng ngay. Chưa nắm rõ đặc thù bài toán thì cứ bắt đầu bằng PPO: nó chịu được siêu tham số đặt sai, và cho một mốc so sánh đáng tin để đánh giá mọi thuật toán khác. Chỉ nên đổi sang SAC khi mỗi bước tương tác với môi trường đắt hơn hẳn mỗi bước cập nhật mạng, vì đó mới đúng là lúc replay buffer trả lại đủ giá trị để bù cho chi phí nuôi ba mạng song song. Sang bài toán nhiều tác nhân, ranh giới nằm ở chỗ đội hình có đồng nhất hay không. Nếu các tác nhân giống nhau và số lượng cố định, chia sẻ tham số của MAPPO khai thác dữ liệu triệt để nhất. Còn khi vai trò bất đối xứng, hoặc khi tác nhân có thể rời nhóm giữa chừng, lúc đó cơ chế attention của MA-POCA mới thật sự cần. DQN thì đứng ngoài cả ba cân nhắc vừa nêu: nó chỉ dùng được với hành động rời rạc, và trong đề tài này vai trò của nó là mốc tham chiếu cho nhóm value-based chứ không phải một ứng viên cạnh tranh. Chương 4 sẽ kiểm chứng từng khuyến nghị bằng số liệu thực đo.

2.5 Unity ML-Agents Toolkit

Unity ML-Agents Toolkit [9] là framework mã nguồn mở của Unity Technologies, dùng để xây dựng môi trường game và mô phỏng 3D làm nền huấn luyện agent thông minh bằng học tăng cường. Ra mắt lần đầu năm 2017 và được cập nhật liên

tục từ đó, ML-Agents nay thuộc nhóm nền tảng phổ biến nhất cho nghiên cứu RL trong môi trường game. Lý do nằm ở chỗ nó ghép liền mạch engine đồ họa Unity với hệ sinh thái Python của cộng đồng học máy.

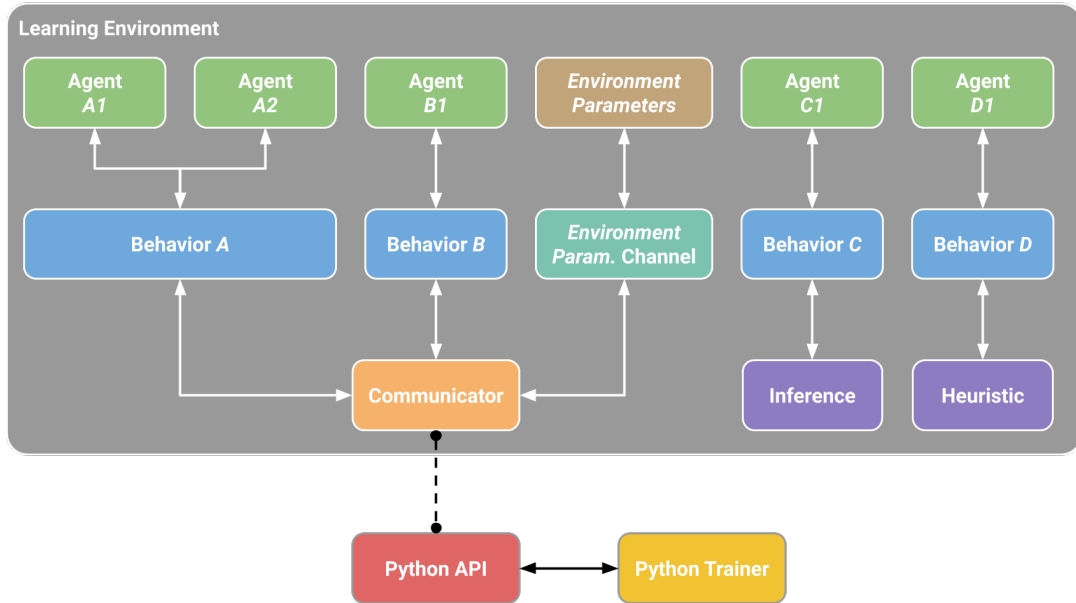
2.5.1 Kiến trúc tổng thể

ML-Agents dựng theo mô hình client-server: Unity Engine làm môi trường mô phỏng, Python làm backend huấn luyện. Hai bên nói chuyện với nhau qua giao thức gRPC trên socket, và nhờ đó có thể chạy đồng thời nhiều instance môi trường Unity để thu thập dữ liệu nhanh hơn.

Bên phía Unity, thành phần trung tâm là Agent, một MonoBehaviour gắn vào GameObject trong scene. Nó lo ba việc: thu thập quan sát qua CollectObservations, nhận hành động qua OnActionReceived, và tính phần thưởng. Đi kèm mỗi Agent là một component BehaviorParameters định nghĩa không gian quan sát, không gian hành động cùng loại hành vi (học hoặc heuristic). Trên tất cả là Academy, singleton quản lý toàn bộ vòng lặp huấn luyện, đồng bộ các bước thời gian giữa mọi agent trong scene và điều phối giao tiếp với Python backend.

Bên phía Python, điểm vào chính của quá trình huấn luyện là công cụ dòng lệnh mlagents-learn: nó đọc tệp cấu hình YAML rồi khởi tạo trainer tương ứng cho từng behavior khai báo trong đó. Các lớp Trainer - tiêu biểu là PPOTrainer và POCATrainer - thực thi toàn bộ logic huấn luyện: thu thập trajectory, tính advantage, cập nhật tham số mạng và lưu checkpoint định kỳ. Ngoài lớp trainer, thư viện mlagents_envs còn mở ra một Python API cấp thấp, cho phép tương tác thẳng với môi trường Unity mà bỏ qua trainer tích hợp sẵn.

Hình 2.4 minh họa kiến trúc tổng thể của hệ thống ML-Agents và luồng dữ liệu giữa các thành phần.



Hình 2.4: Kiến trúc tổng thể của Unity ML-Agents Toolkit

2.5.2 Cơ chế Vật lý và Tính xác định trong Unity

Khi dùng Unity cho học tăng cường, có một khía cạnh cốt lõi phải để ý: sự đồng bộ giữa hệ thống vật lý và quá trình ra quyết định. Unity kết xuất khung hình đồ họa qua hàm `Update()`, với tần số biến thiên tùy hiệu năng máy tính. Nếu Agent cũng ra quyết định hay tính va chạm theo cái tần số trôi sụt ấy, môi trường lập tức mất tính xác định: cùng một chuỗi hành động có thể cho ra những kết quả vật lý khác nhau ở các lần chạy khác nhau.

Cách ML-Agents xử lý là dồn toàn bộ quy trình RL vào hàm `FixedUpdate()`, chu kỳ cập nhật vật lý có tần số cố định - mặc định 0.02 giây mỗi bước, tức 50 lần/giây. Hệ thống vật lý PhysX của Unity tính động lực học, cập nhật tọa độ các Rigidbody và xử lý va chạm (Collider) hoàn toàn theo những bước thời gian cố định này. Nhờ vậy, chỉ cần cùng một seed ngẫu nhiên ban đầu là môi trường Unity hành xử như một hệ thống xác định: cùng một chuỗi hành động luôn cho cùng một quỹ đạo. Thiếu tính chất đó thì mọi so sánh giữa hai lần chạy đều mất căn cứ.

2.5.3 Hệ thống Cảm biến và Tiền xử lý quan sát

Muốn Agent hiểu được một thế giới 3D phức tạp thì phải có cách chuyển thông tin không gian thành vector số thực cho lớp đầu vào. ML-Agents lo việc đó bằng các

thành phần Cảm biến, gồm ba loại phổ biến:

- **Cảm biến tia:** Bắn các tia raycast quanh Agent theo một góc quét cho trước. Tia chạm phải vật thể mang nhãn tương ứng thì ghi lại một vector one-hot cho loại vật thể, kèm một giá trị vô hướng chuẩn hóa trong $[0, 1]$ biểu diễn khoảng cách. Loại này vừa hiệu quả vừa nhẹ về tính toán với các bài toán điều hướng, và là cảm biến chủ đạo của đề tài.
- **Grid Sensor:** Phủ một lưới 2D lên không gian rồi đếm số vật thể rơi vào từng ô. Hợp với những bài toán cần cái nhìn toàn cảnh, chẳng hạn cờ bàn.
- **Camera Sensor:** Lấy thẳng dữ liệu pixel RGB từ camera, kéo theo việc mô hình buộc phải dùng mạng tích chập. Thông tin thì toàn diện, nhưng chi phí tính toán rất lớn và tốc độ hội tụ thường chậm hơn nhiều so với cảm biến tia.

2.5.4 Cơ chế Hành vi và Gom nhóm quyết định

Gắn trên mỗi Agent còn có một module quan trọng khác là BehaviorParameters, nơi khai báo kích thước không gian quan sát và không gian hành động. Chính tại đây ML-Agents đưa ra cơ chế Action Masking (Mặt nạ hành động) để chặn những hành động không hợp lệ. Chẳng hạn khi Agent đang đứng sát tường, hệ thống “đeo mặt nạ” lên hành động tiến lên, ép xác suất chọn nó về 0. Không gian tìm kiếm nhờ thế hẹp lại và mô hình hội tụ nhanh hơn, thay vì phải tự học rằng đâm đầu vào tường thì bị phạt.

Bên cạnh đó, thành phần DecisionRequester cho phép gom nhóm các bước mô phỏng, tức Decision Period. Thay vì ra quyết định mới ở từng bước FixedUpdate (0.02s), Agent có thể giữ nguyên hành động cũ suốt 5 bước liên tiếp (Decision Period = 5). Cách này vừa cho Agent đủ thời gian hoàn thành một quỹ đạo vật lý trước khi quyết định tiếp, vừa cắt số lần gọi mạng nơ-ron đi nhiều lần, nên tốc độ huấn luyện cải thiện rõ rệt. ML-Agents cũng hỗ trợ Heuristic Policy: người dùng trực tiếp điều khiển Agent bằng bàn phím để gỡ lỗi và tự cảm nhận độ khó của môi trường trước khi khởi động AI.

2.5.5 Quy trình huấn luyện Agent

Huấn luyện một agent trong ML-Agents là một vòng lặp chạy đi chạy lại cho tới khi chạm số bước tối đa hoặc người dùng bấm dừng. Ở mỗi bước thời gian, Unity gom quan sát từ tất cả agent đang hoạt động trong scene rồi đẩy sang Python backend qua gRPC. Python trainer nhận quan sát, suy luận hành động từ policy hiện tại (hoặc lấy mẫu ngẫu nhiên nếu còn ở giai đoạn khởi động) và trả hành động về. Unity thực thi hành động đó, cập nhật trạng thái vật lý, tính phần thưởng, rồi gửi kết quả ngược lại Python để tích vào buffer.

Khi buffer đầy một batch với kích thước do `buffer_size` quy định, trainer mới cập nhật tham số mạng nơ-ron. Với PPO, mỗi lần cập nhật gồm nhiều epoch chạy trên cùng batch đó, dùng hàm mục tiêu clipped surrogate đã trình bày ở Mục 2.4.1. Cập nhật xong thì buffer được xóa sạch để mở một chu kỳ thu thập mới. Song song, các chỉ số huấn luyện như cumulative reward, episode length và policy loss được ghi lại liên tục, theo dõi được theo thời gian thực qua TensorBoard.

Thứ điều khiển toàn bộ quá trình huấn luyện là tệp cấu hình YAML. Trong đó khai báo loại trainer (`trainer_type`), siêu tham số huấn luyện (`hyperparameters`), kiến trúc mạng nơ-ron (`network_settings`), cấu hình tín hiệu phần thưởng (`reward_signals`) và số bước huấn luyện tối đa (`max_steps`). Mỗi behavior trong scene có một khối cấu hình riêng, nên hoàn toàn có thể huấn luyện đồng thời nhiều loại agent với thiết lập khác nhau trong cùng một session. Cấu hình chi tiết cho từng thuật toán nằm ở các Phụ lục A đến D.

2.5.6 Hỗ trợ thuật toán và giới hạn tích hợp sẵn

Unity ML-Agents phiên bản 4.0.2 tích hợp sẵn đúng ba trainer. ppo lo bài toán đơn tác nhân và đối kháng; poca (MA-POCA) lo bài toán hợp tác đa tác nhân; sac là lựa chọn off-policy duy nhất. Kèm theo là các tín hiệu phần thưởng nội tại như curiosity và RND, hỗ trợ khám phá trong môi trường phần thưởng thưa. Toàn bộ việc chọn thuật toán và đặt siêu tham số diễn ra trong tệp YAML, không phải động vào mã nguồn C# của môi trường.

Giới hạn đầu tiên lộ ra ngay từ danh sách đó: hai trong năm thuật toán mà đề tài cần so sánh, MAPPO và DQN, đều không có mặt. Giải pháp là bổ sung chúng vào

ML-Agents dưới dạng trainer mở rộng - viết thêm các lớp trainer rồi đăng ký vào cơ chế tra cứu theo tên `trainer_type`. Nhờ vậy chúng dùng chung với ba trainer có sẵn toàn bộ hạ tầng thu thập quỹ đạo, ghi nhật ký TensorBoard và xuất mô hình ONNX. Điểm mấu chốt của cách làm này là cả năm thuật toán cùng chạy trên một đường ống đo đạc, điều kiện cần để mọi so sánh ở Chương 4 có ý nghĩa. Mục 3.7 trình bày chi tiết.

Giới hạn thứ hai nằm ở cơ chế tự chơi của môi trường đối kháng. ML-Agents thiết kế cơ chế này quanh giả định chính sách là ngẫu nhiên và được cập nhật theo lô on-policy. Trên thực tế PPO, MA-POCA, MAPPO và SAC đều ghép được với tự chơi và đều sinh ra chỉ số ELO. Riêng DQN thì không: bộ điều phối đối kháng báo lỗi tra cứu chính sách ngay lúc khởi tạo. Hệ quả là lần chạy DQN trên Football Table buộc phải tắt tự chơi, và vì thế cũng không có ELO để đối chiếu.

Giới hạn cuối cùng thuộc về phần thưởng nhóm. Cơ chế group reward vốn được tối ưu cho POCA; đem áp cho PPO, SAC hay DQN thì phần thưởng nhóm phải phân bổ thủ công về từng agent, và khả năng gán công lao suy giảm theo. Thêm nữa, các trainer tích hợp sẵn không dễ cho người dùng can thiệp sâu vào vòng lặp huấn luyện, chẳng hạn muốn tùy biến hàm mất mát hay lịch trình cập nhật. Khi cần tới mức kiểm soát đó, thư viện `mlagents_envs` mở ra Python Low-Level API: nó cho phép điều khiển thẳng môi trường Unity như một môi trường kiểu Gym, và từ đó nối được với các thư viện RL bên ngoài như Stable-Baselines3 [19].

3. THIẾT KẾ MÔI TRƯỜNG VÀ PHƯƠNG PHÁP THỰC NGHIỆM

Đây là chương trọng tâm của đề án. Mục 1 đã lập luận rằng phần khó nhất của một dự án học tăng cường nằm ở khâu xây dựng môi trường; chương này chính là nơi trình bày cái phần khó ấy, tức quá trình biến ba ý tưởng trò chơi thành ba đặc tả mà tác nhân học được.

Điều được đặt lên hàng đầu ở đây là lý do đằng sau từng quyết định thiết kế, chứ không chỉ kết quả cuối cùng. Với mỗi môi trường, chương sẽ nói rõ đại lượng nào được đưa vào quan sát và vì sao đại lượng khác bị gạt ra, phần thưởng chia thành những thành phần nào và mỗi thành phần chặn lại hành vi sai lệch cụ thể nào, kèm cả những đặc tả đã thử rồi phải bỏ. Phần cuối này quan trọng không kém phần

thành công. Một hàm phần thưởng hồng chẳng làm chương trình báo lỗi; nó chỉ lãng lễ khiến tác nhân học sai. Biết được dấu hiệu nhận ra nó vì thế thực dụng hơn nhiều so với việc chép lại một cấu hình đã chạy được.

Chương được bố cục như sau. Mở đầu là những gì dùng chung cho cả ba môi trường: kiến trúc huấn luyện, cách chọn quan sát và hành động, cùng bốn nguyên tắc thiết kế hàm phần thưởng - mọi con số trong các bảng về sau đều phải giải thích được bằng bốn nguyên tắc đó. Ba mục kế tiếp đi vào từng môi trường một. Sau đó là lối đi vòng qua Python Low-Level API, thứ mà đề tài buộc phải dùng vì trainer tích hợp sẵn không đáp ứng nổi. Khép lại chương là thiết lập thực nghiệm và các tiêu chí đánh giá, dùng để nghiệm thu ba môi trường ở Chương 4.

3.1 Tổng quan thiết kế hệ thống

3.1.1 Kiến trúc huấn luyện tổng thể

Đề tài dựng toàn bộ hệ thống thực nghiệm theo kiến trúc client-server của Unity ML-Agents đã trình bày ở Mục 2.5.1. Unity giữ vai trò mô phỏng: dựng thế giới ba chiều, tính vật lý va chạm, thu quan sát của tác nhân và thực thi những hành động mà tác nhân chọn. Phía Python, qua chương trình `mlagents-learn`, giữ vai trò huấn luyện: nhận quan sát từ Unity, suy ra hành động theo chính sách hiện tại, rồi cập nhật tham số mạng nơ-ron dựa trên phần thưởng thu về. Dữ liệu giữa hai phía chảy qua lại theo từng bước thời gian, thông qua giao thức truyền thông nội bộ.

Ba môi trường của đề tài được tổ chức thành ba khu vực độc lập trong cùng một dự án Unity, tương ứng với ba thư mục `Assets/Football`, `Assets/Collaboration` và `Assets/CrossTheRoad`; riêng phần mã điều khiển của cả ba đã được tách ra thành một gói dùng lại được, trình bày ở Mục 5.7. Mỗi môi trường được gắn với một loại hành vi riêng, lần lượt mang tên `Football`, `PuzzleBehavior` và `CrossTheRoad`. Tách biệt như vậy có hai cái lợi: huấn luyện từng môi trường độc lập mà không nhiễu lẫn nhau, và nạp đúng tệp cấu hình cho từng lần chạy một cách rõ ràng.

Ở mỗi môi trường, toàn bộ nội dung của một lượt chơi được gom vào một khu vực huấn luyện rồi đóng thành prefab độc lập. Nhờ đó có thể nhân khu vực ấy thành

nhiều bản sao đặt cạnh nhau trong cùng một cảnh, thu nhiều mẫu kinh nghiệm cùng lúc. Cả ba cảnh dùng cho những lần huấn luyện báo cáo ở Chương 4 đều chạy song song tám khu vực. Cách này vừa tăng lượng kinh nghiệm thu được trên mỗi bước mô phỏng, vừa giảm tương quan giữa các mẫu trong cùng một lô cập nhật. Nhưng nó kéo theo một hệ quả cần lưu ý: cùng một số bước môi trường ở đây không tương đương với cùng số bước ở cấu hình chỉ dùng một khu vực duy nhất, và đó là chi tiết phải nhớ khi đem kết quả của đề tài đối chiếu với các công bố khác. Ngoài ra, hệ số tăng tốc mô phỏng được đặt ở mức gấp hai mươi lần thời gian thực; Unity nhờ vậy mô phỏng nhanh hơn nhiều tốc độ hiển thị thông thường mà vật lý vẫn đúng. Cả mười lăm lần huấn luyện của đề tài đều do một chương trình Python điều phối tự động, trình bày ở Mục 3.7.4.

3.1.2 Không gian quan sát

Có một ranh giới phải giữ cho rành mạch trong bất kỳ hệ thống học tăng cường nào: ranh giới giữa trạng thái của môi trường và quan sát mà tác nhân nhận được. Trạng thái là toàn bộ thông tin mô tả đầy đủ tình huống hiện tại của thế giới mô phỏng, kể cả những đại lượng tác nhân không cảm nhận trực tiếp được. Quan sát thì hẹp hơn, chỉ gồm phần thông tin mà tác nhân thực sự với tới qua cảm biến của nó. Mọi quyết định thiết kế về sau đều xuất phát từ chỗ phân biệt này, nên trước hết đề tài phát biểu nó một cách hình thức.

Định nghĩa 3.1 (Không gian quan sát). Cho môi trường có không gian trạng thái S . Không gian quan sát Ω là tập hợp tất cả các quan sát khả dĩ mà tác nhân có thể nhận được. Hàm quan sát $O : S \rightarrow \Omega$ ánh xạ mỗi trạng thái $s \in S$ sang một quan sát $o = O(s)$ mà tác nhân dùng làm căn cứ duy nhất để ra quyết định tại bước thời gian tương ứng.

Nếu hàm quan sát O là song ánh thì mỗi quan sát ứng với đúng một trạng thái, và tác nhân coi như “nhìn thấy” trọn vẹn trạng thái; bài toán lúc đó là một quá trình quyết định Markov (MDP) đầy đủ, đúng như Mục 2.2.2 đã trình bày. Còn khi O không song ánh, tức nhiều trạng thái khác nhau cùng cho ra một quan sát, tác nhân mất khả năng phân biệt chúng và bài toán chuyển thành quá trình quyết định Markov quan sát một phần (Partially Observable MDP, viết tắt POMDP). Hầu hết

môi trường game, ba môi trường của đề tài không ngoại lệ, đều là POMDP về bản chất. Tác nhân đâu thể biết chính xác vận tốc tương lai của một phương tiện, cũng chẳng nhìn xuyên tường được để biết đồng đội đang ở đâu. Vậy nên việc của người thiết kế không phải là cung cấp toàn bộ trạng thái, mà là chọn ra một tập quan sát đủ để bài toán xấp xỉ được tính chất Markov - đủ để riêng quan sát hiện tại đã chứa gần như mọi thông tin cần cho một quyết định hợp lý, không bắt tác nhân phải nhớ toàn bộ lịch sử.

Định nghĩa 3.2 (Tính chất Markov của quan sát). Một biểu diễn quan sát được gọi là thỏa mãn tính chất Markov nếu phân phối xác suất của trạng thái kế tiếp và phần thưởng chỉ phụ thuộc vào quan sát và hành động hiện tại, mà không phụ thuộc thêm vào bất kỳ quan sát nào trong quá khứ:

$$\Pr(o_{t+1}, r_{t+1} \mid o_t, a_t, o_{t-1}, a_{t-1}, \dots) = \Pr(o_{t+1}, r_{t+1} \mid o_t, a_t).$$

Tính chất Markov tuyệt đối gần như không bao giờ đạt được trong thực tế, nên mục tiêu thiết kế chỉ là tiệm cận nó với chi phí quan sát thấp nhất. Khi một quan sát tức thời chưa đủ Markov, có hai kỹ thuật hỗ trợ hay được dùng. Thứ nhất là chồng khung: ghép vài quan sát liên tiếp gần nhất thành một vector duy nhất, nhờ đó thông tin về xu hướng chuyển động - chẳng hạn vận tốc suy từ hiệu vị trí giữa hai khung - hiện lên tường minh. Thứ hai là dùng mạng có bộ nhớ hồi tiếp, để chính tác nhân tự tổng hợp lịch sử. Đề tài thiên về cách thứ nhất, kết hợp bổ sung thẳng các đại lượng động như vận tốc vào vector quan sát, cốt giữ cho mạng chính sách đơn giản và ổn định.

Quan sát ở cả ba môi trường đều là dạng vector, tức một chuỗi số thực độ dài cố định, mỗi phần tử mã hóa một đại lượng mang ý nghĩa vật lý hoặc logic. Thường gặp ba loại. Đại lượng liên tục gồm tọa độ, vận tốc, khoảng cách, góc xoay. Còn nhị phân biểu diễn trạng thái logic - bàn đạp đã nhấn hay chưa chẳng hạn - mã hóa bằng không hoặc một. Và mã hóa một-nóng, dùng khi cần biểu diễn một biến rời rạc nhiều mức. Nguyên tắc xuyên suốt là tối thiểu nhưng đủ. Mỗi phần tử thừa đều đẩy số chiều đầu vào của mạng lên và làm việc học chậm lại; ngược lại mỗi phần tử thiếu có thể phá vỡ tính chất Markov, khiến bài toán thành bất khả học.

Trước khi vào mạng nơ-ron, các giá trị quan sát liên tục được chuẩn hóa về lân cận khoảng $[-1, 1]$ cho quá trình huấn luyện ổn định. Với một đại lượng x có kỳ

vọng μ và độ lệch chuẩn σ ước lượng trực tuyến trong lúc chạy, giá trị chuẩn hóa là $\hat{x} = (x - \mu)/\sigma$. Riêng môi trường Football Table thì bước này là bắt buộc: vận tốc bóng và góc xoay thanh gạt ở đó có thang đo chênh nhau nhiều bậc, để nguyên thì những đại lượng giá trị lớn sẽ sinh gradient lẫn át và làm hỏng việc học của các đại lượng nhỏ.

Ngoài quan sát vector, hai môi trường Cross The Road và Capture The Flag còn được gắn thêm cảm biến tia - cơ chế quan sát không gian mô phỏng thị giác của một thực thể trong thế giới ba chiều. Cảm biến phóng từ vị trí tác nhân ra một chùm tia trải đều trong góc quạt phía trước. Mỗi tia dò vật thể đầu tiên nó chạm phải trong một tầm xa giới hạn, rồi trả về hai thứ: loại vật thể bị chạm, mã hóa một-nóng trên tập nhãn cần phát hiện; và khoảng cách chuẩn hóa từ tác nhân tới điểm chạm. Gọi k là số tia, m là số nhãn cần phát hiện, thì mỗi tia sinh $m + 2$ giá trị - m giá trị một-nóng cho nhãn, một giá trị báo tia có chạm gì không, một giá trị khoảng cách - và tổng số chiều quan sát mà cảm biến tia đóng góp là $k \times (m + 2)$. Nhờ cảm biến này, tác nhân trong Capture The Flag phát hiện được tường, cửa, bàn đạp, điểm đích lẫn đồng đội nằm trong tầm nhìn, mà chẳng cần ai đưa cho nó tọa độ tuyệt đối của những thứ đó.

3.1.3 Không gian hành động

Nếu quan sát là đầu vào thì hành động là đầu ra: thứ mà chính sách của tác nhân sinh ra ở mỗi bước để tác động lên môi trường.

Định nghĩa 3.3 (Không gian hành động). Không gian hành động A là tập hợp tất cả các hành động mà tác nhân được phép thực hiện. Không gian này được gọi là rời rạc nếu A là một tập hữu hạn các lựa chọn, và được gọi là liên tục nếu A là một tập con của không gian số thực nhiều chiều $\mathbb{R}^n$, khi đó mỗi hành động là một vector số thực bị chặn trong một khoảng cho trước.

Hai dạng không gian hành động kéo theo hai cách mạng chính sách biểu diễn đầu ra. Không gian rời rạc thì mạng xuất một phân phối xác suất trên tập hành động, tác nhân lấy mẫu từ phân phối đó; nếu có nhiều nhóm hành động độc lập, ML-Agents dùng dạng đa rời rạc, mỗi nhóm là một nhánh có phân phối riêng. Không gian liên tục thì mạng thường xuất tham số của một phân phối chuẩn nhiều chiều - kỳ vọng

và độ lệch chuẩn cho từng chiều - rồi hành động được lấy mẫu và cắt về khoảng hợp lệ. Cách biểu diễn khác nhau ấy giải thích vì sao một số thuật toán hợp với dạng không gian này hơn dạng kia. SAC là ví dụ rõ nhất: nó vốn được thiết kế cho điều khiển liên tục và phát huy thế mạnh nhiều nhất ở đó.

Đề tài cố ý chọn ba môi trường phủ cả hai dạng không gian hành động, để xem từng thuật toán linh hoạt tới đâu. Football Table dùng không gian hành động liên tục tám chiều: bốn thanh gạt, mỗi thanh một chiều điều khiển lực trượt ngang và một chiều điều khiển lực xoay, mỗi chiều nhận một giá trị thực - đúng với bản chất điều khiển vật lý mượt mà của trò bi lắc. Hai môi trường còn lại đều là rời rạc một nhánh. Cross The Road có bốn hành động: đứng yên, sang trái, sang phải, tiến. Capture The Flag có bảy: đứng yên, tiến, lùi, xoay trái, xoay phải, đi ngang sang trái, đi ngang sang phải. Có một điểm đáng nhấn mạnh ở hai môi trường rời rạc này, đó là quan hệ giữa hành động được chọn và chuyển động thực tế của tác nhân - và quan hệ ấy khác nhau ở hai nơi. Trong Capture The Flag, một hành động di chuyển không dịch tác nhân tức thời mà cộng thêm một lượng vận tốc vào thân vật lý của nó; chuyển động cuối cùng do hệ thống vật lý Unity quyết định dưới tác động của lực cản. Trong Cross The Road thì ngược lại: mỗi hành động chỉ định một ô đích cách vị trí hiện tại một đơn vị, tác nhân trượt đều tới đó, không đi qua mô phỏng lực chút nào. Chi tiết định lượng của cả hai cơ chế nằm ở phần thiết kế của từng môi trường.

3.1.4 Nguyên tắc thiết kế hàm phần thưởng

Trong một hệ thống học tăng cường, hàm phần thưởng là thành phần khó thiết kế nhất. Lý do nằm ở chỗ nó là tín hiệu duy nhất định hướng hành vi của tác nhân: chẳng ai bảo tác nhân phải làm gì, nó chỉ học cách tối đa hóa tổng phần thưởng chiết khấu thu về.

Định nghĩa 3.4 (Hàm phần thưởng). Hàm phần thưởng $R : S \times A \times S \rightarrow \mathbb{R}$ gán cho mỗi phép chuyển trạng thái, khi tác nhân thực hiện hành động a ở trạng thái s và môi trường chuyển sang trạng thái s' , một số thực $r = R(s, a, s')$. Mục tiêu của tác nhân là tối đa hóa kỳ vọng của tổng phần thưởng chiết khấu $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$, với $\gamma \in [0, 1)$ là hệ số chiết khấu.

Cả ba môi trường của đề tài đều xây hàm phần thưởng theo cùng một khuôn dạng, phân rã thành ba lớp thành phần với vai trò tách bạch nhau:

$$R(s, a, s') = \underbrace{R_{\text{mt}}(s')}_{\text{thưa, tại cột mốc}} + \underbrace{F(s, s')}_{\text{định hình theo thể năng}} + \underbrace{R_b(s)}_{\text{dày, mỗi bước}}. \quad (3.1)$$

Thành phần mục tiêu R_{mt} chỉ khác không ở một vài trạng thái đánh dấu kết quả nhiệm vụ - bóng vào lưới, hoặc cả nhóm tới đích. Nó chính là thứ định nghĩa bài toán: giữ lại một mình nó thôi thì chính sách tối ưu vẫn đúng là chính sách người thiết kế mong muốn. Cái dở là tín hiệu quá thưa. Giai đoạn đầu, khi tác nhân gần như chưa bao giờ tình cờ chạm tới cột mốc, gradient chính sách xấp xỉ không và quá trình học đứng im [25].

Lấp khoảng trống tín hiệu đó là việc của thành phần định hình F : nó cho điểm liên tục theo mức tiến bộ hướng về mục tiêu. Còn thành phần theo bước R_b gồm những khoản cộng trừ nhỏ phát sinh mỗi bước, chẳng hạn phạt thời gian hay thưởng cho việc duy trì một trạng thái có ích. Hai thành phần này khiến tín hiệu học dày đặc hơn hẳn. Nhưng chúng cũng chính là nơi ẩn chứa rủi ro lớn nhất của kỹ thuật thiết kế phần thưởng.

Rủi ro ấy tên là lợi dụng phần thưởng: tác nhân mò ra một hành vi tối đa hóa tổng phần thưởng mà chẳng hề làm cái nhiệm vụ người thiết kế nhắm tới. Ví dụ kinh điển nằm ở thí nghiệm điều khiển xe đạp của Randaløv và Alstrøm [20]: tác nhân được thưởng mỗi lần tiến lại gần đích đã học cách chạy vòng tròn quanh điểm xuất phát để liên tục gặt phần thưởng tiến gần, mà không bao giờ tới nơi. Amodei và cộng sự [1] xếp hiện tượng này vào nhóm vấn đề an toàn cốt lõi của học máy, và chỉ ra rằng nó phát sinh gần như tất yếu mỗi khi hàm phần thưởng chỉ là đại lượng thay thế cho mục tiêu thật. Đề tài đã gặp đúng hiện tượng đó ở phiên bản phần thưởng đầu tiên của môi trường Capture The Flag; Mục 3.4.7 thuật lại diễn biến cùng cách chẩn đoán.

May thay, có một khuôn khổ lý thuyết cho phép rải tín hiệu định hướng dày đặc mà vẫn an toàn, tức là kèm bảo đảm toán học rằng lời giải tối ưu không đổi. Đó là định hình phần thưởng theo thể năng của Ng và cộng sự [15], cơ sở lý thuyết chính mà đề tài dựa vào mỗi khi đặt các khoản thưởng dẫn đường.

Định nghĩa 3.5 (Định hình phần thưởng theo thế năng). Cho một hàm thế năng $\Phi : S \rightarrow \mathbb{R}$ gán cho mỗi trạng thái một giá trị vô hướng. Phần thưởng định hình được cộng thêm vào phép chuyển từ s sang s' có dạng

$$F(s, s') = \gamma \Phi(s') - \Phi(s).$$

Khi đó chính sách tối ưu của bài toán với phần thưởng $R + F$ trùng đúng với chính sách tối ưu của bài toán gốc với phần thưởng R ; nói cách khác, phần thưởng định hình dạng này không làm thay đổi lời giải tối ưu, mà chỉ định hướng lại quá trình học.

Chính tính chất bất biến này biến việc thêm tín hiệu định hướng từ một canh bạc thành thao tác an toàn. Người thiết kế cứ rải tín hiệu dày đặc mà không sợ làm lệch mục tiêu, và tiện thể loại luôn khả năng tác nhân lợi dụng phần thưởng bằng cách đi tới đi lui. Lý do nằm ở chỗ với mọi chu trình khép kín $s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_n = s_0$, tổng phần thưởng định hình thu về khi $\gamma \rightarrow 1$ bằng

$$\sum_{i=0}^{n-1} [\Phi(s_{i+1}) - \Phi(s_i)] = \Phi(s_n) - \Phi(s_0) = 0, \quad (3.2)$$

tức là mọi hành vi lặp đi lặp lại đều cho lợi ích ròng bằng không. Đẳng thức ấy loại bỏ kiểu lợi dụng phần thưởng của thí nghiệm xe đạp nói trên ngay từ trong thiết kế, không phải bằng cách mò mẫm dò hệ số. Với các bài toán điều hướng, hàm thế năng tự nhiên nhất là $\Phi(s) = -d(s)$, trong đó $d(s)$ là khoảng cách từ tác nhân tới mục tiêu; khi đó F thưởng mỗi bước làm giảm khoảng cách và phạt mỗi bước làm tăng khoảng cách. Đề tài vận dụng thẳng nguyên tắc này trong môi trường Capture The Flag, chi tiết ở Mục 3.4.

Gán công lao theo thời gian mới là một nửa câu chuyện. Môi trường hợp tác còn đặt thêm bài toán gán công lao giữa nhiều tác nhân: cả nhóm cùng nhận một phần thưởng chung thì làm sao biết từng tác nhân thực sự đóng góp bao nhiêu mà cập nhật chính sách cho đúng. Cơ chế phần thưởng nhóm cùng thuật toán MA-POCA [4] sinh ra để giải đúng bài toán ấy, dựa trên ý tưởng cơ sở phản thực của COMA [6]. Mối liên hệ giữa thiết kế phần thưởng nhóm và khả năng gán công lao sẽ rõ dần khi đi vào môi trường Capture The Flag.

Gom lại các cơ sở lý thuyết trên, đề tài áp dụng thống nhất cho cả ba môi trường bốn nguyên tắc thiết kế dưới đây. Mọi giá trị số xuất hiện trong các bảng phần thưởng ở những mục tiếp theo đều phải giải thích được bằng chính bốn nguyên tắc này:

- **Bám theo lời giải mẫu.** Trước khi đặt bất kỳ hệ số nào, người thiết kế phải mô tả được một quỹ đạo mẫu hoàn thành nhiệm vụ, rồi kiểm tra rằng không có thành phần phần thưởng nào phạt một trạng thái nằm trên quỹ đạo đó. Nguyên tắc này bắt nguồn từ nhận xét của Amodei và cộng sự [1] rằng phần lớn lỗi thiết kế phần thưởng đến từ việc mô tả sai mục tiêu chứ không từ việc chọn sai hệ số.
- **Ưu tiên sự kiện một lần hoặc thể năng thay vì cộng dồn theo bước.** Một khoản thưởng cộng dồn ở mỗi bước cho một trạng thái nào đó luôn tạo ra động cơ nấn ná tại trạng thái ấy. Khi cần thưởng cho một cột mốc, đề tài dùng phần thưởng sự kiện chỉ trao một lần trong mỗi lượt chơi; khi cần tín hiệu dẫn đường liên tục, đề tài dùng dạng thể năng ở Định nghĩa 3.5 với bảo đảm (3.2).
- **Giữ trật tự độ lớn.** Ký hiệu $|R_{mt}|$ là độ lớn của phần thưởng mục tiêu và Σ_b là tổng tuyệt đối tối đa mà các khoản theo bước có thể tích lũy trong trọn một lượt chơi, đề tài luôn giữ $\Sigma_b < |R_{mt}|$. Bất đẳng thức này bảo đảm rằng một lượt chơi hoàn thành nhiệm vụ luôn có tổng phần thưởng cao hơn mọi lượt chơi không hoàn thành, bất kể tác nhân xoay sở thế nào với các khoản phụ.
- **Sửa cơ chế môi trường trước khi tăng hệ số phần thưởng.** Khi một hành vi mục tiêu quá khó để khám phá bằng thử - sai, việc tăng hệ số thưởng cho nó không giúp ích, vì tác nhân chưa từng chạm tới hành vi ấy để nhận thưởng. Trong trường hợp đó đề tài điều chỉnh chính cơ chế môi trường hoặc dùng học theo chương trình [3] để hạ độ khó khám phá ban đầu.

Nguyên tắc giữ trật tự độ lớn được cụ thể hóa rất gọn: chia các khoản theo bước cho số bước tối đa N của một lượt chơi. Một khoản có giá trị c/N ở mỗi bước thì dù tác nhân bám lấy trạng thái đó suốt cả lượt, tổng thu về cũng chỉ đúng c . Chọn c nhỏ hơn $|R_{mt}|$ là lập tức thỏa mãn bất đẳng thức nói trên, đồng thời làm hệ số

không còn lệ thuộc vào độ dài lượt chơi. Hàm phần thưởng chi tiết của từng môi trường, kèm lý do đằng sau từng con số, nằm ở các mục tương ứng phía dưới.

3.2 Bộ môi trường thực nghiệm

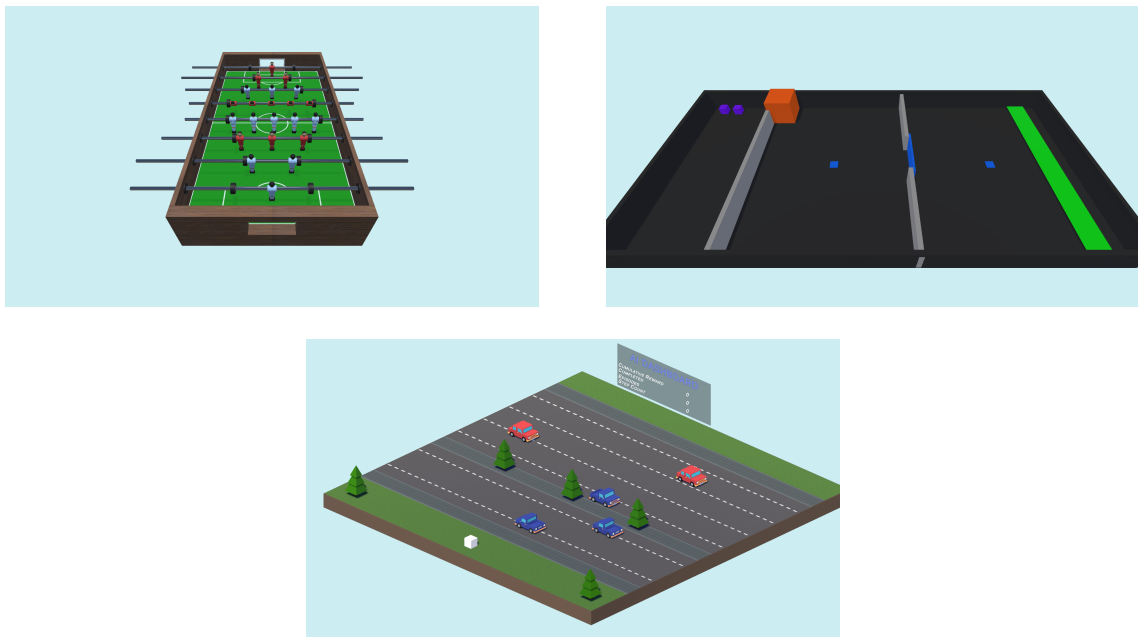
Bộ thực nghiệm của đề tài có ba môi trường ba chiều: Football Table, Capture The Flag và Cross The Road. Ba cái tên này được chọn để phủ ba dạng bài toán học tăng cường điển hình trong game - đối kháng một đối một, hợp tác đa tác nhân, điều hướng tránh vật cản động - và tiện thể phủ luôn cả hai dạng không gian hành động, liên tục lẫn rời rạc. Mục này nói về những quy ước dùng chung cho cả ba, kèm một bản tóm lược thống nhất về quan sát, hành động, phần thưởng và điều kiện kết thúc. Các mục sau đó mới đi sâu vào từng môi trường, theo cùng một bố cục.

Trừ những chỗ nói rõ khác đi, cả ba môi trường đều theo các quy ước sau. Mỗi môi trường là một cảnh Unity riêng, bên trong có đúng một khu vực huấn luyện đóng thành prefab để nhân bản được khi cần tăng tốc thu thập kinh nghiệm. Mỗi môi trường gắn với đúng một hành vi, nhờ đó một lần huấn luyện chỉ nạp một tệp cấu hình và không đụng chạm gì sang môi trường khác. Tác nhân quan sát hoàn toàn bằng vector số thực, không dùng quan sát hình ảnh; hành động thì do một mạng nơ-ron nhiều lớp kết nối đầy đủ tính ra. Cuối cùng, mọi lần huấn luyện đều chạy trên cùng một cấu hình máy, hệ số tăng tốc mô phỏng gấp hai mươi lần thời gian thực, bước vật lý cố định 0,02 giây - tức năm mươi bước vật lý mỗi giây mô phỏng.

3.2.1 Tích hợp với Unity ML-Agents

Nền tảng của cả ba môi trường là gói ML-Agents phiên bản 4.0.2 chạy trong Unity 6000.3, theo kiến trúc mô phỏng - huấn luyện đã trình bày ở Mục 2.5.1. Mỗi loại tác nhân có một lớp riêng kế thừa lớp Agent của ML-Agents, trong đó ba hàm quan trọng nhất được viết đề: thu thập quan sát, nhận hành động, và khởi tạo lại đầu lượt chơi. Ba lớp tác nhân lần lượt là FootballAgent, PuzzleAgent và CrossTheRoadAgent, gắn với ba hành vi Football, PuzzleBehavior và CrossTheRoad.

Bên cạnh quan sát do chính lớp tác nhân cung cấp, hai môi trường Capture The Flag và Cross The Road còn gắn thêm cảm biến tia; phần quan sát cảm biến sinh ra được ML-Agents ghép tự động vào vector quan sát, khỏi phải viết thêm dòng mã nào. Tần suất ra quyết định thì mỗi môi trường đặt riêng, qua số bước vật lý giữa hai lần quyết định. Ở những bước vật lý xen giữa, hành động cũ được giữ nguyên - nhờ vậy chuyển động vẫn liên tục mà chi phí suy luận của mạng nơ-ron giảm đi đáng kể. Huấn luyện xong, chính sách của mỗi môi trường được xuất ra tệp ONNX rồi nạp ngược vào Unity để chạy suy luận; đây cũng chính là dạng mô hình mà ứng dụng trình diễn ở Chương 5 dùng.



Hình 3.1: Ba môi trường thực nghiệm. Trên trái: Football Table, hai đội Xanh - Đỏ mỗi đội bốn thanh gạt. Trên phải: Capture The Flag, hai tác nhân phối hợp giữ cửa để cùng tới đích. Dưới: Cross The Road, một tác nhân băng qua các làn xe đang chạy

Tác nhân Football Table có tám hành động liên tục và sáu mươi quan sát. Hàm phần thưởng gồm ba khoản: một phần thưởng lớn khi ghi bàn, một hình phạt khi bị thùng lưới, và một hình phạt thời gian nhỏ ở mỗi bước để thúc tác nhân dứt điểm sớm. Lượt chơi kết thúc khi bóng vào một trong hai khung thành, hoặc khi chạm mốc ba nghìn bước - tương ứng khoảng một phút mô phỏng.

Tác nhân Capture The Flag có bảy hành động rời rạc và sáu mươi một quan

sát. Hàm phần thưởng ở đây nhiều thành phần hơn: một phần thưởng nhóm khi cả hai tác nhân cùng tới đích, một phần thưởng nhóm trao một lần cho mỗi tác nhân khi vượt được cổng đang khóa, một phần thưởng cá nhân định hình theo thể năng khoảng cách tới đích, một phần thưởng nhỏ khi đứng trên bàn đạp, và một hình phạt thời gian nhóm ở mỗi bước. Lượt chơi kết thúc khi cả hai tác nhân cùng chạm đích, hoặc khi khu vực huấn luyện chạy đủ một trăm nghìn bước vật lý.

Tác nhân Cross The Road có bốn hành động rời rạc và một trăm ba mươi hai quan sát. Hàm phần thưởng đơn giản nhất trong ba môi trường: một phần thưởng khi chạm vạch đích, một hình phạt nhỏ khi va chạm, cộng thêm tín hiệu tò mò cường độ thấp để khuyến khích khám phá. Lượt chơi kết thúc ngay khi tác nhân chạm vạch đích hoặc đâm phải xe, tường, cây; số bước thì không bị giới hạn.

Bảng 3.1 đặt cạnh nhau tốc độ huấn luyện thực đo của ba môi trường trên cùng một máy, mỗi môi trường lấy lần chạy PPO làm đại diện. Đáng chú ý là ba con số nằm khá sát nhau, từ năm trăm sáu mươi lăm tới bảy trăm bước mỗi giây, dù khối lượng vật lý của ba môi trường chênh nhau rất xa. Nguyên nhân nằm ở hai yếu tố tác động ngược chiều và gần như triệt tiêu lẫn nhau: khối lượng vật lý phải mô phỏng trong mỗi khu vực, và số bước vật lý giữa hai lần ra quyết định. Cross The Road nhanh nhất vì nó chỉ mô phỏng một khối tác nhân trượt theo ô lưới cùng năm chiếc xe chạy thẳng, dù mỗi quyết định trải trên năm bước vật lý. Football Table phải giải khớp vật lý của tám thanh gạt cộng va chạm liên tục của quả bóng, nhưng bù lại chỉ cách hai bước vật lý giữa hai quyết định. Còn Capture The Flag, vật lý nhẹ nhất trong ba môi trường, lại hóa ra chậm nhất - vì mỗi quyết định của tác nhân trải tới hai mươi bước vật lý và mỗi lượt chơi kéo dài tối đa một trăm nghìn bước.

Ba tốc độ sát nhau kéo theo một hệ quả thực tiễn: trên cả ba môi trường, một lần chạy của các thuật toán on-policy đều mất khoảng năm mươi phút. Vậy nên chênh lệch thời gian thực đáng kể mà Chương 4 ghi nhận là do bản thân thuật toán, cụ thể là chi phí cập nhật mạng của nhóm off-policy, chứ không phải do môi trường.

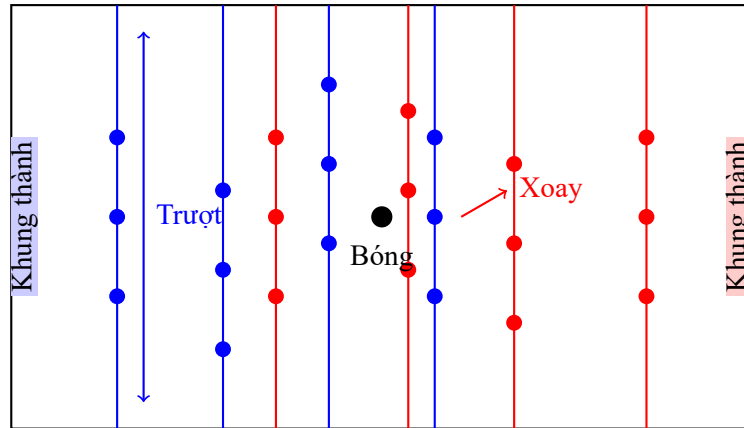
Bảng 3.1: Tốc độ huấn luyện thực đo trên ba môi trường, mỗi môi trường lấy một lần chạy đại diện; số bước là số bước huấn luyện của hành vi tương ứng

Môi trường	Số bước huấn luyện	Thời gian	Số bước mỗi giây	Quan sát	Hành động
Football Table	1.600.000	47 phút 0 giây	567	60	8 liên tục
Capture The Flag	1.700.000	50 phút 8 giây	565	61	7 rời rạc
Cross The Road	2.000.000	47 phút 52 giây	697	132	4 rời rạc

3.3 Môi trường Football Table

3.3.1 Mô tả bài toán

Football Table mô phỏng một bàn bi lắc ba chiều, có đầy đủ vật lý va chạm giữa quả bóng và các cầu thủ gắn trên thanh gạt. Bài toán ở đây là đối kháng một đối một: hai đội đối đầu trực tiếp, mỗi đội điều khiển bốn thanh gạt, vừa phải đưa bóng vào khung thành đối phương vừa phải giữ khung thành của mình. Hai thứ làm nên đặc trưng của nó là tính đối kháng và động lực học liên tục của quả bóng. Bóng lăn, nảy, đổi hướng không ngừng dưới tác động va chạm, nên tác nhân buộc phải phản ứng nhanh và phải nắm cả kỹ năng tấn công lẫn phòng thủ. Riêng tính đối kháng khiến đây thành trường hợp điển hình để kiểm chứng cơ chế tự chơi, nơi tác nhân tiến bộ bằng cách liên tục thi đấu với những phiên bản ngày một mạnh hơn của chính nó. Hình 3.2 thể hiện bố cục sân cùng hai bậc tự do của mỗi thanh gạt.



Hình 3.2: Sơ đồ bố cục sân và cơ chế điều khiển thanh gạt trong Football Table

3.3.2 Không gian quan sát

Vector quan sát của mỗi đội có sáu mươi giá trị, chia làm ba nhóm bổ sung cho nhau. Mười hai giá trị đầu mô tả trạng thái quả bóng: ba giá trị vận tốc đã nén về khoảng $[-1, 1]$ và chín giá trị mã hóa vị trí bóng. Ba mươi hai giá trị kế tiếp dành cho đội nhà - mỗi thanh gạt góp bốn đại lượng gồm vận tốc trượt, vận tốc xoay, vị trí trượt và góc xoay hiện tại, đủ để tác nhân biết chính xác tư thế của toàn bộ hàng cầu thủ bên mình. Mười sáu giá trị cuối mô tả đội đối phương bằng bốn đại lượng tương tự cho mỗi thanh gạt, giúp tác nhân đoán được ý đồ và phản ứng của đối thủ.

Có hai chi tiết trong cách mã hóa quan sát đáng dừng lại. Thứ nhất là kỹ thuật tách chữ số thập phân: thay vì đưa vào mạng một giá trị duy nhất cho vị trí bóng hay góc xoay thanh gạt, giá trị đó được tách thành nhiều thành phần ứng với các hàng thập phân liên tiếp. Nhờ vậy một thay đổi rất nhỏ ở hàng thập phân sâu vẫn tạo ra biến thiên rõ rệt ở một thành phần đầu vào, và mạng nhạy hơn hẳn với những sai lệch tinh vi về vị trí - thứ quyết định thành bại của một cú chạm bóng. Thứ hai là phép đối xứng hóa quan sát: mọi vị trí và vận tốc được đưa về hệ tọa độ địa phương của sân rồi nhân với một hệ số dấu tùy theo phía của đội. Sau biến đổi này, cả hai phía đều thấy khung thành đối phương ở cùng một hướng quy ước. Một chính sách duy nhất vì thế vận hành được cho cả hai phía, và cơ chế tự chơi trở nên nhất quán bởi hai đội thực sự dùng chung một mạng nơ-ron.

Các đại lượng vật lý trong môi trường này có thang đo chênh nhau nhiều bậc, nên chế độ chuẩn hóa quan sát được bật trong cấu hình huấn luyện, đúng theo nguyên tắc ở Mục 3.1.2.

3.3.3 Không gian hành động và cơ chế điều khiển

Mỗi đội chỉ có một tác nhân duy nhất, quản đồng thời cả bốn thanh gạt. Ở mỗi bước quyết định, tác nhân xuất ra tám giá trị thực nằm trong khoảng $[-1, 1]$. Mỗi thanh gạt chiếm hai giá trị: một điều khiển chuyển động trượt ngang để đưa cầu thủ vào đúng vị trí, một điều khiển chuyển động xoay để thực hiện cú đá:

$$a = (f_1^{\text{trượt}}, f_1^{\text{xoay}}, \dots, f_4^{\text{trượt}}, f_4^{\text{xoay}}) \in [-1, 1]^8. \quad (3.3)$$

Gộp cả đội vào một tác nhân duy nhất, thay vì mỗi thanh gạt một tác nhân, đem lại hai cái lợi. Một, bài toán phối hợp giữa bốn thanh gạt trở thành bài toán điều khiển tập trung mà một chính sách duy nhất học được, khỏi phải xoay xở với chuyện đồng bộ nhiều tác nhân độc lập. Hai, nó khớp thẳng với cơ chế tự chơi, bởi khi ấy mỗi phía của bàn chỉ cần đúng một chính sách để thi đấu.

Về mặt vật lý, một giá trị hành động không dịch chuyển thanh gạt tức thời. Nó được diễn giải thành mức thay đổi vận tốc trượt hoặc vận tốc xoay, đặt lên thân vật lý của thanh gạt tương ứng với hệ số lần lượt là một và hai mươi lăm; chuyển động cuối cùng, cùng va chạm giữa thanh gạt và quả bóng, do hệ thống vật lý Unity quyết định. Tác nhân ra quyết định sau mỗi hai bước vật lý, tức hai mươi lăm quyết định mỗi giây mô phỏng, và hành động giữ nguyên ở bước vật lý xen giữa. Chính tính liên tục ấy biến Football Table thành phép thử tiêu biểu cho các thuật toán điều khiển liên tục, khác hẳn hai môi trường rời rạc còn lại.

3.3.4 Hàm phần thưởng

Bảng 3.2 tổng hợp hàm phần thưởng của môi trường. Chiếu vào khuôn dạng (3.1), ở đây chỉ có hai lớp được dùng. Thành phần mục tiêu thừa tại trạng thái kết thúc: cộng năm điểm khi ghi bàn, trừ một điểm khi bị thủng lưới. Thành phần theo bước: hình phạt thời gian $1/3000$ điểm mỗi bước. Thành phần định hình theo thể năng thì không dùng, vì lý do sẽ nói ở cuối mục này.

Đặt phần thưởng ghi bàn lớn gấp năm lần hình phạt thủng lưới không phải một lựa chọn tùy hứng, mà xuất phát từ đặc thù của bài toán đối kháng. Với hàm phần thưởng đối xứng, chính sách an toàn nhất là ôm bóng ở phần sân nhà và không bao giờ dâng lên, bởi mọi pha tấn công đều mở ra nguy cơ bị phản công. Nhận định này lượng hóa được. Hãy xét tình huống tác nhân phải chọn giữa cầm cự, tức giữ bóng

tới hết lượt chơi, và dứt điểm, tức tổ chức một pha tấn công thành công với xác suất p và bị phản công thủng lưới với xác suất $1 - p$. Gọi R_+ là phần thưởng ghi bàn, R_- là hình phạt thủng lưới, và bỏ qua hình phạt thời gian ở nhánh dứt điểm vì lượt chơi kết thúc sớm; giá trị kỳ vọng của hai lựa chọn là

$$V_{\text{cầm cự}} = -T \cdot \frac{1}{3000} = -1, \quad V_{\text{dứt điểm}} = p R_+ - (1 - p) R_-, \quad (3.4)$$

với $T = 3000$ là số bước tối đa của một lượt chơi. Lối chơi tấn công chỉ được ưa chuộng khi $V_{\text{dứt điểm}} > V_{\text{cầm cự}}$, tức khi

$$p > \frac{R_- - 1}{R_+ + R_-}. \quad (3.5)$$

Thử cấu hình đối xứng $R_+ = R_- = 5$ thì ngưỡng này bằng 0,4, nghĩa là tác nhân chỉ chịu tấn công khi tự tin thắng ít nhất bốn mươi phần trăm số pha bóng. Ở giai đoạn đầu huấn luyện, điều đó gần như không bao giờ đúng, nên chính sách hội tụ về phòng ngự tuyệt đối. Đổi sang cấu hình được chọn, $R_+ = 5$ và $R_- = 1$, ngưỡng tụt xuống đúng bằng không: mọi cơ hội ghi bàn dù nhỏ đến mấy cũng đáng thử. Tỷ lệ nằm trên một ra đời từ đó. Phép tính này còn cho thấy hình phạt thời gian chẳng phải chi tiết phụ, mà là thành phần quyết định dấu của vế phải trong (3.5).

Con số $1/3000$ của hình phạt thời gian đến từ nguyên tắc giữ trật tự độ lớn ở Mục 3.1.4: nhân với $T = 3000$ bước, tổng phạt tích lũy trọn một lượt chơi vừa đúng một điểm, tức bằng đúng hình phạt thủng lưới. Cách chuẩn hóa này để lại một hệ quả có chủ đích. Một lượt chơi hòa không bàn thắng bị đánh giá ngang với việc để thủng lưới một bàn, nên tác nhân không thể xem thế trận bé tắc là kết quả chấp nhận được. Trong khi đó tổng phạt vẫn nhỏ hơn phần thưởng ghi bàn rất nhiều, nên nó chẳng bao giờ lấn át được mục tiêu chính.

Bảng 3.2: Hàm phần thưởng của môi trường Football Table

Sự kiện	Giá trị	Loại
Ghi bàn vào khung thành đối phương	+5,0	Thưởng mục tiêu
Bị thủng lưới	-1,0	Phạt mục tiêu
Mỗi bước trôi qua	-1/3000	Phạt thời gian

Mã nguồn của tác nhân còn cài sẵn hai thành phần định hướng tùy chọn. Một là phần thưởng tỉ lệ với thành phần vận tốc của bóng chiếu lên hướng nối tới khung thành đối phương, chỉ kích hoạt khi tác nhân đang kiểm soát bóng. Hai là hình phạt khi thanh gạt xoay quá mạnh. Ở toàn bộ các lần huấn luyện báo cáo trong Chương 4, hai hệ số này đều bằng không, tức cả hai thành phần đứng ngoài tín hiệu học. Có ba căn cứ cho quyết định đó.

Trước hết, phần thưởng theo vận tốc bóng không mang dạng thế năng. Nó phụ thuộc vào vận tốc - một đại lượng của phép chuyển, chứ không phải hàm của riêng trạng thái - nên không viết được thành $\gamma \Phi(s') - \Phi(s)$, và vì thế nằm ngoài phạm vi bảo đảm bất biến của Định nghĩa 3.5. Đẳng thức (3.2) lập tức mất hiệu lực: tác nhân dư sức đẩy bóng tới lui theo hướng khung thành đối phương để gạt phần thưởng vô hạn mà chẳng bao giờ dứt điểm, đúng cái cơ chế đã thấy trong thí nghiệm xe đạp của Randløv và Alstrøm [20].

Kế đến, hình phạt xoay mạnh vi phạm nguyên tắc bám theo lời giải mẫu. Một cú sút mạnh nhất thiết phải đi kèm vận tốc xoay lớn của thanh gạt, nên khoản phạt này giáng thẳng vào hành động nằm trên quỹ đạo hoàn thành nhiệm vụ. Để nó khác không thì tác nhân sẽ trôi về lối chơi rê bóng chậm chạp thay vì dứt điểm.

Căn cứ cuối cùng: cơ chế tự chơi tự nó đã là một chương trình học có độ khó tăng dần, chẳng cần tín hiệu định hướng thủ công nào. Đối thủ luôn mạnh tương đương tác nhân ở thời điểm hiện tại, nên tỉ lệ thắng thua giữ được mức cân bằng và tín hiệu thua vẫn xuất hiện đủ thường xuyên để việc học tiến triển. Lập luận này đã được kiểm chứng ở quy mô lớn hơn nhiều trong AlphaGo Zero [24], nơi một chính sách mạnh hơn con người được học chỉ từ tín hiệu thắng thua ở cuối ván, không hề có phần thưởng trung gian nào. Hai thành phần vẫn được giữ nguyên trong mã nguồn để có thể bật lại khi cần khảo sát ảnh hưởng của phần thưởng định hướng.

3.3.5 Điều kiện kết thúc lượt chơi

Lượt chơi khép lại khi có bàn thắng, tức khi bóng đi vào một trong hai khung thành. Lúc đó cả hai tác nhân nhận phần thưởng tương ứng, rồi lượt chơi được đặt lại với bóng và các thanh gạt trở về vị trí ban đầu. Trường hợp thứ hai là chạm mốc ba nghìn bước, tương ứng khoảng một phút mô phỏng. Giới hạn này chặn những thế

trận bế tắc kéo dài vô hạn và giữ cho dữ liệu huấn luyện luôn được làm mới.

3.3.6 Cấu hình huấn luyện

Bản chất đối kháng cộng không gian hành động liên tục khiến PPO kết hợp tự chơi trở thành lựa chọn huấn luyện chính cho môi trường này. Mạng chính sách có hai lớp ẩn, mỗi lớp hai trăm năm mươi sáu đơn vị, và bật chuẩn hóa quan sát. Cơ chế tự chơi được cấu hình như sau: lưu một phiên bản đối thủ sau mỗi năm mươi nghìn bước, đổi phía sau mỗi một trăm nghìn bước, giữ cửa sổ mười đối thủ gần nhất với xác suất đấu phiên bản mới nhất là một nửa, và theo dõi tiến bộ qua chỉ số xếp hạng khởi tạo ở mức một nghìn hai trăm. Số bước huấn luyện đặt ở một triệu sáu trăm nghìn, dùng chung cho cả năm thuật toán trên môi trường này. Bốn thuật toán còn lại - SAC, MA-POCA, MAPPO và DQN - đều huấn luyện dưới cùng điều kiện để mặt bằng so sánh được công bằng. Riêng DQN không ghép nối với cơ chế tự chơi nên phải chạy trên một bản dựng có không gian hành động rời rạc, như đã nêu ở Mục 3.8.3. Cấu hình siêu tham số đầy đủ nằm trong các Phụ lục A đến D.

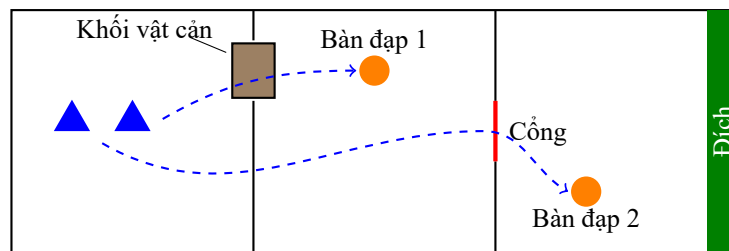
3.4 Môi trường Capture The Flag

3.4.1 Mô tả bài toán

Capture The Flag là một câu đố không gian mang tính hợp tác: hai tác nhân phải phối hợp chặt chẽ mới cùng về đích được. Mấu chốt nằm ở chỗ nhiệm vụ được thiết kế sao cho không tác nhân nào tự mình hoàn thành nổi. Muốn đi từ khu vực xuất phát tới đích, cả hai buộc phải thực hiện một màn bàn giao, trong đó lần lượt mỗi tác nhân giữ cửa mở cho tác nhân kia đi qua. Sự phụ thuộc lẫn nhau ấy biến bài toán thành trường hợp điển hình để làm nổi thách thức gán công lao trong học tăng cường hợp tác - tức việc xác định từng tác nhân thực sự đóng góp bao nhiêu vào thành quả chung. Và đây cũng đúng là dạng bài toán mà MA-POCA sinh ra để giải.

Hình 3.3 thể hiện bố cục không gian: một hành lang dài trải theo một trục, chia thành ba khoang bởi hai bức tường ngăn có khe hở lệch nhau. Khoang đầu là phòng xuất phát, nơi đặt sẵn hai tác nhân và một khối vật cản; lối ra duy nhất là khe hở nằm lệch về một bên của bức tường ngăn thứ nhất, và ngay từ đầu lượt chơi khe

hở ấy bị chính khối vật cản bịt kín, nên muốn rời phòng thì trước hết phải đẩy khối đi chỗ khác. Khoang giữa chứa bàn đạp áp lực thứ nhất. Ngăn khoang giữa với khoang cuối là một cổng cửa đặt ở khe hở chính giữa bức tường ngăn thứ hai, gồm hai cánh có thể hạ xuống để mở hoặc nâng lên để chặn. Khoang cuối có bàn đạp áp lực thứ hai, và ở tận cùng là vách đích trải suốt chiều rộng hành lang. Điểm thiết kế cốt lõi nằm ở chỗ hai bàn đạp cùng điều khiển một cổng cửa duy nhất, nhưng lại nằm ở hai phía đối diện của cổng đó - bàn đạp thứ nhất ở phía trong, cùng khoang với điểm xuất phát; bàn đạp thứ hai ở phía ngoài, đã qua cổng. Chính cách bố trí đối xứng ấy sinh ra cấu trúc bàn giao mà lời giải buộc phải tuân theo.



Hình 3.3: Sơ đồ bố cục không gian và trình tự bàn giao trong Capture The Flag

3.4.2 Cửa ải khối vật cản

Khối vật cản trong phòng xuất phát không phải vật trang trí, mà là chương ngại bắt buộc đầu tiên của lượt chơi. Khe hở của bức tường ngăn thứ nhất rộng chừng bảy phẩy sáu đơn vị, tính từ mặt trong của tường biên tới đầu mút bức tường ngăn. Khối vật cản là một hình lập phương cạnh bảy đơn vị, khối lượng một trăm đơn vị, đặt nằm gọn trong khe hở ấy và chỉ chừa lại hai kẽ trống rộng 0,25 và 0,36 đơn vị ở hai bên, trong khi bề ngang của tác nhân là hai đơn vị. Vậy nên không tồn tại quỹ đạo nào lách qua được: muốn ra khỏi phòng xuất phát, tác nhân buộc phải đẩy khối trượt dọc theo khe cho tới khi lối đi hở ra. Đây cũng là lý do khối vật cản có mặt trong danh sách nhân của cảm biến tia ở Mục 3.4.3 - tác nhân cần phân biệt được nó với tường để biết chỗ nào đẩy được, chỗ nào không.

Có một điều phải nói rõ để tránh hiểu nhầm về mức độ ràng buộc của cửa ải này: nó không ép buộc hợp tác theo nghĩa chặt như cổng cửa. Bởi lệnh di chuyển cộng thẳng một lượng vận tốc vào thân vật lý chứ không tác dụng một lực hữu hạn, khả năng đẩy của tác nhân không bị khối lượng của khối chặn lại. Mỗi bước vật

lý, một tác nhân nặng mười đơn vị khối lượng truyền vào hệ một xung lượng bằng mười bốn; chia cho tổng khối lượng của hệ tác nhân cộng khối là một trăm mười, hệ nhận thêm chừng 0,13 đơn vị vận tốc. Hai tác nhân cùng đẩy thì con số ấy là hai mươi tám chia cho một trăm hai mươi, tức khoảng 0,23 - nhanh gần gấp đôi. Nói cách khác, một tác nhân đơn độc vẫn dọn được lối đi, chỉ là chậm hơn hẳn. Thêm nữa, khối chỉ bị khóa xoay quanh hai trục ngang mà vẫn xoay tự do quanh trục đứng, nên một tác nhân đẩy lệch tâm dễ làm khối xoay chéo rồi kẹt trong khe, còn hai tác nhân đẩy hai bên thì khối đi thẳng. Cửa ải này vì thế chỉ tạo ra một sức ép hợp tác mềm: hợp tác thì nhanh hơn và ít rủi ro hơn, nhưng không hợp tác vẫn qua được. Ràng buộc hợp tác cứng, tức thứ không có lời giải cá nhân nào, nằm ở cơ chế cổng cửa trình bày tại Mục 3.4.5.

Vai trò thiết kế của khối vì vậy là một cửa ải khởi động đặt trước pha bàn giao thật sự. Nó tách lượt chơi thành hai giai đoạn có yêu cầu khác hẳn nhau, và quan trọng hơn là đặt một việc phải làm ngay trong tầm với của chính sách ngẫu nhiên lúc khởi đầu: tác nhân chưa biết gì về bàn đạp hay cổng vẫn có lý do va vào khối rồi trôi ra khỏi phòng xuất phát, thay vì lang thang trong một căn phòng kín chằng có tín hiệu nào. Nó cũng giữ hai tác nhân ở gần nhau trong những bước đầu, nhờ đó cấu hình cần thiết cho màn bàn giao có cơ hội xuất hiện sớm hơn trong quá trình khám phá.

3.4.3 Không gian quan sát

Mỗi tác nhân quan sát sáu mươi một giá trị, đến từ ba nguồn bổ sung cho nhau. Bốn giá trị đầu là cờ nhị phân mang tính logic: bàn đạp thứ nhất đã nhấn chưa, bàn đạp thứ hai đã nhấn chưa, bản thân tác nhân đã vượt cổng chưa, và đồng đội đã vượt cổng chưa. Hai cờ đầu cho tác nhân biết cửa có đang được giữ mở hay không mà chẳng cần nhìn thấy cổng. Hai cờ sau mới là mấu chốt: chúng cho tác nhân biết khi nào nên rời bàn đạp, bởi chỉ khi đồng đội đã sang bên kia thì nhà bàn đạp mới hợp lý. Thiếu cờ trạng thái của đồng đội, chính sách phi tập trung mất hẳn căn cứ trực tiếp để định thời điểm bàn giao, và bài toán khó học hơn hẳn. Kèm bốn cờ này còn một cờ nhị phân riêng ghi nhận tác nhân đã chạm đích hay chưa.

Năm mươi sáu giá trị còn lại do cảm biến tia sinh ra, theo mô tả ở Mục 3.1.2.

Cấu hình cụ thể: bảy tia trải trong góc quạt bảy mươi độ phía trước, tầm xa hai mươi đơn vị, nhận diện sáu nhân vật thể gồm đồng đội, tường, khối vật cản, bàn đạp, cửa và đích. Mỗi tia cho ra tám giá trị - sáu giá trị mã hóa một-nóng cho nhân, một giá trị báo tia có chạm vật thể nào không, một giá trị khoảng cách chuẩn hóa. Cảm biến này giúp tác nhân nắm được cấu trúc không gian xung quanh lần vị trí tương đối của đồng đội trong tầm nhìn, mà không ai phải đưa cho nó tọa độ tuyệt đối. Ghép các cờ logic tường minh với quan sát không gian, tác nhân vừa suy luận được về trạng thái phối hợp của cả nhóm, vừa điều hướng chính xác trong hành lang hẹp.

3.4.4 Không gian hành động và cơ chế điều khiển

Mỗi tác nhân điều khiển một khối hộp có thân vật lý, di chuyển trong không gian ba chiều bằng bảy hành động rời rạc: đứng yên, tiến, lùi, xoay trái, xoay phải, đi ngang sang trái, đi ngang sang phải. Chừng đó đủ để tác nhân vừa di chuyển linh hoạt vừa định hướng chính xác khi cần đứng vào đúng vị trí bàn đạp. Như Mục 3.1.3 đã nêu, một lệnh di chuyển không dịch tác nhân tức thời; nó cộng thêm 1,4 đơn vị trên giây vào thân vật lý theo hướng tương ứng ở mỗi bước vật lý, còn hệ số cản tuyến tính đặt ở mức bốn thì liên tục hãm vận tốc lại. Hai lệnh xoay quay tác nhân quanh trục đứng với tốc độ hai trăm độ mỗi giây. Tác nhân ra quyết định sau mỗi hai mươi bước vật lý, tức hai phẩy năm quyết định mỗi giây mô phỏng, và hành động giữ nguyên ở các bước vật lý xen giữa.

Chính thể cân bằng giữa lượng vận tốc cộng thêm và hệ số cản khiến tác nhân có vận tốc tối đa hữu hạn thay vì tăng tốc vô hạn. Xuất phát từ trạng thái dừng, vận tốc tối đa đạt được là

$$v_{\max} = \frac{\Delta v}{c \cdot \Delta t} = \frac{1,4}{4 \times 0,02} = 17,5 \text{ đơn vị/giây}, \quad (3.6)$$

với Δv là lượng vận tốc cộng thêm mỗi bước, c là hệ số cản và Δt là độ dài một bước vật lý. Ngoài ý nghĩa vật lý thuần túy, giá trị này còn là yếu tố then chốt bảo đảm tính đúng đắn của cơ chế bàn giao, như sẽ phân tích ngay dưới đây.

3.4.5 Cơ chế cửa và pha bàn giao

Cổng cửa chỉ mở chừng nào còn tác nhân đứng trên một trong hai bàn đạp. Nhưng nếu cửa sập xuống ngay khoảnh khắc tác nhân rời bàn đạp, màn bàn giao gần như

không thể khám phá nổi bằng thử - sai, vì nó đòi hỏi đồng bộ thời gian hoàn hảo. Để hành vi hợp tác trở nên khả học, đề tài thêm vào một khoảng ân hạn: tác nhân rời bàn đạp rồi thì cửa vẫn mở thêm một giây mô phỏng nữa mới đóng. Cửa sổ thời gian ấy đủ để đồng đội kịp lên giữ bàn đạp còn lại, làm mượt quá trình học mà không đọng gì tới bản chất bài toán.

Có một câu hỏi thiết kế phải trả lời ở đây: liệu khoảng ân hạn có vô tình mở đường cho một tác nhân tự mình vượt cổng không? Nghĩa là bước lên bàn đạp thứ nhất rồi phóng thật nhanh qua cổng trước khi cửa kịp đóng, khiến hợp tác thành thừa thãi. Một lập luận định lượng loại trừ được khả năng đó. Khoảng cách từ bàn đạp thứ nhất tới cổng là mười chín đơn vị, còn khoảng ân hạn chỉ dài một giây. Cứ giả định rộng rãi rằng tác nhân đã chạy sẵn ở vận tốc tối đa 17,5 đơn vị trên giây theo công thức (3.6), quãng đường nó đi được trong một giây vẫn chưa tới mười chín đơn vị. Mà thực tế thì tác nhân xuất phát từ trạng thái đứng yên trên bàn đạp, còn phải mất thời gian tăng tốc, nên quãng đường thực chỉ khoảng mười ba phẩy hai đơn vị - chưa nổi bảy mươi phần trăm khoảng cách cần vượt. Vậy là không tồn tại quỹ đạo nào cho phép một tác nhân đơn độc vượt cổng trong khoảng ân hạn. Kết luận đó bảo đảm mọi lần vượt cổng thành công đều là bằng chứng của hợp tác thật, không phải của một lối tắt cá nhân; nó cũng là cơ sở để đọc đúng các chỉ số phần thưởng ở Chương 4.

3.4.6 Hàm phần thưởng

Hàm phần thưởng ở đây phức tạp nhất trong ba môi trường của đề tài. Đây cũng là môi trường duy nhất phải thiết kế lại sau khi phiên bản đầu tiên thất bại. Mục này trình bày thiết kế cuối cùng kèm lý do của từng thành phần; còn quá trình đi tới thiết kế đó thì Mục 3.4.7 thuật lại.

Mọi thứ khởi đầu từ lời giải mẫu, đúng theo nguyên tắc bám theo lời giải mẫu ở Mục 3.1.4. Với bố cục đã mô tả, một lượt chơi thành công bắt buộc đi qua chuỗi trạng thái này: khối vật cản bị đẩy khỏi khe hở của bức tường ngăn thứ nhất; hai tác nhân rời phòng xuất phát; một tác nhân bước lên bàn đạp phía trong rồi đứng yên giữ cửa; tác nhân còn lại vượt cổng; tác nhân vừa vượt cổng bước lên bàn đạp phía ngoài để giữ cửa ngược lại; tác nhân đang ở bàn đạp phía trong rời chỗ và

vượt công; cuối cùng cả hai cùng tiến tới vạch đích. Mỗi thành phần phần thưởng đều được soi chiếu vào chuỗi này, với một yêu cầu bắt buộc: không thành phần nào được phạt một trạng thái nằm trên chuỗi. Yêu cầu tương tự như hiển nhiên ấy chính là thứ mà phiên bản phần thưởng đầu tiên vi phạm.

Phần thưởng mục tiêu. Khoản lớn nhất, bằng một điểm, chỉ trao khi cả hai tác nhân cùng đến được đích - và trao cho nhóm chứ không cho cá nhân. Chọn phạm vi nhóm là quyết định có hệ quả sâu xa: hàm giá trị mà mạng phê bình phải học trở thành giá trị của cả đội, nhờ đó một tác nhân đứng yên giữ bàn đạp vẫn được ghi công khi đồng đội về đích. Ngược lại, nếu trao theo cá nhân, tác nhân giữ cửa chẳng nhận được gì cho hành vi hy sinh của mình, và chính sách sẽ nhanh chóng học cách bỏ mặc đồng đội. Cấu hình phần thưởng nhóm này chính là điều kiện để các thuật toán gán công lao đa tác nhân như MA-POCA [4] phát huy tác dụng, bởi phê bình tập trung của chúng vốn sinh ra để phân rã một phần thưởng chung thành đóng góp của từng tác nhân [6].

Phần thưởng định hình theo khoảng cách. Thành phần dẫn đường đúng đắn theo khuôn khổ ở Định nghĩa 3.5, lấy hàm thế năng bằng đối số của khoảng cách từ tác nhân tới đích, tức $\Phi(s) = -d(s)$. Phần thưởng cộng thêm mỗi bước do đó có dạng

$$F(s_{t-1}, s_t) = \kappa [d(s_{t-1}) - d(s_t)], \quad \kappa = 0,01, \quad (3.7)$$

nghĩa là thưởng cho mỗi bước rút ngắn khoảng cách và phạt cho mỗi bước nói rộng nó. Cài đặt dùng hiệu thế năng thuần túy chứ không dùng dạng chiết khấu $\gamma \Phi(s') - \Phi(s)$; với $\gamma = 0,99$ trong cấu hình huấn luyện, sai khác giữa hai dạng là $(1 - \gamma)\Phi$ - nhỏ hơn hai bậc so với chính tín hiệu dẫn đường - nên tính bất biến chính sách vẫn giữ được ở mức xấp xỉ chấp nhận được [5]. Lựa chọn này kéo theo ba hệ quả đáng nêu. Một, tín hiệu dẫn đường phủ kín cả hành lang, kể cả những đoạn chẳng có cột mốc nào, nên tác nhân không bao giờ lọt vào vùng hoàn toàn trống gradient học. Hai, đẳng thức (3.2) bảo đảm mọi hành vi đi tới đi lui đều cho lợi ích rỗng bằng không, nên thành phần này miễn nhiệm với việc bị lợi dụng. Ba, và quan trọng nhất với bài toán hợp tác: một tác nhân đứng yên giữ bàn đạp có $d(s_t) = d(s_{t-1})$ nên không bị thành phần này phạt - hành vi hy sinh nằm trên lời giải mẫu được giữ trung lập thay vì bị trừng phạt.

Tổng phần thưởng định hình mà một tác nhân thu về trong trọn một lượt chơi hoàn toàn do tính chất khớp nối của tổng (3.2) quyết định:

$$\sum_{t=1}^T F(s_{t-1}, s_t) = \kappa [d(s_0) - d(s_T)] \leq \kappa d(s_0), \quad (3.8)$$

nghĩa là nó chỉ phụ thuộc khoảng cách ban đầu, chứ không dính dáng gì tới độ dài lượt chơi hay quỹ đạo mà tác nhân chọn. Hệ số $\kappa = 0,01$ được chọn theo đúng ràng buộc đó: muốn thành phần dẫn đường không lấn át phần thưởng mục tiêu thì cần $\kappa d(s_0) < 1$, tức $d(s_0) < 100$ đơn vị - điều mà chiều dài hành lang của bố cục hiện tại thỏa mãn với biên độ an toàn rất rộng.

Phần thưởng bàn giao. Thành phần hợp tác then chốt được trao mỗi khi một tác nhân vừa vượt qua cổng đang khóa: cả nhóm cộng nửa điểm, và mỗi tác nhân chỉ nhận khoản này đúng một lần trong mỗi lượt chơi. Ba chi tiết trong định nghĩa ấy đều có lý do riêng. Điều kiện cổng đang khóa, ghép với kết luận định lượng ở Mục 3.4.5 rằng không tồn tại quỹ đạo nào cho một tác nhân đơn độc vượt cổng, khiến sự kiện kích hoạt phần thưởng tương đương về mặt logic với sự kiện hợp tác thành công. Tín hiệu vì thế không còn là một đại lượng thay thế gần đúng cho hành vi mong muốn, mà là chỉ báo chính xác của nó - tránh được đúng cái bẫy mà Amodei và cộng sự [1] cảnh báo. Phạm vi nhóm thì bảo đảm tác nhân đang giữ bàn đạp cũng được ghi công cho cú vượt cổng của đồng đội. Còn đặc tính một lần khiến khoản thưởng không thể bị khai thác bằng cách ra vào cổng lặp đi lặp lại, khác hẳn một khoản cộng dồn theo từng bước; đây chính là bài học rút ra từ phiên bản thiết kế đầu tiên.

Các khoản theo bước. Tác nhân đang đứng trên bàn đạp nhận một khoản thưởng nhỏ, và việc đẩy khối vật cản cũng vậy. Hai khoản này cố tình đi ngược nguyên tắc ưu tiên sự kiện một lần, và lý do là khám phá. Giữ bàn đạp là hành vi phải duy trì theo thời gian nên không cách nào diễn đạt bằng một sự kiện tức thời; mà nếu chẳng có tín hiệu nào cho nó thì xác suất tác nhân tình cờ đứng yên đủ lâu trên bàn đạp là quá nhỏ. Rủi ro nản ná được không chế bằng cách chuẩn hóa theo độ dài lượt chơi: khoản thưởng chỉ $0,5/N$ mỗi bước, nên tác nhân có đứng lì trên bàn đạp suốt lượt chơi thì tổng thu cũng vừa đúng 0,5 điểm - thua xa mức 1,5 điểm nếu nó tham gia trọn vẹn một lời giải bàn giao. Khoản thưởng cho việc đẩy khối vật

cản đi theo đúng logic ấy, nhưng kèm một lựa chọn cài đặt cần nói rõ: nó được trao ở mỗi bước vật lý mà tác nhân còn tiếp xúc với khối, chứ không trao theo quãng đường khối dịch chuyển. Cách đo theo quãng đường thoát nghe sát mục tiêu hơn, song nó phụ thuộc vào phép chuyển chứ không phải hàm của riêng trạng thái, tức không mang dạng thế năng của Định nghĩa 3.5, nên tác nhân có thể đẩy khối tới lui để gặt phần thưởng vô hạn - đúng cơ chế đã khiến đề tài tắt phần thưởng theo vận tốc bóng ở Mục 3.3. Đo theo tiếp xúc thì tín hiệu bị chặn cứng bởi độ dài lượt chơi: chỉ $0,25/N$ mỗi bước, nên tác nhân có tì vào khối suốt lượt chơi cũng chỉ thu về 0,25 điểm. Cái giá phải trả là tín hiệu này đo tiếp xúc chứ không đo tiến bộ, nghĩa là tì vào một khối đã kẹt cứng vẫn được thưởng; với mức trần bằng một phần tư điểm, sai lệch đó vô hại so với hai điểm của các khoản mục tiêu, nên đề tài chấp nhận đánh đổi để lấy tính không thể khai thác. Sau cùng là khoản phạt thời gian nhóm $0,1/N$ mỗi bước, để thúc nhóm hoàn thành nhanh hơn. Giá trị này cố ý giữ rất nhỏ, tổng phạt trọn lượt chơi chỉ 0,1 điểm: đủ phá thế hòa giữa hai chính sách cùng hoàn thành nhiệm vụ, nhưng không đủ để chi phối tín hiệu học - đúng cái sai lầm mà phiên bản đầu tiên đã mắc.

Bảng 3.3: Hàm phần thưởng của môi trường Capture The Flag

Sự kiện	Giá trị	Phạm vi
Cả hai tác nhân đến đích	+1,0	Nhóm
Vượt được cổng đang khóa (một lần mỗi tác nhân)	+0,5	Nhóm
Giảm khoảng cách tới đích (định hình thế năng)	$0,01 \times \Delta d$	Cá nhân
Đứng trên bàn đạp áp lực	$+0,5/N$ mỗi bước	Cá nhân
Đẩy khối vật cản	$+0,25/N$ mỗi bước	Cá nhân
Phạt thời gian mỗi bước	$-0,1/N$	Nhóm

trong đó Δd là mức giảm khoảng cách từ tác nhân tới đích giữa hai bước liên tiếp, và $N = 100\,000$ là số bước tối đa của một lượt chơi, dùng để giữ tổng phần thưởng tích lũy ở một thang đo hợp lý.

Xếp cạnh nhau, các thành phần trên tạo nên một trật tự độ lớn được kiểm soát

rất chặt. Một lượt chơi hoàn thành trọn vẹn mang về cho nhóm một điểm ở mục tiêu cuối, cộng tối đa một điểm cho hai lần vượt công, cộng phần định hình vốn đã bị chặn trên bởi $\kappa d(s_0)$. Trong khi đó toàn bộ các khoản theo bước mà một tác nhân có thể thu về, dù nó xoay xở kiểu gì, cũng chỉ nằm trong khoảng từ $-0,1$ tới $+0,75$ điểm. Bất đẳng thức $\Sigma_b < |R_{mt}|$ của nguyên tắc giữ trật tự độ lớn nhờ vậy được thỏa mãn, và không tồn tại chính sách nào vừa không hoàn thành nhiệm vụ vừa thu được tổng phần thưởng cao hơn một chính sách hoàn thành nhiệm vụ. Phiên bản phần thưởng đầu tiên thiếu đúng tính chất này.

3.4.7 *Quá trình hiệu chỉnh hàm phần thưởng*

Thiết kế vừa trình bày không phải phương án đầu tiên, mà là kết quả của một vòng hiệu chỉnh. Kỹ thuật thiết kế phần thưởng vốn là phần khó nhất và cũng mang tính phương pháp luận nhất của đề tài, nên mục này ghi lại trung thực phiên bản thất bại cùng cách chẩn đoán nó - qua đó thấy được các nguyên tắc ở Mục 3.1.4 rút ra từ đâu.

Phiên bản đầu tiên đi theo trực giác thông thường: thưởng cho cột mốc trung gian, phạt cho những trạng thái bị coi là “xấu”. Ngoài phần thưởng nhóm khi cả hai tới đích, nó còn một khoản thưởng cá nhân khi đứng trên bàn đạp, một khoản khi đã rời phòng xuất phát, một khoản nhóm khi cả hai đã rời phòng, một khoản cho việc đẩy khối vật cản, và một khoản phạt thời gian mỗi bước. Đáng chú ý nhất là hai khoản phạt nhóm nhằm ép hai tác nhân hành động đồng bộ: phạt khi một tác nhân đứng trên bàn đạp mà đồng đội chưa rời phòng, và phạt nặng hơn khi một tác nhân đã ra ngoài còn tác nhân kia vẫn kẹt lại.

Huấn luyện với MA-POCA, phần thưởng nhóm trung bình mắc kẹt ngay ở mức sàn - đúng bằng khoản phạt thời gian tích lũy trọn một lượt chơi - và không nhúc nhích suốt hàng triệu bước. Việc chỉ số đứng im ở đúng giá trị của riêng khoản phạt thời gian là manh mối định lượng rất đáng giá: nó nói rằng không lượt chơi nào chạm đích, và cũng không hành vi trung gian nào được tưởng thưởng đủ để kéo chỉ số lên. Đào sâu thêm thì lộ ra ba khuyết tật đan vào nhau.

Nghiêm trọng nhất: hai khoản phạt nhóm giáng đúng vào những trạng thái mà lời giải bắt buộc phải đi qua. Chuỗi trạng thái mẫu ở đầu mục đã chỉ rõ, màn bàn

giao nhất thiết phải trải qua giai đoạn bất đối xứng - một tác nhân giữ bàn đạp, tác nhân kia vượt công. Mà đó lại chính xác là cấu hình bị hai khoản phạt nhóm nhắm tới. Mọi con đường dẫn tới lời giải đều xuyên qua vùng bị phạt, nên chính sách tối ưu cục bộ hóa thành “không ai cố gắng bàn giao cả”. Đây đúng là dạng lỗi mà nguyên tắc bám theo lời giải mẫu sinh ra để phòng ngừa.

Khuyết tật thứ hai là lợi dụng phần thưởng ở giai đoạn cửa mở. Các khoản thưởng trung gian được cộng theo từng bước mà không chuẩn hóa theo độ dài lượt chơi, nên tác nhân sớm nhận ra rằng chỉ cần nấn ná ở một cấu hình cho điểm là đã tích được phần thưởng, khỏi cần hoàn thành nhiệm vụ. Chỉ số phần thưởng nhóm khi đó phình lên tới gần gấp đôi giá trị của một lời giải sạch, tạo ảo giác tiến bộ trong khi hành vi thực chất vô nghĩa. Đó là biểu hiện điển hình của việc vi phạm nguyên tắc giữ trật tự độ lớn.

Khuyết tật cuối nằm ở cơ chế môi trường chứ không ở phần thưởng: cửa sập xuống tức thì khi tác nhân rời bàn đạp. Màn bàn giao vì thế đòi hỏi đồng bộ thời gian gần như hoàn hảo, và xác suất khám phá ra nó bằng thử - sai thấp đến mức gần bằng không. Hàm phần thưởng có đúng đến mấy cũng vô ích, vì hành vi cần học hầu như không bao giờ xuất hiện để mà được tưởng thưởng.

Bốn thay đổi phối hợp đã khắc phục tình hình. Trước hết, toàn bộ khoản phạt nhóm cho trạng thái bất đối xứng bị gỡ bỏ, thay bằng phần thưởng bàn giao một lần - tức chuyển từ trừng phạt sang tưởng thưởng đúng cái hành vi mà bài toán đòi hỏi. Tiếp đó, các khoản thưởng định hướng cộng dồn theo bước được thay bằng định hình theo thế năng [15], kèm bảo đảm bất biến chính sách. Cơ chế cửa được thêm khoảng ân hạn một giây: đủ ngắn để không tác nhân nào tự vượt công một mình, nhưng đủ dài để màn bàn giao từ chỗ gần như không thể khám phá trở thành khả học. Cuối cùng, quan sát của mỗi tác nhân được bổ sung hai cờ trạng thái vượt công của bản thân và của đồng đội, cho chính sách phi tập trung một căn cứ trực tiếp để định thời điểm nhả bàn đạp; đi kèm là chương trình học hai bài trình bày ở Mục 3.4.9.

Với thiết kế mới, phần thưởng nhóm thoát khỏi mức sàn và leo lên tới giá trị tương ứng với một lời giải bàn giao trọn vẹn. Để chắc chắn chỉ số đẹp không đến từ một lỗi tắt cá nhân, đề tài kiểm chứng bằng lập luận định lượng ở Mục 3.4.5:

vận tốc tối đa của tác nhân là hữu hạn và khoảng ân hạn chỉ một giây, nên không quỹ đạo nào cho phép một tác nhân đơn độc vượt cổng - phần thưởng cao chỉ có thể là bằng chứng của hợp tác thật. Bài học đáng giá nhất từ vòng hiệu chỉnh này: một hàm phần thưởng hồng thường để lại dấu vết định lượng rất rõ ngay trên các chỉ số huấn luyện, chẳng hạn việc mắc kẹt đúng ở giá trị của một thành phần phạt. Biết đọc những dấu vết ấy là công cụ chẩn đoán hiệu quả hơn hẳn việc dò tìm hệ số một cách mù quáng.

3.4.8 Điều kiện kết thúc lượt chơi

Có hai đường dẫn tới kết thúc một lượt chơi. Đường thành công là cả hai tác nhân cùng chạm vạch đích: nhóm nhận một điểm, lượt chơi của cả nhóm khép lại, và toàn bộ khu vực huấn luyện được đặt lại - vị trí hai tác nhân, khối vật cản, các cờ trạng thái, trạng thái cửa. Đường còn lại là khu vực huấn luyện chạy đủ một trăm nghìn bước vật lý mà nhiệm vụ vẫn dang dở. Lúc này lượt chơi bị ngắt chứ không phải kết thúc, nghĩa là giá trị của trạng thái cuối vẫn được ước lượng và đưa vào cập nhật chính sách; nếu không, thuật toán sẽ hiểu nhầm trạng thái đó là một thất bại tuyệt đối.

3.4.9 Cấu hình huấn luyện và học theo chương trình

Có đủ các tín hiệu phần thưởng ở trên rồi thì việc học màn bàn giao ngay từ đầu vẫn rất khó, bởi tác nhân phải học một lúc cả kỹ năng điều hướng lẫn kỹ năng phối hợp theo trình tự. Để hạ độ khó ban đầu, đề tài dùng học theo chương trình với một tham số môi trường điều khiển trạng thái cổng. Bài học thứ nhất vô hiệu hóa hai cánh cửa nên cổng luôn mở, tác nhân chỉ việc học điều hướng tới đích. Đến khi phần thưởng trung bình đã làm tron vượt ngưỡng 0,6 và giữ được qua ít nhất một trăm lượt, hệ thống tự chuyển sang bài học thứ hai: cổng bị khóa, màn bàn giao thành bắt buộc.

$$\text{Bài học} = \begin{cases} 0 \text{ (cổng mở)} & \text{nếu } \bar{r} < 0,6, \\ 1 \text{ (cổng khóa)} & \text{nếu } \bar{r} \geq 0,6 \text{ trong ít nhất 100 lượt liên tiếp.} \end{cases} \quad (3.9)$$

Chia hai bài học như vậy cho phép tác nhân dựng xong nền tảng điều hướng rồi mới phải đối mặt với thách thức phối hợp.

Môi trường này dùng cơ chế phần thưởng nhóm, nên xét về lý thuyết MA-POCA là thuật toán hợp nhất: phê bình tập trung kết hợp cơ sở phần thực cho phép gán công lao chính xác tới từng tác nhân. Đối chứng gần nó nhất là MAPPO, cũng dùng phê bình tập trung nhìn quan sát của cả nhóm nhưng tính công lao theo cách khác. Ba thuật toán còn lại - PPO, SAC, DQN - không có cơ chế nào dành riêng cho phối hợp, nên đóng vai trò mốc so sánh để trả lời câu hỏi phê bình tập trung có thật sự đáng giá hay không; với chúng, ML-Agents phân bổ phần thưởng nhóm về từng tác nhân. Cả năm cùng dùng mạng ba lớp ẩn hai trăm năm mươi sáu đơn vị mỗi lớp, cùng một tín hiệu tò mò cường độ 0,02 để khuyến khích khám phá trong môi trường phần thưởng thưa, và cùng một chương trình học. Nhờ giữ mọi thứ giống nhau như vậy, khác biệt về kết quả mới phản ánh đúng bản chất thuật toán. Số bước huấn luyện đặt ở một triệu bảy trăm nghìn cho cả năm.

3.5 Môi trường Cross The Road

3.5.1 Mô tả bài toán

Cross The Road là bài toán điều hướng đơn tác nhân: một tác nhân phải băng qua con đường có phương tiện chạy liên tục để tới vạch đích bên kia mà không va chạm. Hình thức thì đơn giản, nhưng đây là mô hình thu nhỏ của rất nhiều tình huống điều hướng tránh vật cản động hay gặp trong game - nhân vật né đạn, né kẻ địch, hay vượt qua các chướng ngại đang chuyển động. Thách thức cốt lõi là tác nhân phải học được lúc nào nên đi và lúc nào nên đợi. Lao lên đúng lúc có phương tiện qua thì va chạm; mà chờ mãi thì chẳng bao giờ tới đích.

Khu vực huấn luyện có năm làn xe nằm giữa điểm xuất phát và vạch đích. Mỗi làn một chiếc xe chạy theo chiều cố định, ra khỏi biên thì quay lại đầu làn. Tốc độ mỗi xe được bốc ngẫu nhiên lại sau từng lượt chơi, trong một dải riêng của làn đó, dao động từ một tới tám đơn vị trên giây tùy làn. Chính sự ngẫu nhiên này chặn tác nhân học vẹt một chuỗi hành động cố định, buộc nó phải học một chính sách phản ứng thật sự dựa trên tình huống quan sát được. Ngoài xe còn có tường và cây làm vật cản tĩnh. Cả khu vực được đóng thành prefab độc lập nên nhân bản được thành nhiều bản sao chạy song song khi cần tăng tốc thu thập kinh nghiệm.

3.5.2 Không gian quan sát

Tác nhân quan sát một trăm ba mươi hai giá trị, chia làm hai phần với vai trò rất rõ rệt. Sáu giá trị đầu là tọa độ ba chiều của chính tác nhân cộng tọa độ ba chiều của điểm đích, cả hai đều tính trong hệ tọa độ địa phương của khu vực. Từ hai bộ tọa độ ấy, tác nhân suy ra hướng và khoảng cách cần đi - tức thông tin định hướng tầm xa.

Một trăm hai mươi sáu giá trị còn lại thuộc về cảm biến tia. Cấu hình ở đây là hai mươi một tia trải đều trong góc quạt một trăm tám mươi độ phía trước, tầm xa bốn mươi đơn vị, nhận diện bốn nhãn vật thể gồm xe, tường, cây và đích. Mỗi tia cho sáu giá trị: bốn giá trị mã hóa một-nóng cho nhãn, một giá trị báo tia có chạm vật thể nào không, một giá trị khoảng cách chuẩn hóa.

$$o_{\text{tia}_j} = (\text{chạm}_j, d_j/d_{\max}, \underbrace{c_{j,1}, \dots, c_{j,m}}_{\text{mã hóa một-nóng}}), \quad \dim(o) = 6 + k \times (m + 2), \quad (3.10)$$

với $k = 21$ là số tia và $m = 4$ là số nhãn vật thể.

Cách phân vai giữa hai phần tám sát đúng cấu trúc bài toán. Thông tin vị trí trả lời câu hỏi đi về đâu; cảm biến tia trả lời câu hỏi lúc nào thì an toàn để đi. Bỏ cảm biến tia đi, tác nhân mất căn cứ né tránh và chỉ còn nước đoán mù thời điểm băng qua. Bỏ thông tin vị trí, tác nhân có thể né rất giỏi nhưng chẳng biết phải tiến về hướng nào để tới đích.

3.5.3 Không gian hành động và cơ chế điều khiển

Tác nhân đi theo từng ô lưới, với bốn hành động rời rạc: đứng yên, sang trái, sang phải, tiến về phía trước. Một hành động di chuyển sẽ đặt ô đích cách vị trí hiện tại đúng một đơn vị theo hướng tương ứng, rồi tác nhân trượt đều tới đó; chừng nào chưa tới nơi thì mọi hành động mới đều bị bỏ qua. Khác hai môi trường kia, chuyển động ở đây không đi qua mô phỏng lực mà chỉ là phép nội suy vị trí, nên quỹ đạo hoàn toàn xác định. Tác nhân ra quyết định sau mỗi năm bước vật lý, tức mười quyết định mỗi giây mô phỏng.

Rời rạc hóa chuyển động theo ô lưới là một lựa chọn có chủ đích. Nó giữ không gian trạng thái gọn gàng, và quan trọng hơn, nó đẩy bản chất cốt lõi của bài toán lên hàng đầu: đây là bài toán quyết định thời điểm - chọn đúng khoảnh khắc để tiến

hay để chờ - chứ không phải bài toán điều khiển vận động tinh vi. Hành động đứng yên vì thế chẳng phải lựa chọn thừa, mà là một nước đi chiến lược. Đứng yên đúng lúc để nhường một phương tiện đi qua thường chính là mấu chốt của một lượt chơi thành công.

3.5.4 Hàm phần thưởng

Bảng 3.4 trình bày hàm phần thưởng, và nó được thiết kế đơn giản hết mức: một điểm khi đến đích, trừ 0,025 điểm khi va chạm. Chiếu vào khuôn dạng (3.1) thì môi trường này chỉ dùng thành phần mục tiêu - không định hình, không khoản nào cộng dồn theo bước. Sự tối giản ấy là chủ ý. Quan sát vốn đã chứa sẵn tọa độ đích nên chẳng cần tín hiệu dẫn đường nhân tạo, còn bài toán thì đủ ngắn để tín hiệu thừa vẫn xuất hiện thường xuyên.

Tham số nhạy cảm nhất của thiết kế này là tỉ lệ giữa hình phạt va chạm và phần thưởng mục tiêu, chỉ bằng một phần bốn mươi. Lý do: môi trường không giới hạn số bước cho một lượt chơi, nên tác nhân luôn có sẵn một lựa chọn tầm thường là đứng yên vĩnh viễn, với giá trị kỳ vọng đúng bằng không. Gọi p là xác suất băng qua đường thành công của một chính sách và R_- là hình phạt va chạm; tiến lên chỉ hơn đứng yên khi

$$p \cdot 1 - (1 - p) R_- > 0 \iff p > \frac{R_-}{1 + R_-}. \quad (3.11)$$

Thay $R_- = 0,025$ vào, ngưỡng chỉ còn 2,4 phần trăm. Nghĩa là một chính sách vụng về, băng qua thành công chưa nổi ba lần trong một trăm lần thử, đã ăn đứt việc đứng yên - và quá trình học luôn có động lực tiến lên ngay từ những bước đầu. Thử đặt hình phạt cân bằng với phần thưởng, tức $R_- = 1$, ngưỡng lập tức vọt lên năm mươi phần trăm. Tác nhân khi ấy sẽ học được rằng đứng yên là tối ưu cho tới lúc nó thắng quá nửa số lần thử, một điều không tài nào đạt được nếu nó chưa từng thử. Hiện tượng chính sách tự đóng băng đó là một dạng cực tiểu cục bộ do chính hàm phần thưởng tạo ra, và (3.11) cho thấy nó nằm hoàn toàn dưới quyền điều khiển của tỉ lệ giữa hai hằng số.

Giữ hình phạt nhỏ không làm yếu đi tác dụng răn đe, vì hình phạt vốn không phải cơ chế răn đe chính. Va chạm kết thúc lượt chơi ngay tức khắc, nên tổn thất thật sự của một pha va chạm là mất trắng cơ hội nhận một điểm ở đích, chứ đâu

phải khoản 0,025 bị trừ. Chi phí cơ hội mới là thứ rắn đẽ; hằng số R_- chỉ để phá thể hòa giữa các quỹ đạo thất bại, và vì vậy giữ ở mức tối thiểu là hợp lý.

Thành phần cuối cùng là tín hiệu tò mò nội tại, theo cơ chế dự đoán tự giám sát của Pathak và cộng sự [17]: tác nhân được thưởng theo sai số dự đoán trạng thái kế tiếp của chính nó, tức được đẩy về phía những tình huống nó chưa mô hình hóa tốt. Tín hiệu này bù cho việc môi trường không có hình phạt thời gian. Đúng yên là trạng thái dễ dự đoán nhất nên hầu như chẳng sinh phần thưởng nội tại nào, và chính sách bất động vì thế mất động cơ - mà không cần đưa vào một khoản phạt cộng dồn vô hạn theo số bước, thứ sẽ vi phạm ngay nguyên tắc giữ trật tự độ lớn trong một môi trường không giới hạn độ dài lượt chơi. Cường độ đặt ở 0,02, tức một phần năm mươi trọng số của tín hiệu ngoại lai: đủ để dẫn dắt khám phá lúc đầu, nhưng quá nhỏ để làm lệch mục tiêu khi chính sách đã về đích đều đặn.

Bảng 3.4: Hàm phần thưởng của môi trường Cross The Road

Sự kiện	Giá trị	Loại
Đến được vạch đích	+1,0	Thưởng mục tiêu
Va chạm với xe, tường hoặc cây	-0,025	Phạt mục tiêu
Tín hiệu tò mò khuyến khích khám phá	cường độ 0,02	Nội tại

3.5.5 Điều kiện kết thúc lượt chơi

Lượt chơi dừng ngay lập tức trong hai trường hợp: tác nhân chạm vạch đích, hoặc tác nhân đâm phải một chiếc xe, một bức tường hay một thân cây. Cả hai trường hợp đều đưa tác nhân về vị trí xuất phát, đặt lại các xe và bốc tốc độ mới cho chúng. Môi trường không giới hạn số bước cho một lượt chơi. Điều đó an toàn, vì hình phạt va chạm cộng tín hiệu tò mò đã đủ để tác nhân không đứng lì mãi mãi; và trên thực tế, ở cuối quá trình huấn luyện độ dài lượt chơi chỉ còn khoảng vài chục bước.

3.5.6 Cấu hình huấn luyện

Đơn tác nhân, không gian hành động rời rạc nhỏ - môi trường này rất hợp để so sánh trực tiếp cả năm thuật toán trên cùng một bài toán trong điều kiện tương đương. Nó cũng là môi trường duy nhất của đề tài mà cả năm thuật toán chạy được mà không phải sửa gì bên phía Unity. Mạng nơ-ron dùng hai lớp ẩn, mỗi lớp một trăm hai

mười tám đơn vị, một kích thước vừa phải tương xứng độ phức tạp của bài toán. PPO, SAC, MA-POCA, MAPPO và DQN được cấu hình giống nhau hết mức có thể về kích thước lô lẫn kiến trúc mạng, để khác biệt về kết quả phản ánh đúng bản chất thuật toán. Một lưu ý: đây là môi trường đơn tác nhân, không có nhóm, nên ưu thế phê bình tập trung của MAPPO và MA-POCA chẳng có đối tượng nào để phát huy - cả hai được kỳ vọng suy biến về PPO. Vai trò chính của Cross The Road vì thế là kiểm chứng tính nhất quán của các thuật toán trên một bài toán điều hướng chuẩn. Số bước huấn luyện tối đa đặt ở hai triệu cho cả năm.

3.6 Bảng tổng hợp đặc trưng ba môi trường

Ba môi trường vừa trình bày khác nhau về hình thức, nhưng chúng được chọn có chủ đích để phủ ba dạng bài toán học tăng cường điển hình, đồng thời ứng với ba dạng cơ chế hay gặp trong game. Bảng 3.5 đối chiếu nhanh các đặc trưng chính, làm cơ sở cho phân phân tích kết quả ở Chương 4. Điều đáng chú ý là ba môi trường lệch nhau trên nhiều trục cùng một lúc: dạng bài toán, số lượng tác nhân, dạng không gian hành động, có dùng phần thưởng nhóm hay không, nhip ra quyết định, và cơ chế đặc thù đi kèm. Lềch đồng thời như vậy là cố ý, bởi một bộ môi trường chỉ khác nhau ở đúng một trục sẽ không đủ sức cho biết thuật toán nào thắng nhờ đâu.

Bảng 3.5: Tổng hợp đặc trưng ba môi trường thực nghiệm

Đặc trưng	Football Table	Capture The Flag	Cross The Road
Dạng bài toán	Đối kháng một đối một	Hợp tác đa tác nhân	Điều hướng đơn tác nhân
Số tác nhân	Hai, đối kháng	Hai, hợp tác	Một
Không gian quan sát	60 giá trị	61 giá trị	132 giá trị
Không gian hành động	Liên tục, tám chiều	Rời rạc, bảy hành động	Rời rạc, bốn hành động
Cảm biến tia	Không	Bảy tia, sáu nhãn	Hai mươi một tia, bốn nhãn
Nhịp ra quyết định	2 bước vật lý	20 bước vật lý	5 bước vật lý
Phần thưởng nhóm	Không	Có	Không
Kết thúc lượt chơi	Bàn thắng hoặc 3.000 bước	Cả hai tới đích hoặc 100.000 bước	Tới đích hoặc va chạm
Cơ chế đặc thù	Tự chơi	Bàn giao và học theo chương trình	Vật cản động
Ngân sách bước thực dùng	1,60 triệu	1,70 triệu	2,00 triệu
Thuật toán kỳ vọng tốt nhất	PPO tự chơi	MA-POCA hoặc MAPPO	SAC

Dòng cuối của bảng ghi lại kỳ vọng đặt ra trước khi chạy thực nghiệm, chứ không phải kết quả. Chương 4 cho thấy chỉ một phần kỳ vọng đó được số liệu xác nhận, và chính những chỗ lệch mới là phần đáng đọc nhất của kết quả. Ngân sách bước ghi ở dòng trên là số bước thực tế mà cả năm thuật toán trong cùng một môi trường đều tiêu thụ; giá trị `max_steps` khai báo trong tệp cấu hình của từng lần chạy được đặt đúng bằng con số này để mọi so sánh diễn ra trên cùng lượng kinh nghiệm.

3.7 Triển khai thuật toán ngoài ML-Agents

3.7.1 Hai thuật toán không có sẵn trong ML-Agents

Mục 2.5.6 đã phân tích: ML-Agents 4.0.2 chỉ có sẵn ba trainer là ppo, poca và sac. Hai thuật toán còn lại trong lưới so sánh của đề tài, MAPPO và DQN, phải tự đưa vào bằng tay. Có hai con đường khả dĩ, và chọn con đường nào sẽ quyết định luôn chất lượng của mọi phép so sánh về sau.

Con đường thứ nhất: bỏ hẳn chương trình huấn luyện của ML-Agents, nối với môi trường Unity qua giao diện lập trình cấp thấp rồi dùng một thư viện học tăng cường bên ngoài. Con đường thứ hai: giữ nguyên chương trình huấn luyện, chỉ thêm thuật toán mới dưới dạng trainer mở rộng. Đề tài chọn con đường thứ hai cho cả MAPPO lẫn DQN, và lý do mang tính quyết định với mục tiêu kiểm chứng môi trường. Chỉ khi cả năm thuật toán chạy trên cùng một đường ống thu thập quỹ đạo, cùng một cách ghi nhật ký, cùng một định dạng mô hình xuất ra, thì chênh lệch đo được giữa chúng mới nói lên đặc tính của môi trường - chứ không phải nói lên sự khác biệt của hạ tầng đo.

3.7.2 Bổ sung thuật toán bằng cơ chế trainer mở rộng

ML-Agents mở sẵn một điểm mở rộng theo tên để đăng ký thêm trainer: mỗi trainer mới khai báo một mục trong danh sách `ML_AGENTS_TRAINER_TYPE`, gán một chuỗi định danh với hàm khởi tạo tương ứng. Cài gói mở rộng vào rồi thì chương trình `mlagents-learn` sẽ nhận ra giá trị mới của trường `trainer_type` trong tệp YAML và khởi tạo đúng lớp trainer đã đăng ký; mọi phần khác của quy trình giữ nguyên. Nhờ đó MAPPO và DQN dùng chung với ba trainer có sẵn toàn bộ cơ chế giao tiếp gRPC với Unity, cơ chế ghi TensorBoard, cơ chế lưu điểm kiểm tra và cơ chế xuất mô hình ONNX.

MAPPO cài đặt bằng cách kế thừa bộ tối ưu của MA-POCA, giữ nguyên mạng phê bình đa tác nhân vốn nhận quan sát của cả nhóm làm đầu vào. Chỗ khác biệt nằm ở cách tính lợi thế. MA-POCA trừ đi một đường cơ sở phản thực ước lượng riêng cho từng tác nhân; MAPPO tính lợi thế trực tiếp so với giá trị tập trung và bỏ hẳn số hạng đường cơ sở khỏi hàm mất mát. Đó chính xác là điểm phân biệt hai phương pháp về mặt thuật toán, nên cài đặt kiểu này bảo đảm mọi chênh lệch quan

sát được giữa MAPPO và MA-POCA đều đến từ cơ chế gán công lao, chứ không từ kiến trúc mạng hay siêu tham số. Cấu hình MAPPO vì vậy dùng đúng bộ siêu tham số của PPO và MA-POCA.

DQN thì kế thừa lớp trainer off-policy có sẵn - chính lớp mà SAC đang dùng - nên thừa hưởng nguyên vẹn bộ đệm phát lại và lịch cập nhật theo tham số `steps_per_update`. Phần phải viết mới là bộ tối ưu: một mạng Q kèm mạng mục tiêu cập nhật mềm, hàm mất mát sai số thời gian khác biệt, và quy tắc chọn hành động ϵ -tham lam. Ràng buộc kỹ thuật đáng kể nhất là DQN chỉ làm việc được với không gian hành động rời rạc. Vì thế môi trường Football Table phải dựng lại thành một bản riêng, trong đó tham số hành vi chuyển từ tám kênh liên tục sang tám nhánh ba mức.

3.7.3 Kết nối thư viện ngoài qua *Python Low-Level API*

Con đường thứ nhất vẫn được đề tài dựng xong và kiểm chứng, bởi nó cần thiết cho những trường hợp phải can thiệp sâu vào chính vòng lặp huấn luyện. Thư viện `mlagents_envs` cho phép nối thẳng với một môi trường Unity đang chạy, rồi bọc môi trường ấy thành một môi trường tuân theo giao diện chuẩn quen thuộc trong cộng đồng học tăng cường; từ đó đưa được vào bất kỳ thư viện bên ngoài nào, ở đây là Stable-Baselines3 [19]. Quy trình có bốn bước: khởi chạy môi trường Unity ở chế độ chờ kết nối, tạo đối tượng kết nối rồi bọc bằng lớp chuyển đổi giao diện, khởi tạo tác nhân với kiến trúc mạng mong muốn, và gọi vòng lặp huấn luyện của thư viện. Cái hay của cách này là môi trường Unity giữ nguyên, không phải sửa dòng mã nguồn nào, mà thuật toán lại nắm toàn quyền kiểm soát vòng lặp huấn luyện.

Một điểm cần nói rõ để khỏi hiểu nhầm khi đọc kết quả: mười lần chạy báo cáo ở Chương 4 đều dùng chương trình huấn luyện của ML-Agents, kể cả năm lần chạy SAC. Đường đi qua Stable-Baselines3 xuất hiện ở đây với tư cách một phương án đã dựng xong và dùng lại được, chứ không phải nguồn của bất kỳ con số nào trong bảng kết quả. Đoạn mã minh họa nằm ở Phụ lục B.

3.7.4 Cấu trúc mã nguồn bổ sung

Bên cạnh các tệp cấu hình cho từng lần huấn luyện, hệ thống còn có thêm vài thành phần mã nguồn Python phục vụ tự động hóa và phân tích. Đầu tiên là một chương trình điều phối: nó chạy tuần tự mười lăm lần huấn luyện, tự nạp đúng tệp cấu hình cho từng tổ hợp môi trường - thuật toán, quản lý trạng thái tiến độ, ghi nhật ký, và cho phép tiếp tục từ điểm dừng nếu quá trình bị gián đoạn. Tiếp theo là gói trainer mở rộng đã mô tả ở mục trên, cung cấp MAPPO và DQN, cùng lớp bọc môi trường phục vụ con đường tích hợp qua thư viện ngoài. Cuối cùng là chương trình phân tích: nó đọc tệp sự kiện TensorBoard của cả mười lăm lần chạy, trích ra các chỉ số như phần thưởng, độ dài lượt chơi, hàm mất mát và entropy, rồi tự sinh biểu đồ so sánh kèm một báo cáo tổng hợp. Vài trích đoạn mã nguồn minh họa của các thành phần này nằm ở Phụ lục E.

3.8 Thiết lập thực nghiệm

3.8.1 Cấu hình phần cứng và phần mềm

Mọi thực nghiệm đều chạy trên một máy tính cá nhân với cấu hình cố định, tóm tắt ở Bảng 3.6. Dùng chung một cấu hình cho tất cả các lần chạy là điều kiện bắt buộc nếu muốn so sánh thời gian huấn luyện và tốc độ hội tụ một cách công bằng, bởi hai chỉ số này phụ thuộc khá nhiều vào năng lực tính toán của phần cứng. Đáng nói là cấu hình chỉ dùng một bộ xử lý đồ họa tầm trung - đúng điều kiện thực tế mà phần lớn sinh viên và nhà phát triển độc lập có thể với tới - nên các kết luận của đề tài cũng thực tiễn hơn.

Bảng 3.6: Cấu hình phần cứng và phần mềm thực nghiệm

Thành phần	Chi tiết
Bộ xử lý trung tâm	Intel Core i5-14600KF
Bộ xử lý đồ họa	NVIDIA GeForce RTX 3050
Bộ nhớ trong	32 GB
Hệ điều hành	Windows 11
Unity ML-Agents	phiên bản 4.0.2 trên Unity 6000.3
Nền tảng học sâu	PyTorch 2.2.2 với CUDA 12.1
Thư viện bổ sung	Stable-Baselines3 và giao diện Python của ML-Agents
Hệ số tăng tốc mô phỏng	gấp hai mươi lần thời gian thực

3.8.2 Tham số huấn luyện từng thuật toán

Bảng 3.7 tóm tắt các siêu tham số chính của năm thuật toán trên từng môi trường. Nguyên tắc thiết lập rất đơn giản: những tham số chung như tốc độ học, hệ số chiết khấu và kiến trúc mạng phải giống nhau nhiều nhất có thể giữa năm thuật toán trên cùng một môi trường, để khác biệt về kết quả phản ánh bản chất thuật toán chứ không phải chênh lệch tham số. Cụ thể, cả năm lần chạy trong một môi trường dùng chung tốc độ học $3e-4$, hệ số chiết khấu 0,99, và đúng một kiến trúc mạng - hai lớp một trăm hai mươi tám đơn vị cho Cross The Road, hai lớp hai trăm năm mươi sáu đơn vị cho Football Table, ba lớp hai trăm năm mươi sáu đơn vị cho Capture The Flag. Khác biệt về tham số chỉ xuất hiện ở những thành phần đặc thù của từng loại thuật toán, chẳng hạn kích thước bộ đệm kinh nghiệm rất lớn của hai thuật toán off-policy, hay hệ số cập nhật mềm của mạng mục tiêu. Cấu hình đầy đủ cho từng thuật toán nằm ở các Phụ lục A đến D.

Bảng 3.7: Tóm tắt siêu tham số huấn luyện chính

Tham số	PPO	SAC	MA-POCA	MAPPO	DQN
Tốc độ học	$3e-4$	$3e-4$	$3e-4$	$3e-4$	$3e-4$
Hệ số chiết khấu	0,99	0,99	0,99	0,99	0,99
Kích thước lô	512-2048	256-1024	512-2048	512-2048	256-1024
Kích thước bộ đệm	10K-41K	100K-500K	10K-41K	10K-41K	100K-500K
Số đơn vị ẩn	128-256	128-256	128-256	128-256	128-256
Số lớp ẩn	2-3	2-3	2-3	2-3	2-3
Cơ chế riêng	cắt tỉ lệ 0,2	cập nhật mềm 0,005	phê bình tập trung	phê bình tập trung + chia sẻ	mạng mục tiêu + ε -tham lam

Hai thuật toán off-policy còn dùng chung tham số `steps_per_update` đặt bằng hai mươi: mạng chỉ được cập nhật một lần sau mỗi hai mươi bước môi trường. Giá trị này chọn để thời gian chạy thực tế nằm gọn trong quỹ thời gian của đề tài, và nó ảnh hưởng trực tiếp lên tốc độ hội tụ đo được của SAC, như Chương 4 sẽ phân tích.

3.8.3 Tiêu chí đánh giá

Hiệu năng của mỗi thuật toán trên mỗi môi trường được chấm theo bốn tiêu chí định lượng, mỗi tiêu chí soi một khía cạnh khác nhau của chất lượng học. Mức năng lực đạt được là phần thưởng trung bình của mười phần trăm cuối quá trình huấn luyện; lấy trung bình như vậy để phản ánh mức hiệu năng ổn định mà chính sách giữ được ở cuối, thay vì một giá trị tức thời có khi chỉ là đỉnh nhiễu. Riêng môi trường đối kháng dùng thêm chỉ số xếp hạng ELO, bởi trong cơ chế tự chơi, phần thưởng tuyệt đối không nói lên tiến bộ khi cả hai phía cùng mạnh lên.

Tốc độ hội tụ đo bằng bước huấn luyện đầu tiên mà từ đó trở đi đường cong phần thưởng không còn tụt xuống dưới chín mươi phần trăm mức năng lực ổn định cuối cùng. Cách xác định này bền hơn tiêu chí “lần đầu chạm ngưỡng”, vì nó không bị một đỉnh nhiễu đơn lẻ kéo về đầu chuỗi. Đổi lại, một lần chạy còn dao động mạnh ở cuối sẽ chẳng có bước hội tụ nào - mà bản thân điều đó cũng là một kết quả cần

báo cáo. Độ ổn định huấn luyện thì đo bằng độ lệch chuẩn của đường cong phần thưởng trên mười phần trăm cuối: đường cong càng ít dao động, quá trình học càng bền và càng đáng tin.

Tiêu chí thứ tư là độ dài trung bình của một lượt chơi. Cả ba môi trường của đề tài đều kết thúc lượt chơi ngay khi nhiệm vụ xong, nên độ dài lượt chơi trở thành đại lượng thay thế đo được cho tỷ lệ hoàn thành: lượt chơi chạm trần thời gian nghĩa là tác nhân không giải xong, còn lượt chơi ngắn dần nghĩa là chính sách kết thúc nhiệm vụ ngày một nhanh. Chỉ số này đặc biệt có ích ở chỗ nó tách được hai kiểu kết thúc sớm mà riêng phần thưởng không phân biệt nổi: về đích nhanh, hay thất bại sớm. Toàn bộ các chỉ số vừa nêu đều được thu thập tự động từ tệp sự kiện TensorBoard của mỗi lần chạy, rồi trực quan hóa thành biểu đồ và bảng ở Chương 4.

Về dữ liệu, đề tài đánh giá một lưới đầy đủ mười lăm tổ hợp môi trường - thuật toán, tức cả năm thuật toán PPO, SAC, MA-POCA, MAPPO và DQN trên cả ba môi trường. Trong mỗi môi trường, năm lần chạy dùng chung một ngân sách bước, đúng bằng giá trị ghi ở Bảng 3.5: hai triệu bước cho Cross The Road, một triệu bảy trăm nghìn bước cho Capture The Flag, một triệu sáu trăm nghìn bước cho Football Table. Đây là điều kiện cần để mọi so sánh có nghĩa, bởi chênh lệch phần thưởng giữa hai thuật toán chỉ quy được cho bản thân thuật toán khi cả hai đã tiêu thụ cùng lượng kinh nghiệm. Việc ba môi trường có ngân sách khác nhau thì không sao, vì đề tài không so sánh chéo giữa các môi trường mà chỉ xếp hạng trong nội bộ từng môi trường.

Có đúng một ô của lưới không đem so trực tiếp với các ô cùng hàng được: lần chạy DQN trên Football Table. Thuật toán này đòi hỏi hành động rời rạc nên phải dùng bản dựng riêng của cảnh; và khi ghép thêm cơ chế tự chơi thì bộ điều phối đối kháng báo lỗi ngay lúc khởi tạo, buộc lần chạy phải tắt tự chơi. Ô đó vì thế không có chỉ số ELO và chỉ được đọc như một tham chiếu rời. Toàn bộ số liệu còn lại đều trích thẳng từ tệp sự kiện TensorBoard của mười lăm lần chạy, không qua bước làm mượt nào, để bảng số liệu và đồ thị ở Chương 4 luôn khớp với nhau.

4. KIỂM CHỨNG MÔI TRƯỜNG QUA THỰC NGHIỆM

Chương 3 đã đưa ra đặc tả của ba môi trường. Nhưng đặc tả chỉ nằm trên giấy chùng nào chưa có thứ gì học được trong nó. Mã nguồn biên dịch trót lọt chẳng chứng minh được rằng hàm phần thưởng dẫn tới hành vi ta muốn; còn một môi trường hổng thì thường im lặng, chỉ lặng lẽ dạy tác nhân điều khác. Chương này vì thế dùng thực nghiệm để nghiệm thu ba môi trường đã xây, xoay quanh ba câu hỏi:

- **Môi trường có học được không?** Không thuật toán nào tiến bộ thì khả năng cao lỗi nằm ở đặc tả - quan sát thiếu thông tin, hoặc phần thưởng không dẫn đường - chứ không phải ở thuật toán.
- **Môi trường có phân biệt được không?** Mọi thuật toán cùng về đích ngang nhau ở mức tối đa thì hoặc môi trường quá dễ, hoặc đang có một lối tắt cho phép gom phần thưởng mà khỏi giải nhiệm vụ. Một môi trường dùng được phải tách các hướng tiếp cận thành thứ hạng *giải thích được* bằng chính cơ chế của bài toán.
- **Kết quả có ổn định và tái lập được không?** Đường cong phải hội tụ chứ không dao động mãi, và mọi lần chạy trên cùng một môi trường phải so sánh được với nhau.

Công cụ trả lời ba câu hỏi ấy là mười lăm lần huấn luyện, tức lưới đầy đủ năm thuật toán trên ba môi trường. Vai trò của năm thuật toán ở đây cần được nhấn mạnh: chúng là *phép thử*, không phải *đối tượng* đánh giá. Chọn năm thuật toán khác hẳn nhau về nguyên lý - on-policy, off-policy, phê bình tập trung, dựa trên giá trị - là chủ đích, bởi một môi trường chỉ được kiểm chứng chắc chắn khi nó phản ứng nhất quán trước nhiều hướng tiếp cận độc lập, chứ không phải khi nó được chỉnh vừa khít cho đúng một thuật toán. Kết quả sẽ đọc lần lượt theo từng môi trường, rồi mới đặt cạnh nhau theo bốn tiêu chí ở Mục 3.8.3. Một điều phải nói trước vì nó chi phối cách đọc cả chương: trong cơ chế tự chơi, phần thưởng tuyệt đối là chỉ báo gây hiểu nhầm, nên mọi kết luận về Football Table đều dựa trên chỉ số ELO chứ không dựa trên phần thưởng.

Số liệu trong chương đều trích thẳng từ tệp sự kiện TensorBoard của các lần chạy huấn luyện, qua kịch bản `export_training_data.py`. Thước đo chính là phần thưởng tích lũy mà tác nhân nhận từ môi trường, ứng với chuỗi `Environment/Cumulative Reward` trong tệp sự kiện. Mọi đường cong và mọi chỉ số thống kê đều lấy từ chuỗi thô, không qua bước làm mượt nào. Nhờ vậy từng điểm trên đồ thị đối chiếu được với đúng một bản ghi trong tệp sự kiện, và bảng số liệu luôn khớp đồ thị. Có một khác biệt nhỏ cần lưu ý khi đối chiếu với ứng dụng trình diễn ở Chương 5: đường cong hiển thị trong ứng dụng được làm mượt bằng trung bình trượt cho dễ đọc trên màn hình nhỏ, nên con số ở đó lệch chút ít so với chương này. Mỗi thuật toán ở đây tương ứng đúng một lần chạy hoàn chỉnh.

Trong mỗi môi trường, năm thuật toán nhận cùng một ngân sách bước: hai triệu bước cho Cross The Road, một triệu bảy trăm nghìn bước cho Capture The Flag, một triệu sáu trăm nghìn bước cho Football Table. Đây là chỗ khác với các thực nghiệm sơ bộ trước đó, vốn để mỗi lần chạy một độ dài riêng. Và nó là điều kiện cần để mọi so sánh trong chương có nghĩa: chênh lệch phần thưởng giữa hai thuật toán chỉ quy được cho bản thân thuật toán khi cả hai đã tiêu thụ cùng lượng kinh nghiệm.

Mỗi cảnh huấn luyện được nhân thành tám khu vực chạy song song. Cách này vừa tăng lượng kinh nghiệm thu trên mỗi bước mô phỏng, vừa giảm tương quan giữa các mẫu trong cùng một lô cập nhật, nên thời gian thực rút ngắn đáng kể. Đây là chi tiết phải nhớ khi đối chiếu với những công bố dùng một khu vực duy nhất: cùng một số bước môi trường ở hai cấu hình này không tương đương nhau về độ khó của bài toán học.

Bốn chỉ số thống kê được dùng xuyên suốt chương. Thứ nhất, phần thưởng trung bình mười phần trăm cuối - mức năng lực ổn định mà chính sách giữ được ở cuối quá trình. Thứ hai, phần thưởng tốt nhất, tức giá trị lớn nhất đạt được. Thứ ba là bước hội tụ, và thứ tư là độ lệch chuẩn của mười phần trăm cuối. Bước hội tụ ở đây được định nghĩa là bước huấn luyện mà từ đó trở đi đường cong không còn tụt xuống dưới chín mươi phần trăm mức ổn định cuối cùng; định nghĩa này bền hơn tiêu chí “lần đầu chạm ngưỡng” vì không bị một đỉnh nhiễu đơn lẻ kéo về đầu chuỗi. Kéo theo đó, một lần chạy còn dao động mạnh ở cuối sẽ không có bước hội

tự nào, và bảng số liệu ghi rõ “không đạt” chứ không gán bừa cho nó bước cuối cùng.

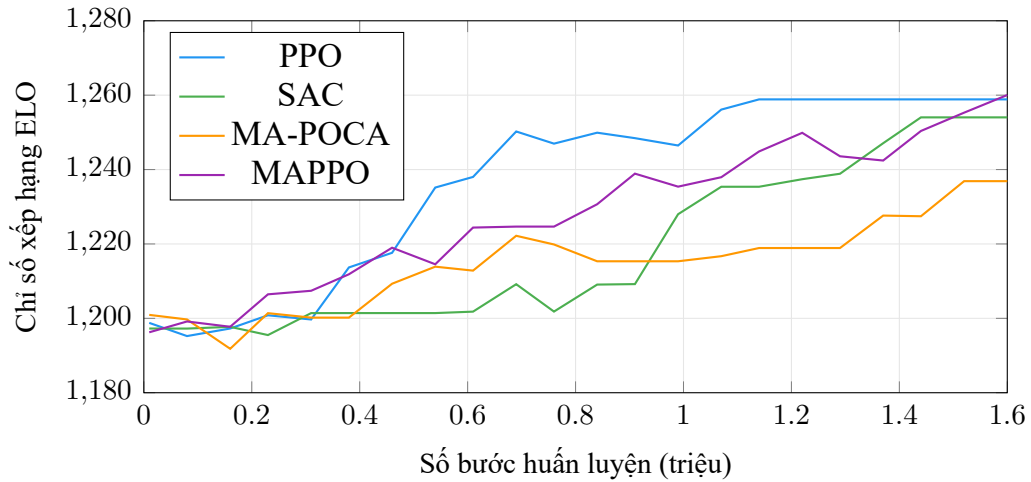
Chữ “cuối” được hiểu thống nhất cho mọi đại lượng trong chương, không riêng phần thưởng. Giá trị cuối của entropy, của phần thưởng tò mò, của hàm mất mát phê bình hay của độ dài lượt chơi đều là trung bình mười phần trăm cuối chuỗi tương ứng. Lấy trung bình trên một đoạn thay vì đọc đúng điểm cuối giúp các con số không lệ thuộc vào một bản ghi đơn lẻ, thứ có khi chệch khá xa xu hướng chung. Ngoại lệ duy nhất là chỉ số xếp hạng ELO: đại lượng này tích lũy theo thời gian nên chỉ giá trị tại bước cuối mới có nghĩa.

4.1 Kết quả môi trường Football Table

Môi trường Football Table được đánh giá qua năm lần chạy hoàn chỉnh: FB01 (PPO), Football_SAC_01 (SAC), Football_POCA_01 (MA-POCA), Football_MAPPO_01 (MAPPO) và football_dqn_v1 (DQN), ứng với năm mô hình ONNX cùng tên. Bốn lần chạy đầu đều dùng cơ chế tự chơi, trong đó tác nhân thi đấu với các phiên bản trước đó của chính nó.

Riêng DQN thì không chạy nổi cùng cơ chế tự chơi. Thuật toán này đòi hỏi không gian hành động rời rạc nên cần một bản dựng riêng của cảnh, với tham số hành vi chuyển từ tám kênh liên tục sang tám nhánh ba mức. Ghép thêm tự chơi vào, ML-Agents báo lỗi tra cứu chính sách trong bộ điều phối đối kháng rồi dừng ngay lúc khởi tạo, nên lần chạy DQN đành tắt tự chơi. Hệ quả là DQN trên Football Table học trong điều kiện khác hẳn bốn thuật toán kia, không có chỉ số ELO; mọi con số của nó ở môi trường này chỉ nên đọc như một tham chiếu rời, không đặt cạnh để xếp hạng.

Bản chất của cơ chế tự chơi khiến phần thưởng tuyệt đối thành chỉ báo tiến bộ kém tin cậy. Cả hai phía cùng mạnh lên thì lợi thế bên này bị bù trừ bởi tiến bộ bên kia, nên đường cong phần thưởng có xu hướng dao động quanh một mức gần như đứng yên thay vì tăng đều. Chỉ báo phản ánh tiến bộ chính xác hơn trong trường hợp này là chỉ số xếp hạng ELO, thứ mà ML-Agents tính riêng cho từng lần chạy dựa trên kết quả đối đầu giữa chính sách hiện tại và kho các phiên bản cũ. Hình 4.1 biểu diễn tiến trình ELO thực đo của bốn thuật toán dùng cơ chế này.

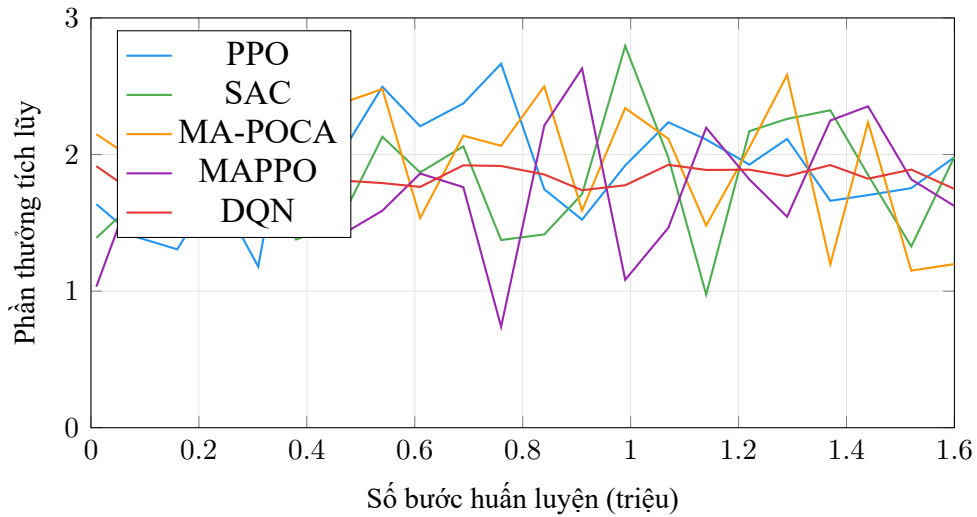


Hình 4.1: Tiến trình chỉ số xếp hạng ELO trong cơ chế tự chơi trên Football Table

Cần nói ngay rằng ELO của hai lần chạy khác nhau không đem so trực tiếp với nhau được, bởi mỗi lần chạy tính điểm dựa trên kho đối thủ của riêng nó. Đại lượng có nghĩa ở đây là mức tăng so với giá trị khởi tạo một nghìn hai trăm. Chiều theo tiêu chí đó, cả bốn thuật toán đều tiến bộ so với các phiên bản trước của chính mình. MAPPO tăng khoảng sáu mươi bốn điểm, từ 1196 lên 1260. PPO tăng khoảng sáu mươi điểm, từ 1199 lên 1259. SAC tăng khoảng năm mươi bảy điểm, từ 1197 lên 1254. MA-POCA tăng khoảng ba mươi sáu điểm, từ 1201 lên 1237. Không lần chạy nào kết thúc dưới điểm xuất phát, nên kết luận rút từ hình này là cả bốn thuật toán đều học được cách đánh bại chính mình - chứ không phải chuyển vài thuật toán tiến bộ còn số khác thì không.

Điểm cuối gần nhau, nhưng diễn biến các đường cong lại khác nhau khá nhiều. PPO tăng gần như đơn điệu tới khoảng một triệu một trăm nghìn bước rồi đứng im hẳn ở 1258,9 cho tới hết, dấu hiệu cho thấy chính sách của nó không còn vượt nổi kho đối thủ nữa. MAPPO tăng chậm hơn nhưng vẫn dốc lên ở đoạn cuối, đạt đỉnh 1265,7 trước khi khép lại ở 1260,0 - nghĩa là chưa chạm trần trong ngân sách đã cho. SAC thì gần như bất động quanh mốc khởi tạo suốt chín trăm nghìn bước đầu rồi mới bứt lên ở nửa sau, đúng đặc điểm của một thuật toán off-policy cần thời gian nạp đầy bộ đệm trước khi chính sách bắt đầu cải thiện.

Hình 4.2 biểu diễn đường cong phần thưởng tích lũy của cả năm thuật toán, còn Bảng 4.1 tổng hợp các chỉ số định lượng.



Hình 4.2: Đường cong phần thưởng tích lũy trên môi trường Football Table

Bảng 4.1: Kết quả năm thuật toán trên môi trường Football Table

Thuật toán	Số bước	Reward TB (10% cuối)	Reward tốt nhất	Độ lệch chuẩn	ELO cuối
PPO	1,60M	1,691	3,548	0,276	1259
SAC	1,60M	1,842	3,237	0,432	1254
MA-POCA	1,60M	1,752	2,867	0,590	1237
MAPPO	1,60M	2,117	3,394	0,486	1260
DQN	1,60M	1,825	2,028	0,073	—

Xét theo ELO, thứ hạng được là MAPPO (1260), PPO (1259), SAC (1254) rồi MA-POCA (1237). Có điều khoảng cách một điểm giữa MAPPO và PPO nhỏ hơn nhiều so với biên độ dao động của chính đường cong ELO trong quá trình huấn luyện - riêng MAPPO đã dao động trong dải gần hai mươi điểm ở đoạn cuối - nên số liệu hiện có không đủ để tách hai thuật toán này. Phân tách duy nhất đứng vững là MA-POCA tụt lại sau ba thuật toán còn lại.

Đáng nói hơn cả thứ hạng là chuyện biên độ chung quá hẹp. Cơ chế tự chơi tạo ra một chương trình học tự nhiên, độ khó của đối thủ tăng dần theo năng lực của tác nhân. Nhưng khi cả hai phía cùng mạnh lên với tốc độ xấp xỉ nhau, ELO gần như đứng yên dù chính sách vẫn đang cải thiện. Mức tăng sáu mươi điểm trên thang khởi tạo 1200 vì vậy chỉ nói được rằng các tác nhân có tiến bộ so với phiên bản cũ

của chính mình, chưa cho phép kết luận thuật toán nào mạnh hơn thuật toán nào.

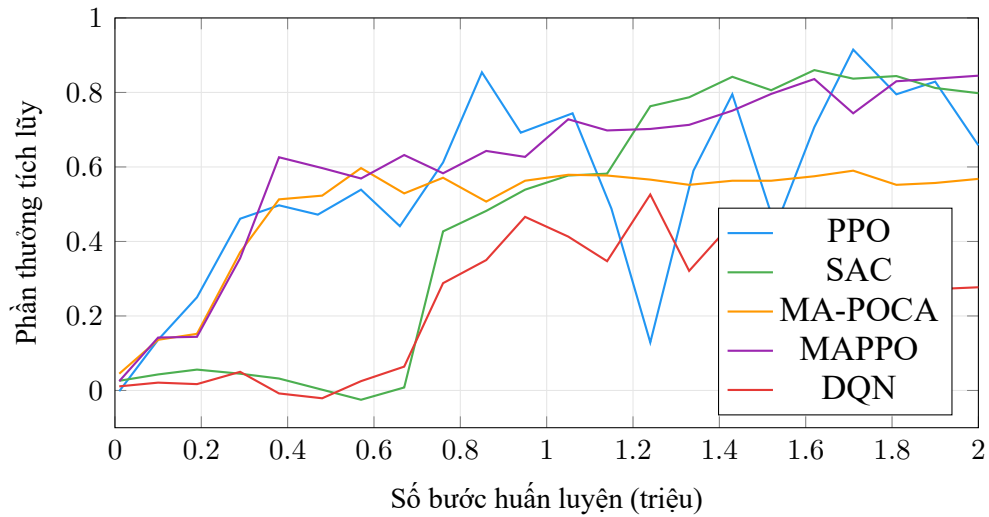
Xét theo phần thưởng thì MAPPO dẫn đầu khá rõ (2,12), hơn nhóm còn lại chừng mười lăm phần trăm, trong khi SAC (1,84), MA-POCA (1,75) và PPO (1,69) chen nhau rất sát. Con số này phải đọc kèm ELO chứ không thay được ELO: trong tự chơi, phần thưởng cao có thể chỉ phản ánh việc khai thác tốt các thành phần phần thưởng định hướng, chẳng đồng nghĩa với sức mạnh đối kháng. MAPPO là trường hợp hiếm hoi mà hai thước đo cùng chỉ về một hướng, nên nó là ứng viên đáng tin nhất cho vị trí dẫn đầu ở môi trường này.

Độ lệch chuẩn của mười phần trăm cuối lại hé lộ một khác biệt về tính ổn định. Trong nhóm tự chơi, PPO dao động ít nhất (0,276) còn MA-POCA nhiều nhất (0,590). Giá trị 0,073 của DQN tuy thấp hơn hẳn phần còn lại nhưng không đem so được: DQN không chạy tự chơi nên đối thủ của nó cố định, mà một môi trường không đổi thì đương nhiên cho đường cong phẳng hơn môi trường có đối thủ liên tục mạnh lên.

Khác các thực nghiệm sơ bộ, năm lần chạy ở đây dùng chung ngân sách một triệu sáu trăm nghìn bước, nên so sánh phần thưởng tuyệt đối giữa bốn thuật toán tự chơi là hợp lệ. Hạn chế nằm ở chỗ khác: mỗi thuật toán chỉ có một lần chạy, không đủ nhiều hạt giống ngẫu nhiên để ước lượng phương sai giữa các lần chạy (Mục 6.3).

4.2 Kết quả môi trường Cross The Road

Cross The Road là môi trường điều hướng đơn tác nhân, không gian hành động rời rạc, nên cả năm thuật toán chạy được mà chẳng phải chỉnh gì bên phía Unity. Năm lần chạy tương ứng là ctr01 (PPO), CrossTheRoad_SAC_01 (SAC), ctr_poca_v1 (MA-POCA), ctr_mappo_v1 (MAPPO) và ctr_dqn_v1 (DQN), tất cả cùng ngân sách hai triệu bước. Trong ba môi trường của đề tài, đây là nơi cho phép so sánh sạch nhất: không có cơ chế tự chơi làm nhiễu thước đo, cũng không có tác nhân thứ hai gây phụ thuộc lẫn nhau. Hình 4.3 vẽ đường cong phần thưởng thực đo, còn Bảng 4.2 tổng hợp các chỉ số định lượng.



Hình 4.3: Đường cong phần thưởng trên môi trường Cross The Road

Bảng 4.2: Kết quả năm thuật toán trên môi trường Cross The Road

Thuật toán	Số bước	Reward TB (10% cuối)	Reward tốt nhất	Bước hội tụ	Độ dài lượt chơi
PPO	2,00M	0,690	1,000	~1,97M	811,6
SAC	2,00M	0,846	0,914	~1,79M	61,8
MA-POCA	2,00M	0,556	0,616	~450K	42,2
MAPPO	2,00M	0,835	0,900	~1,72M	52,0
DQN	2,00M	0,336	0,633	Không đạt	124,7

Ghi chú: “không đạt” nghĩa là đường cong vẫn còn tụt xuống dưới chín mươi phần trăm mức ổn định cuối cùng cho tới bước huấn luyện cuối, tức chưa hội tụ theo tiêu chí ở Mục 3.8.3.

Phần thưởng trung bình cuối cao nhất thuộc về SAC (0,846), nhưng MAPPO bám sát gót (0,835). Khoảng cách 0,011 ấy còn nhỏ hơn biên độ dao động của chính các đường cong ở giai đoạn cuối, nên số liệu một lần chạy chưa đủ để khẳng định ai thực sự dẫn đầu. Điều nói chắc được là cả hai tách hẳn khỏi phần còn lại: PPO dừng ở 0,690, MA-POCA ở 0,556, DQN ở 0,336.

Kết quả của SAC phải đọc kèm một thay đổi quan trọng về cấu hình. Các thực nghiệm sơ bộ để `steps_per_update` bằng một, tức cập nhật mạng sau mỗi bước môi trường, và khi ấy SAC chạm mức ổn định chỉ sau chừng một trăm năm mươi nghìn bước. Cấu hình dùng trong chương này nâng tham số đó lên hai mươi để thời gian chạy thực tế còn chấp nhận được; cái giá là SAC ngốn nhiều mẫu hơn,

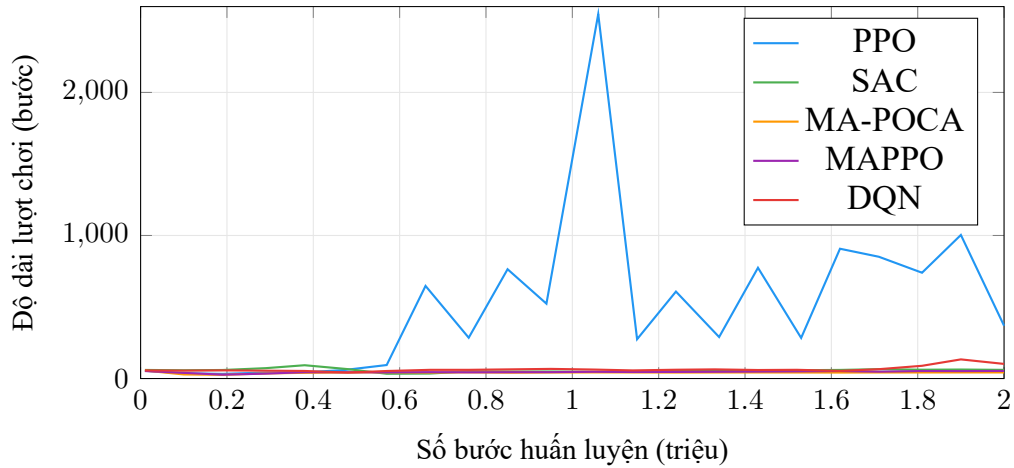
bước hội tụ lùi tới khoảng một triệu bảy trăm chín mươi nghìn. Nói cách khác, ưu thế hiệu quả mẫu mà lý thuyết off-policy hứa hẹn phụ thuộc thẳng vào tần suất cập nhật, chứ không phải một thuộc tính cố hữu của thuật toán. Chi phí tính toán mỗi bước cập nhật của SAC vẫn cao hơn PPO vì phải nuôi ba mạng nơ-ron cùng lúc: lần chạy này mất một trăm ba mươi chín phút, so với bốn mươi tám phút của PPO.

MA-POCA hội tụ sớm nhất nhóm, khoảng bốn trăm năm mươi nghìn bước - nhưng hội tụ về một mức thấp rồi nằm im ở đó tới cuối. Đó là sự ổn định của một chính sách tầm thường, không phải dấu hiệu học nhanh. Kết quả này khớp với nhận định rằng MA-POCA suy biến gần về PPO khi gặp bài toán đơn tác nhân: không có phần thưởng nhóm thì thế mạnh phê bình tập trung chẳng có chỗ phát huy, trong khi chi phí của kiến trúc phức tạp hơn vẫn phải gánh.

MAPPO ở môi trường này lại đảo ngược dự đoán ban đầu của đề tài. Xét thiết kế, MAPPO nhắm tới bài toán hợp tác đa tác nhân, nên trên một môi trường đơn tác nhân nó được kỳ vọng suy biến thành PPO thường, không lợi thế gì. Thực đo cho thấy điều ngược lại: MAPPO vượt PPO một khoảng đáng kể, 0,835 so với 0,690, dù hai thuật toán chỉ khác nhau ở phần phê bình tập trung - thứ lẽ ra vô hiệu khi nhóm chỉ có một thành viên. Đề tài chưa giải thích được cơ chế đằng sau chênh lệch này, và với đúng một lần chạy mỗi thuật toán thì cũng chưa loại trừ được khả năng đây chỉ là dao động ngẫu nhiên giữa các hạt giống. Nếu mở rộng thực nghiệm, đây là điểm đáng kiểm chứng trước tiên.

DQN xếp cuối, và cách biệt khá xa: 0,336, chưa bằng một nửa SAC. Theo tiêu chí hội tụ dừng trong chương, đường cong của nó không đạt mốc ổn định nào trước khi hết ngân sách hai triệu bước - tức thuật toán vẫn còn dao động lúc dừng. Cộng thêm thời gian chạy một trăm bốn mươi tám phút, gấp ba lần PPO, DQN là lựa chọn kém hiệu quả nhất trên môi trường này xét cả về mẫu lẫn thời gian.

Ngoài phần thưởng, còn một chỉ báo bổ sung rất đáng quan tâm là độ dài trung bình mỗi lượt chơi, bởi nó cho biết chính sách kết thúc lượt chơi theo kiểu nào. Hình 4.4 vẽ độ dài lượt chơi thực đo trong quá trình huấn luyện.



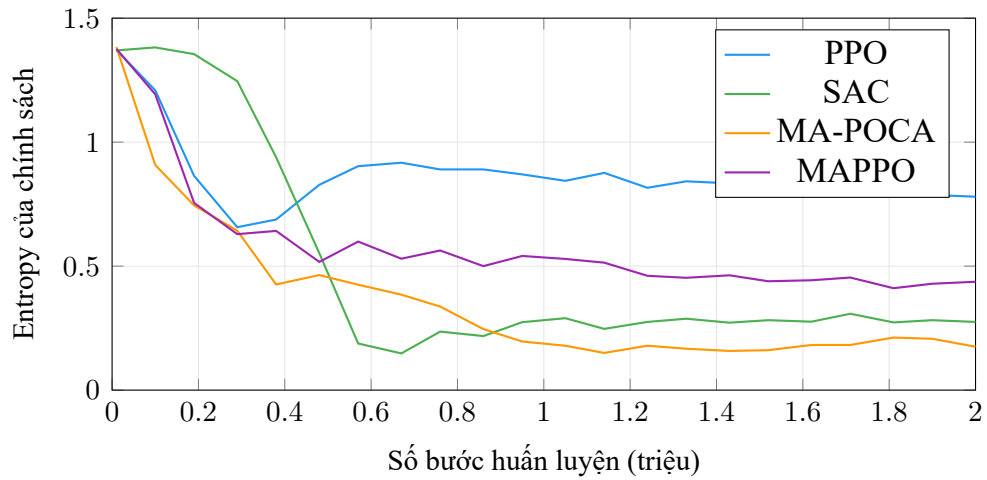
Hình 4.4: Độ dài lượt chơi trên môi trường Cross The Road

Chỉ số này tách các thuật toán rõ hơn cả phần thưởng. Bốn trong năm thuật toán giữ độ dài lượt chơi trong khoảng bốn mươi hai tới một trăm ba mươi bước suốt quá trình. Riêng PPO mất ổn định hoàn toàn từ khoảng sáu trăm nghìn bước trở đi: đường cong nhảy giữa hai trăm bảy mươi và hai nghìn năm trăm bốn mươi bảy bước, trung bình mười phần trăm cuối lên tới 811,6. Lượt chơi kéo dài như thế nghĩa là tác nhân không kết thúc nổi - nó lang thang, không về đích mà cũng chẳng va chạm.

Ghép với phần thưởng, độ dài lượt chơi phân biệt được hai kiểu kết thúc sớm vốn trông y hệt nhau nếu chỉ nhìn một chỉ số. MAPPO có lượt chơi ngắn (52,0 bước) kèm phần thưởng cao (0,835): về đích nhanh. MA-POCA cũng ngắn (42,2 bước) nhưng phần thưởng thấp (0,556): kết thúc do va chạm. SAC nằm giữa về độ dài (61,8 bước) và cao nhất về phần thưởng. DQN thì trôi dạt từ khoảng sáu mươi bước lúc đầu lên hơn một trăm ba mươi bước ở cuối, trong khi phần thưởng thấp nhất - dấu hiệu của một chính sách ngày càng lưỡng lự chứ không dứt khoát thêm.

Muốn hiểu sâu hơn hành vi học thì ngoài phần thưởng còn có thể nhìn vào các chỉ số nội tại của quá trình huấn luyện. Đầu tiên là entropy của chính sách, tức mức ngẫu nhiên khi tác nhân chọn hành động. Entropy cao nghĩa là tác nhân còn đang thử nhiều hành động khác nhau; entropy thấp nghĩa là nó đã hội tụ về một chính sách xác định. Hình 4.5 thể hiện diễn biến entropy của bốn thuật toán dựa trên chính sách. DQN không xuất hiện trong hình này vì nó thuộc nhóm phương pháp dựa trên giá trị: DQN học hàm giá trị hành động rồi chọn hành động theo quy tắc tham lam

có nhiều, nên không tồn tại phân phối chính sách để tính entropy.



Hình 4.5: Diễn biến entropy của chính sách trên môi trường Cross The Road

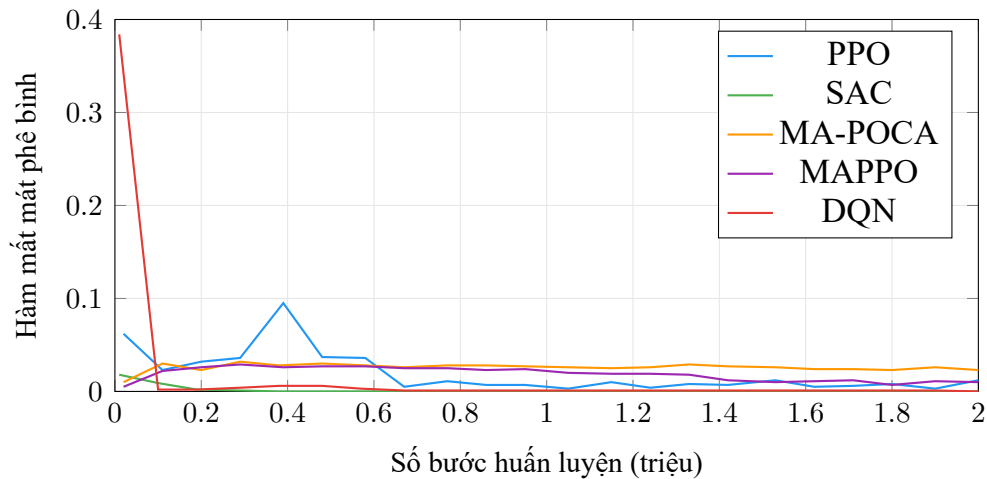
Cả bốn thuật toán đều xuất phát quanh mức 1,37, đúng với giai đoạn khám phá ngẫu nhiên ban đầu, rồi rẽ theo những hướng khác hẳn nhau. MA-POCA giảm đều và kết thúc thấp nhất (0,204), tức chính sách quyết đoán nhất nhóm. Nhưng đặt cạnh phần thưởng chỉ 0,556 thì đó lại là dấu hiệu của một chính sách chốt sớm vào phương án tầm thường - khớp với mốc hội tụ rất sớm ở khoảng bốn trăm năm mươi nghìn bước. SAC giảm dốc rồi ổn định quanh 0,280. MAPPO dừng ở mức cao hơn (0,429), nghĩa là vẫn giữ được một phần khả năng khám phá trong khi phần thưởng đã gần chạm mức cao nhất.

Bất thường và đáng chú ý nhất trong hình là đường cong của PPO. Entropy của nó giảm xuống 0,657 ở khoảng hai trăm chín mươi nghìn bước, rồi quay đầu tăng ngược và giữ quanh 0,8 cho tới hết. Entropy tăng nghĩa là chính sách ngày càng kém dứt khoát theo thời gian, ngược hẳn diễn biến của một quá trình huấn luyện lành mạnh. Dấu hiệu này ăn khớp với hai quan sát trước đó: độ dài lượt chơi của PPO bùng nổ từ khoảng cùng thời điểm, và phần thưởng của nó dừng ở mức trung bình dù ngân sách bước ngang bằng các thuật toán khác. Ba chỉ số cùng dẫn tới một cách giải thích: chính sách PPO đã rời khỏi vùng nghiệm tốt sau giai đoạn đầu và không tìm được đường quay lại.

Chỉ số cuối là hàm mất mát của mạng phê bình, cho biết việc ước lượng giá trị trạng thái chính xác đến đâu. Hình 4.6 cho thấy hàm mất mát của SAC rơi về gần

không chỉ sau vài trăm nghìn bước, nhờ cơ chế off-policy tái dùng dữ liệu đã lưu. Ba thuật toán họ PPO thì giữ một mức dương nhỏ và ổn định suốt quá trình - đặc trưng của các phương pháp on-policy, nơi mạng phê bình phải liên tục thích ứng với dữ liệu mới đến từ chính sách đang thay đổi.

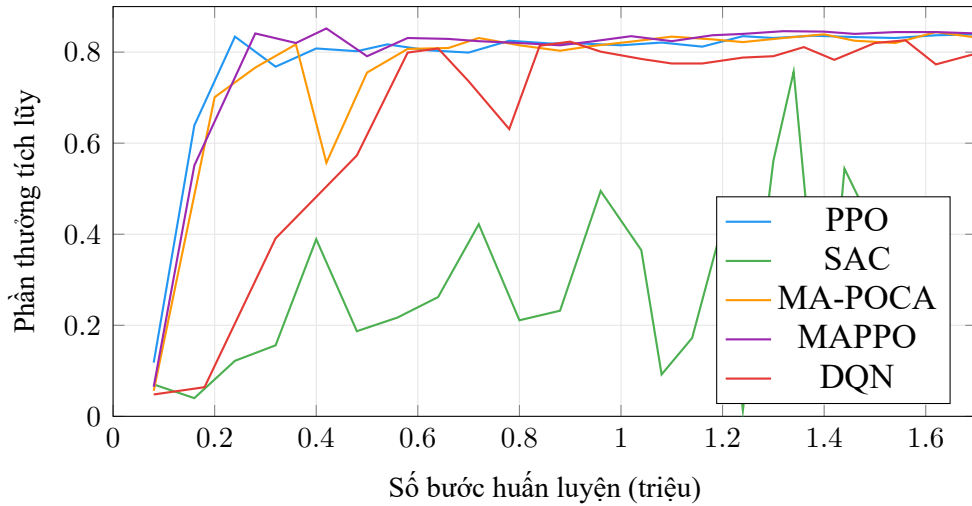
DQN lại minh họa rất rõ giới hạn của chỉ số này. Sau một đỉnh nhọn ở vài nghìn bước đầu, hàm mất mát của nó rơi xuống khoảng 0,001 rồi nằm im tới cuối, thấp hơn cả ba thuật toán họ PPO. Vậy mà DQN chính là thuật toán có phần thưởng thấp nhất ở môi trường này. Hàm mất mát phê bình nhỏ chỉ nói rằng mạng dự đoán khớp với mục tiêu do chính nó đặt ra, chứ chẳng bảo đảm mục tiêu ấy dẫn tới một chính sách tốt. Đọc chỉ số này tách rời khỏi phần thưởng là con đường ngắn nhất tới kết luận sai.



Hình 4.6: Hàm mất mát của mạng phê bình trên môi trường Cross The Road

4.3 Kết quả môi trường Capture The Flag

Capture The Flag bắt hai tác nhân phối hợp chặt chẽ trong khi phần thưởng vừa thua vừa đến muộn, lại thêm một chương trình học hai bài đặt trên tham số `handoff_required`. Cả năm thuật toán đều chạy đủ một triệu bảy trăm nghìn bước: `ctf_ppo_v1` (PPO), `ctf_sac_v1` (SAC), `ctf_poca_v2` (MA-POCA), `ctf_mappo_v2` (MAPPO) và `ctf_dqn_v1` (DQN). Kết quả nằm ở Hình 4.7 và Bảng 4.3.



Hình 4.7: Đường cong phần thưởng trên môi trường Capture The Flag

Bảng 4.3: Kết quả năm thuật toán trên môi trường Capture The Flag

Thuật toán	Số bước	Reward TB (10% cuối)	Reward tốt nhất	Bước hội tụ	Độ dài lượt chơi
PPO	1,70M	0,834	0,844	~180K	292,9
SAC	1,70M	0,183	0,795	Không đạt	4999,0
MA-POCA	1,70M	0,838	0,844	~540K	174,3
MAPPO	1,70M	0,840	0,852	~260K	153,3
DQN	1,70M	0,800	0,836	~1,04M	2017,9

Kết quả chia thành hai nhóm rất rõ. MAPPO (0,840), MA-POCA (0,838), PPO (0,834) và DQN (0,800) cùng về đích trong một dải rộng 0,04; SAC đứng lẻ loi ở 0,183. Khoảng cách giữa bốn thuật toán dẫn đầu còn nhỏ hơn dao động của chính các đường cong, nên ở cấu hình hiện tại môi trường này không phân biệt nổi chúng. Bảng chứng đáng tin hơn phần thưởng nằm ở độ dài lượt chơi, trình bày trong Hình 4.8.

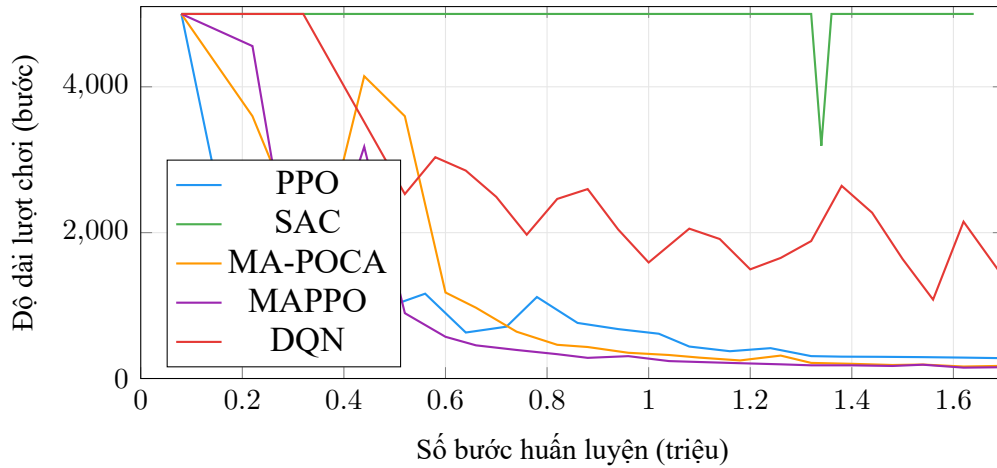
Bốn thuật toán dẫn đầu đều kéo được độ dài lượt chơi xuống thấp - từ một trăm năm mươi ba bước của MAPPO tới hai nghìn không trăm mười tám bước của DQN - nghĩa là chúng đều học được cách kết thúc câu đố thay vì để hết giờ. MAPPO và MA-POCA xuống thấp nhất và sớm nhất, hợp lý vì cả hai đều có phê bình tập trung nhìn được quan sát của cả nhóm. Đáng chú ý là PPO thường, vốn chẳng có cơ chế nào dành riêng cho phối hợp, cũng đạt 0,834 và rút lượt chơi xuống hai trăm chín

mười ba bước.

SAC là trường hợp thất bại duy nhất, mà lại thất bại theo một kiểu rất riêng. Độ dài lượt chơi của nó nằm lì ở trần bốn nghìn chín trăm chín mươi chín bước gần như suốt quá trình - hai tác nhân hầu như không bao giờ giải xong câu đố. Nhưng đường cong phần thưởng thì không hề phẳng: nó nhảy dữ dội giữa $-0,018$ và $0,795$, với mười một trên bốn mươi bốn điểm đo vượt $0,5$ và cũng đúng mười một điểm nằm dưới $0,1$, độ lệch chuẩn toàn chuỗi lên tới $0,221$. Vậy nên chính sách của SAC không phải là không học được gì; nó học được rồi đánh mất, lặp đi lặp lại. Con số $0,183$ ở mười phần trăm cuối chỉ là kết quả của việc lấy trung bình trên một chuỗi dao động như thế, chứ không mô tả năng lực ổn định nào cả.

Trên đồ thị có một chi tiết cần giải thích để khỏi hiểu nhầm. Đường cong phần thưởng của SAC dừng ở khoảng một triệu sáu trăm bốn mươi nghìn bước, trong khi bốn đường kia kéo tới hết một triệu bảy trăm nghìn bước. Nhưng lần chạy này không hề ngắn hơn: các chuỗi entropy và hàm mất mát của nó đều đủ điểm tới bước cuối cùng. Nguyên do là ML-Agents chỉ ghi phần thưởng tích lũy khi một lượt chơi kết thúc, mà lượt chơi của SAC gần như luôn chạm trần thời gian, nên sáu mươi nghìn bước cuối chẳng có lượt nào kết thúc để sinh bản ghi mới. Bản thân chuỗi phần thưởng thừa thớt của SAC - vốn vụn bốn mươi bốn điểm so với bảy mươi chín điểm của MA-POCA và MAPPO - đã là một dấu hiệu thất bại.

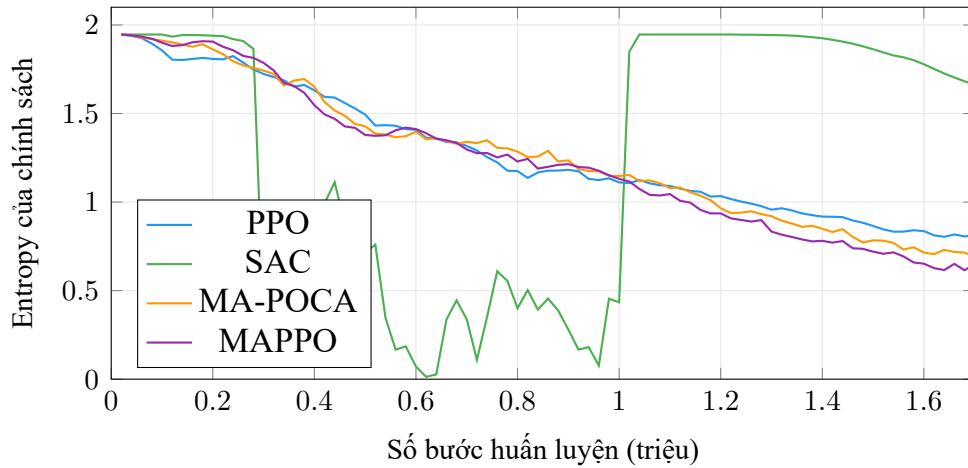
Cơ chế đằng sau hiện tượng này thì đề tài chưa xác định được. Có một giả thuyết khớp với dữ liệu: bộ đệm phát lại của SAC lưu lẫn lộn kinh nghiệm của hai bài học trong chương trình học, khiến mạng phê bình phải khớp đồng thời hai chế độ phần thưởng khác nhau. Kiểm chứng giả thuyết đó đòi hỏi thêm thực nghiệm với bộ đệm tách riêng theo bài học, nằm ngoài phạm vi đã làm.



Hình 4.8: Độ dài lượt chơi trên môi trường *Capture The Flag*: bốn thuật toán rút được lượt chơi xuống mức thấp, riêng SAC nằm ở trần thời gian gần như suốt quá trình

Các chỉ số nội tại giải thích rõ vì sao SAC tách khỏi phần còn lại. Hình 4.9 vẽ diễn biến entropy của bốn thuật toán dựa trên chính sách; DQN vắng mặt vì lý do đã nêu ở môi trường *Cross The Road*. Cả năm lần chạy đều xuất phát từ entropy 1,946, đúng mức cực đại ứng với bảy hành động chọn ngẫu nhiên đều nhau. Tới cuối quá trình, MAPPO còn 0,645, MA-POCA còn 0,719, PPO còn 0,820 - cả ba đều đã định hình được một chính sách có cấu trúc. Ba đường cong ấy giảm gần như đơn điệu và song song nhau, chỉ khác độ dốc.

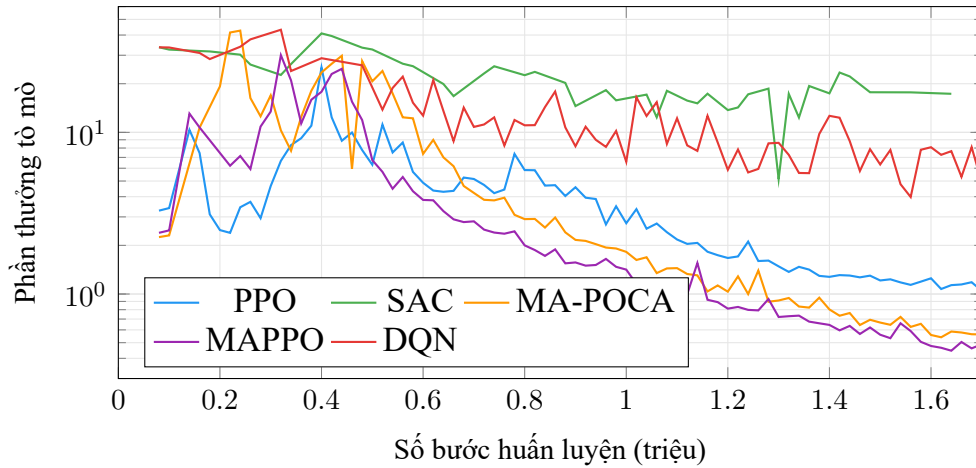
Đường cong của SAC thì mang một hình dạng hoàn toàn khác, và đó là bằng chứng trực tiếp nhất cho thất bại của nó. Từ ba trăm nghìn tới một triệu bước, entropy của SAC lao xuống rất sâu, chạm đáy 0,013 tại bước sáu trăm hai mươi nghìn - thấp hơn hẳn cả ba thuật toán họ PPO ở cùng thời điểm, gần như một chính sách xác định hoàn toàn. Rồi ngay sau đó nó bật ngược lên đúng mức cực đại 1,946, nằm lì ở đó suốt đoạn từ một triệu không trăm bốn mươi nghìn tới một triệu hai trăm sáu mươi nghìn bước, và chỉ giảm chậm về 1,663 ở bước cuối. Nói cách khác, SAC từng học được một chính sách có cấu trúc rồi đánh mất sạch, quay về trạng thái chọn hành động ngẫu nhiên đều nhau. Diễn biến này khớp chính xác với hình răng cưa của đường cong phần thưởng mô tả ở trên, và nó chỉ ra rằng vấn đề không phải SAC học quá chậm, mà là nó không giữ nổi thứ đã học.



Hình 4.9: Diễn biến entropy của chính sách trên môi trường Capture The Flag: ba thuật toán họ PPO giảm đều, riêng SAC sụp xuống rồi bật ngược về mức cực đại

Phần thưởng tò mò - thứ đo mức “bất ngờ” của tác nhân trước những trạng thái nó gặp - dẫn tới cùng một kết luận. Hình 4.10 vẽ chỉ số này với trục tung theo thang lô-ga-rit, vì biên độ giữa các thuật toán trải rộng hơn một bậc độ lớn. MAPPO kéo chỉ số xuống 0,478, MA-POCA xuống 0,577, PPO xuống 1,146: môi trường đã thành quen thuộc với cả ba. Riêng SAC chỉ giảm từ 33,6 xuống 18,7 rồi dừng, dấu hiệu mô hình động lực của nó vẫn liên tục dự đoán sai. DQN nằm giữa hai nhóm với giá trị cuối 7,014, hợp với chuyện nó giải được nhiệm vụ nhưng kém dứt khoát hơn ba thuật toán dẫn đầu - điều đã thấy qua độ dài lượt chơi còn cao.

Những đỉnh nhọn ở nửa đầu của cả năm đường cong ứng với thời điểm chương trình học chuyển bài. Tham số `handoff_required` vừa bật lên là môi trường đột ngột sinh ra những trạng thái mà mô hình động lực chưa từng gặp, nên sai số dự đoán vọt lên. Phần đáng đọc nhất của hình nằm sau các đỉnh đó: ba thuật toán họ PPO đưa chỉ số về dưới một đơn vị rồi giữ nguyên, còn SAC vẫn ở mức cao gấp hơn ba mươi lần cho tới hết. Sau một triệu bảy trăm nghìn bước, thế giới của SAC vẫn xa lạ y như lúc bắt đầu.



Hình 4.10: Phần thưởng tò mò trên môi trường Capture The Flag (trục tung thang lô-ga-rit)

Ở trường hợp SAC, bốn chỉ số độc lập cùng chỉ về một hướng. Entropy sụp xuống rồi bật ngược lên mức cực đại thay vì giảm đều. Phần thưởng tò mò vẫn cao gấp hàng chục lần nhóm còn lại. Độ dài lượt chơi nằm ở trần thời gian. Và phần thưởng thì dao động, không hội tụ. Chừng đó đủ mạnh để kết luận SAC không hình thành nổi một chính sách ổn định trên bài toán hợp tác này - dù cơ chế cụ thể gây ra điều đó thì vẫn chưa xác định được.

4.4 Đánh giá tổng hợp

4.4.1 So sánh tốc độ hội tụ

Số bước hội tụ chỉ có nghĩa khi đọc kèm mức năng lực mà chính sách hội tụ tới, và Cross The Road minh họa điều đó rõ nhất. MA-POCA chạm mốc ổn định sớm nhất nhóm, khoảng bốn trăm năm mươi nghìn bước, nhưng ổn định ở mức 0,556. Trong khi đó SAC và MAPPO phải tới khoảng một triệu bảy trăm nghìn bước mới ổn định, nhưng đạt 0,846 và 0,835. Hội tụ sớm ở đây là dấu hiệu của việc chót vào phương án tầm thường, không phải của việc học nhanh. Còn DQN thì chẳng đạt mốc ổn định nào trong suốt ngân sách hai triệu bước.

Ưu thế hiệu quả mẫu của SAC, thứ được nêu như một dự đoán ở Chương 2, hóa ra chỉ xuất hiện có điều kiện. Với cấu hình cập nhật mạng sau mỗi bước môi trường, SAC ổn định chỉ sau chừng một trăm năm mươi nghìn bước. Giảm tần suất cập nhật xuống một phần hai mươi để thời gian chạy khả thi, mốc đó lùi tụt một triệu bảy

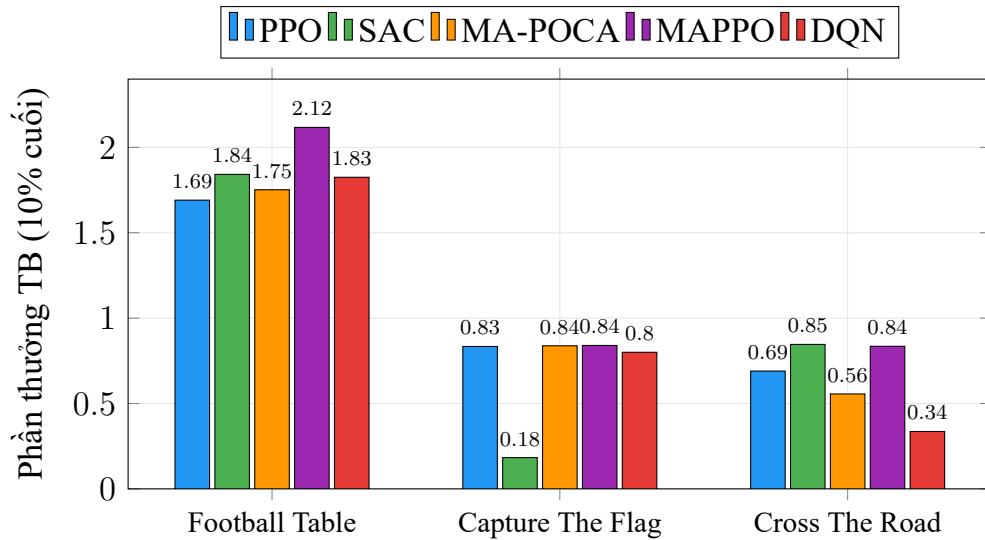
trăm chín mươi nghìn bước. Vậy nên hiệu quả mẫu của thuật toán off-policy là một đại lượng đánh đổi được với thời gian tính toán, chứ không phải hằng số của thuật toán. Còn xét theo thời gian đồng hồ, trên cùng ngân sách bước, SAC chạy một trăm ba mươi chín phút so với bốn mươi tám phút của PPO.

4.4.2 So sánh mức năng lực đạt được

Hình 4.11 đặt cạnh nhau phần thưởng trung bình mười phần trăm cuối của năm thuật toán trên cả ba môi trường - cũng chính là đại lượng mà ứng dụng trình diễn dùng ở tab so sánh. Bức tranh hiện ra không giống giả thuyết ban đầu của đề tài: chẳng có quy luật “mỗi thuật toán thắng trên đúng loại bài toán nó được thiết kế cho”.

Trên Capture The Flag - môi trường hợp tác, đúng sân của các phương pháp đa tác nhân - bốn thuật toán về đích trong dải 0,04, và PPO thường bám sát MAPPO. Cơ chế phê bình tập trung của MA-POCA và MAPPO không tạo nổi khác biệt đo được so với PPO ở bài toán này. Trên Cross The Road, môi trường đơn tác nhân nơi MAPPO lẽ ra suy biến thành PPO, thì MAPPO lại vượt PPO một khoảng lớn. Còn trên Football Table, biên độ ELO hẹp tới mức không phân định nổi bốn thuật toán tự chơi.

Cái mà dữ liệu ủng hộ chắc chắn là một dạng khác của luận điểm trung tâm: sự phù hợp giữa thuật toán và môi trường có thật, nhưng nó lộ ra qua thất bại rõ hơn qua chiến thắng. SAC dẫn đầu Cross The Road rồi sụp đổ hoàn toàn ở Capture The Flag; chênh lệch giữa hai môi trường lớn hơn mọi chênh lệch giữa các thuật toán trong cùng một môi trường. DQN xếp cuối Cross The Road nhưng lại nằm trong nhóm giải được câu đố ở Capture The Flag. Nói cách khác, chọn sai thuật toán cho một cơ chế game gây thiệt hại lớn hơn nhiều so với cái lợi của việc chọn đúng thuật toán tốt nhất trong nhóm khả dụng.



Hình 4.11: So sánh phần thưởng trung bình mười phần trăm cuối của năm thuật toán trên ba môi trường

Chiều cao các cột chỉ đem so được trong nội bộ từng môi trường, vì thang phần thưởng của ba môi trường vốn thiết kế khác nhau. Football Table cộng dồn nhiều thành phần định hướng nên thang lớn hơn hẳn; còn Cross The Road và Capture The Flag thì phần thưởng đã chuẩn hóa quanh mức một đơn vị cho mỗi lần hoàn thành nhiệm vụ.

4.4.3 So sánh độ ổn định

Thước đo độ ổn định ở đây là độ lệch chuẩn của đường cong phần thưởng trên mười phần trăm cuối. Trên Cross The Road, MA-POCA cho đường cong mượt nhất (0,017), rồi tới MAPPO (0,024) và SAC (0,025); PPO dao động mạnh nhất (0,116) do giai đoạn mất ổn định đã mô tả ở trên. Trên Capture The Flag, cả ba thuật toán họ PPO đều rất mượt, từ 0,002 của MAPPO tới 0,006 của PPO, trong khi SAC vọt lên 0,210 - lớn hơn hai bậc độ lớn.

Tính mượt của nhóm họ PPO ở hai môi trường hành động rời rạc giải thích được bằng cơ chế cắt tỉ lệ đặc trưng của họ này, vốn giới hạn độ lớn mỗi bước cập nhật. Nhưng quy luật đó không kéo sang Football Table: ở đó MA-POCA mới là kẻ dao động mạnh nhất (0,590), còn PPO mượt nhất trong nhóm tự chơi (0,276). Lý do là trong cơ chế tự chơi, độ lệch chuẩn phản ánh cả biến động của đối thủ chứ không riêng của chính sách đang học, nên ở môi trường đối kháng chỉ số này mất gần hết

ý nghĩa so sánh.

4.4.4 Bảng chứng từ chỉ số huấn luyện nội tại

Bên cạnh phần thưởng, các chỉ số nội tại của quá trình huấn luyện đưa thêm bằng chứng khách quan củng cố những nhận định trên. Bảng 4.4 tổng hợp chúng cho các lần chạy có hành động rời rạc - nơi entropy của chính sách cùng bản chất nên đối chiếu tương đối được với nhau. Trừ đúng ngoại lệ SAC trên Capture The Flag đã phân tích ở trên, entropy đều giảm mạnh từ đầu tới cuối, xác nhận rằng các tác nhân đã chuyển từ khám phá ngẫu nhiên sang một chính sách có định hướng. Bảng còn thêm cột phần thưởng tò mò, bởi trong ba chỉ số nội tại thì đây là chỉ số phân tách các lần chạy mạnh nhất: biên độ của nó trải hơn ba bậc độ lớn, từ 0,007 của MA-POCA trên Cross The Road tới 18,726 của SAC trên Capture The Flag.

Bảng 4.4: Chỉ số huấn luyện nội tại (các môi trường hành động rời rạc)

Lần chạy	Thuật toán	Entropy đầu → cuối	Mất mát phê bình cuối	Thưởng tò mò cuối
Cross The Road	PPO	1,37 → 0,79	0,006	0,201
Cross The Road	SAC	1,37 → 0,28	$\approx 0,000$	0,038
Cross The Road	MA-POCA	1,38 → 0,20	0,025	0,007
Cross The Road	MAPPO	1,38 → 0,43	0,010	0,015
Cross The Road	DQN	—	0,001	0,112
Capture The Flag	PPO	1,95 → 0,82	0,051	1,146
Capture The Flag	SAC	1,95 → 1,74	$\approx 0,000$	18,726
Capture The Flag	MA-POCA	1,95 → 0,72	0,047	0,577
Capture The Flag	MAPPO	1,95 → 0,65	0,047	0,478
Capture The Flag	DQN	—	0,001	7,014

Mọi giá trị “cuối” đều là trung bình mười phần trăm cuối của chuỗi tương ứng. DQN không có entropy vì nó chọn hành động theo quy tắc ϵ -tham lam chứ không theo một phân phối chính sách.

Entropy cuối phân tách các lần chạy đúng theo mức năng lực đạt được, nhưng theo chiều ngược với trực giác thông thường. Trên Capture The Flag, ba thuật toán giải được câu đố đều hạ entropy xuống dải 0,65–0,82, còn SAC giữ nguyên 1,74 trên mức cực đại 1,95. Chênh lệch đó là chỉ báo sớm nhất và rõ nhất cho thất bại của SAC - nó xuất hiện trước cả khi ta nhìn vào phần thưởng. Nhưng sang Cross

The Road, entropy cuối lại chẳng dự đoán được thứ hạng: MA-POCA hạ entropy thấp nhất (0,20) mà phần thưởng chỉ 0,556, còn PPO giữ entropy cao nhất (0,79) và cũng chỉ được 0,690. Vậy entropy thấp cho biết chính sách đã quyết đoán, chứ không nói được nó quyết đoán về một phương án tốt hay xấu.

Hàm mất mát phê bình thì phải đọc trong ngữ cảnh riêng của từng thuật toán. Giá trị gần không của SAC cho thấy mạng phê bình hội tụ nhanh nhờ tái dùng dữ liệu đã lưu. Giá trị 0,001 của DQN lại có nguyên nhân khác hẳn: mạng dự đoán khớp với mục tiêu do chính nó sinh ra - chuyện vẫn xảy ra ngay cả khi chính sách kém, đúng như trường hợp DQN xếp cuối Cross The Road. Ba thuật toán họ PPO giữ một mức dương nhỏ, và cao hơn hẳn ở Capture The Flag (0,047–0,051) so với Cross The Road (0,006–0,025) - hợp lý, vì phần thưởng nhóm vừa thừa vừa trì hoãn khiến việc ước lượng giá trị khó hơn nhiều.

4.4.5 Phân tích ưu nhược điểm từng thuật toán

Bảng 4.5 tổng hợp thứ hạng của năm thuật toán trên từng môi trường.

Bảng 4.5: Xếp hạng thuật toán theo từng môi trường

Môi trường	Hạng 1	Hạng 2	Hạng 3	Hạng 4	Hạng 5
Football Table	MAPPO	PPO	SAC	MA-POCA	DQN (N/A)
Cross The Road	SAC	MAPPO	PPO	MA-POCA	DQN
Capture The Flag	MAPPO	MA-POCA	PPO	DQN	SAC

Thứ hạng ở Football Table xếp theo ELO cuối; hai môi trường còn lại xếp theo phần thưởng trung bình mười phần trăm cuối. Riêng DQN trên Football Table không xếp hạng được, vì chạy không có tự chơi nên chẳng có ELO.

Bảng này phải đọc kèm một cảnh báo về độ phân giải. Ở Football Table, bốn thuật toán tự chơi chen nhau trong dải ELO 1237–1260. Ở Capture The Flag, bốn thuật toán dẫn đầu nằm gọn trong dải phần thưởng 0,800–0,840. Những khoảng cách ấy còn nhỏ hơn dao động của chính các đường cong, nên thứ tự trong nội bộ hai nhóm đó đừng coi là kết luận. Chỉ ba phân tách là thực sự vững: SAC thất bại trên Capture The Flag, DQN xếp cuối trên Cross The Road, và nhóm dẫn đầu tách

khỏi nhóm sau ở Cross The Road.

MAPPO là thuật toán duy nhất góp mặt ở nhóm dẫn đầu cả ba môi trường - hạng nhất ở Football Table lẫn Capture The Flag, hạng nhì ở Cross The Road với khoảng cách không đáng kể so với SAC. Kết quả này vượt xa phạm vi mà thiết kế của MAPPO nhắm tới: thuật toán vốn được đề xuất cho bài toán hợp tác đa tác nhân, vậy mà chạy tốt cả ở môi trường đơn tác nhân lẫn đối kháng. Đề tài chưa giải thích được cơ chế đằng sau, và với đúng một lần chạy cho mỗi ô thì cũng chưa loại được yếu tố ngẫu nhiên.

PPO đóng vai một lựa chọn mặc định hợp lý. Nó chẳng dẫn đầu ở đâu, nhưng cũng không thất bại ở đâu, luôn nằm trong nhóm giữa hoặc nhóm đầu. Nhược điểm lộ ra trên Cross The Road, nơi entropy tăng ngược và độ dài lượt chơi bùng nổ ở nửa sau: PPO có thể rời khỏi vùng nghiệm tốt rồi không tự quay lại được trong ngân sách đã cho.

SAC là thuật toán phân hóa mạnh nhất. Nó dẫn đầu Cross The Road (0,846), đứng thứ ba ở Football Table, rồi sụp đổ hoàn toàn trên Capture The Flag - entropy gần như không giảm, lượt chơi luôn chạm trần thời gian. Biên độ giữa thành công và thất bại của riêng SAC còn lớn hơn mọi chênh lệch giữa các thuật toán trong cùng một môi trường, nên khi chưa thử nghiệm trên bài toán cụ thể thì đây là lựa chọn rủi ro nhất.

MA-POCA cho kết quả trái hẳn kỳ vọng. Là thuật toán thiết kế riêng cho việc gán công lao nhóm, nó chỉ ngang bằng MAPPO và PPO ở môi trường hợp tác, lại xếp áp chót ở Cross The Road và cuối cùng trong nhóm tự chơi ở Football Table. Cơ chế phê bình dựa trên sự chú ý của nó không tạo nổi lợi thế nào đo được trong ba môi trường của đề tài.

DQN vừa yếu nhất xét toàn cục vừa khó triển khai nhất. Nó xếp cuối Cross The Road với khoảng cách lớn, chỉ chen được vào nhóm giữa ở Capture The Flag, còn trên Football Table thì cần một bản dựng riêng có không gian hành động rời rạc mà vẫn không ghép nổi với tự chơi. Ưu điểm duy nhất ghi nhận được là đường cong ổn định - nhưng sự ổn định đó đến từ việc chính sách ít thay đổi, chứ không từ chất lượng của chính sách.

4.5 Thảo luận

4.5.1 Khả năng ứng dụng trong phát triển game

Từ kết quả thực nghiệm rút ra được vài hàm ý thực tiễn cho việc đưa học tăng cường vào phát triển game. Quan trọng nhất: chọn thuật toán nên dựa trên cơ chế game cụ thể, chứ đừng đi tìm một thuật toán “tốt nhất” chung chung. Với game đối kháng - người chơi đấu người chơi, hoặc đấu máy, kiểu game thể thao hay game đấu trường - thì các thuật toán họ PPO kết hợp tự chơi là lựa chọn hợp lý. Chúng tự sinh ra đối thủ máy có trình độ tăng dần một cách tự nhiên, cho người chơi trải nghiệm vừa thử thách vừa công bằng.

Với game điều hướng, né vật cản hay giải đồ đơn tuyến, SAC đạt mức năng lực cao nhất trong thực nghiệm - nhưng kèm hai điều kiện. Một, hiệu quả sử dụng dữ liệu của nó phụ thuộc vào tần suất cập nhật mạng. Hai, thời gian chạy thực tế cao gấp gần ba lần PPO trên cùng ngân sách bước. Nhóm phát triển ít tài nguyên nên cân nhắc kỹ hai điều đó trước khi chọn SAC chỉ vì lý do “ít mẫu hơn”.

Với game hợp tác cần nhiều nhân vật máy phối hợp, thực nghiệm cho một kết quả đáng lưu ý: PPO thường bám sát MAPPO lẫn MA-POCA, nên công sức tích hợp một thuật toán đa tác nhân chuyên biệt chưa chắc được đền đáp. Khuyến nghị thực dụng là khởi đầu bằng PPO để có mốc so sánh, rồi mới tính tới MAPPO nếu mốc đó chưa đủ. Còn điều cần tránh thì rõ ràng nhất: đừng dùng thuật toán off-policy như SAC cho bài toán hợp tác có phần thưởng nhóm thừa kèm chương trình học nhiều bài, bởi đó là cấu hình duy nhất trong đề tài dẫn tới thất bại hoàn toàn. Ngoài phần hướng dẫn chọn thuật toán, bản thân ba môi trường của đề tài cùng quy trình thiết kế quan sát - hành động - phần thưởng cũng dùng lại được ngay, làm nền thử nghiệm trí tuệ nhân tạo cho các dự án game Unity khác và tiết kiệm kha khá công dụng môi trường từ đầu.

5. XÂY DỰNG ỨNG DỤNG TRÌNH DIỄN TƯƠNG TÁC

Các chương trước đã thuật lại quá trình huấn luyện và so sánh năm thuật toán học tăng cường trên ba môi trường game ba chiều. Có điều biểu đồ và bảng số liệu chẳng nói được gì trực tiếp về chất lượng hành vi của tác nhân trong game. Để khép kín

quy trình từ huấn luyện tới triển khai, đề tài dựng thêm một ứng dụng trình diễn tương tác. Người dùng ở đó chọn môi trường, chọn chế độ chơi, chọn mô hình đã huấn luyện cho tác nhân ngay trong game, và tra cứu lại chính những số liệu huấn luyện đã phân tích ở Chương 4 mà khỏi phải rời ứng dụng. Ứng dụng vừa minh họa trực quan cho các kết quả so sánh, vừa chứng minh rằng việc đưa mô hình ONNX đã huấn luyện vào một sản phẩm game hoàn chỉnh là khả thi.

5.1 Mục tiêu và yêu cầu chức năng

Ứng dụng trình diễn được đặt ra bốn nhóm yêu cầu chức năng. Nhóm thứ nhất là lựa chọn nội dung chơi: phải có một màn hình menu để người dùng chọn một trong ba môi trường (Cross The Road, Capture The Flag, Football Table) cùng một trong ba chế độ chơi ứng với các cách phân vai giữa người và máy.

Nhóm thứ hai quy định chính ba cách phân vai đó. Chế độ *Người chơi* để người dùng tự điều khiển tác nhân bằng bàn phím, qua đó nếm thử độ khó của nhiệm vụ mà tác nhân phải học. Chế độ *Agent tự chơi* giao toàn bộ tác nhân cho mô hình đã huấn luyện, phục vụ việc quan sát và đánh giá trực quan chất lượng chính sách. Chế độ *Chơi với Agent* trộn cả hai: người chơi và tác nhân máy cùng có mặt trong một phiên, đóng vai đồng đội hay đối thủ tùy bản chất từng môi trường.

Nhóm thứ ba là lựa chọn mô hình. Ở mọi chế độ có tác nhân máy tham gia, người dùng phải chọn được mô hình cụ thể - tương ứng PPO, SAC, MA-POCA, MAPPO, DQN, hoặc các phiên bản huấn luyện khác nhau - cho tác nhân dùng. Yêu cầu này biến ứng dụng thành một công cụ so sánh trực quan: có thể chạy lần lượt cùng một môi trường với các mô hình khác nhau để xem hành vi khác ra sao, hoặc cho hai mô hình đối đầu trực tiếp trong Football Table.

Nhóm cuối là tra cứu kết quả huấn luyện ngay trong ứng dụng. Thay vì bắt người xem mở TensorBoard hay lật lại báo cáo, menu có thêm hai tab chuyên biệt: một tab hiển thị đường cong phần thưởng và bảng thống kê của cả năm thuật toán trên môi trường đang chọn, một tab tổng hợp và xếp hạng các thuật toán trên cả ba môi trường. Hai tab này phải phủ đúng phạm vi thực nghiệm của Chương 4 - mười lăm lần chạy, không thiếu thuật toán nào. Người dùng nhờ vậy xem số liệu trước rồi chuyển sang tab chơi để đối chiếu với hành vi quan sát được.

Ngoài chức năng, còn một ràng buộc kỹ thuật rất quan trọng: ứng dụng không được phá cấu hình huấn luyện hiện có của các scene. Ba scene môi trường vẫn dùng hằng ngày để huấn luyện, nên mọi thay đổi phục vụ chế độ chơi chỉ được phép xảy ra tại thời gian chạy, khi người dùng đi vào từ menu. Còn mở scene trực tiếp trong Unity Editor để huấn luyện thì mọi thứ phải y nguyên như cũ.

5.2 Kiến trúc hệ thống

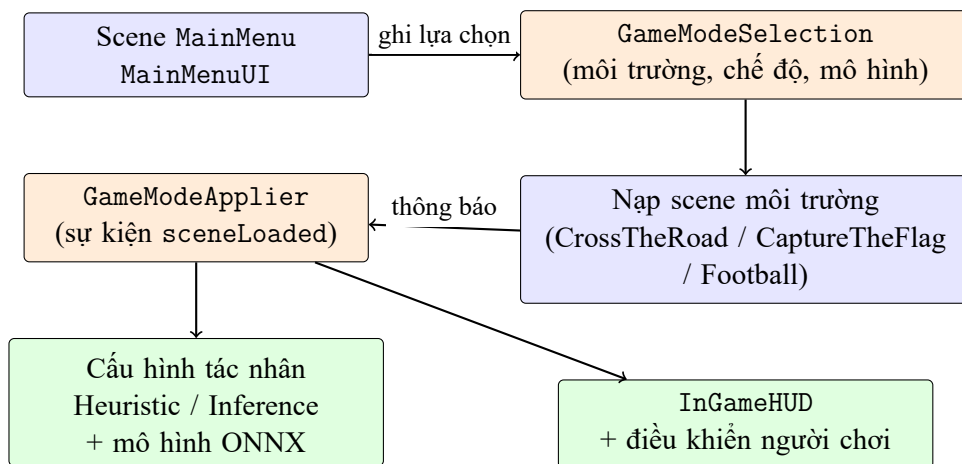
Mã nguồn của ứng dụng trình diễn nằm gọn trong thư mục riêng Assets/GameHub, tách hẳn khỏi mã nguồn ba môi trường - phần này thuộc một gói riêng (Mục 5.7) và được ứng dụng tiêu thụ qua ranh giới assembly. Bảng 5.1 tóm tắt các thành phần chính của hệ thống.

Bảng 5.1: Các thành phần chính của ứng dụng trình diễn

Thành phần	Vai trò
GameModeSelection	Lớp tính lưu lựa chọn của người dùng (môi trường, chế độ chơi, tên mô hình); tồn tại xuyên suốt giữa các scene và quyết định scene nào được nạp cho lựa chọn hiện tại.
ModelRegistry	Liệt kê và nạp các mô hình ONNX từ thư mục Resources/AgentModels tại thời gian chạy.
MainMenuUI	Dựng toàn bộ giao diện menu ba tab bằng mã lệnh khi scene MainMenu khởi động.
UIBuilder	Thư viện dựng thành phần giao diện (canvas, panel bo góc, nút, văn bản) dùng chung cho menu và HUD.
Preview3D	Dựng bản sao “chỉ có hình khối” của prefab môi trường và render bằng camera riêng ra RenderTexture để xem trước xoay được.
TrainingDataStore	Nạp tệp training_data.json chứa đường cong và thống kê huấn luyện thật đã xuất từ TensorBoard.
UILineChart	Biểu đồ đường tự vẽ bằng mesh trên uGUI, hiển thị nhiều chuỗi reward trong cùng một hệ trục.
GameModeApplier	Lắng nghe sự kiện nạp scene; chọn số khu vực huấn luyện cần giữ lại và cấu hình lại các tác nhân trong đó (loại hành vi, mô hình, chu kỳ quyết định) theo lựa chọn từ menu.
InGameHUD	Giao diện phủ trong game: nút quay về menu, thông tin chế độ và dòng hướng dẫn phím.

Thành phần	Vai trò
Các thành phần điều khiển	CrossRoadHumanInput, PuzzleHumanInput, FootballHumanInput: chuyển tín hiệu bàn phím thành hành động của tác nhân ở chế độ người chơi.

Hình 5.1 minh họa luồng hoạt động của hệ thống. Người dùng bấm nút bắt đầu ở menu, các lựa chọn được ghi vào GameModeSelection, rồi scene môi trường tương ứng được nạp. Thành phần GameModeApplier đã đăng ký với sự kiện sceneLoaded của Unity ngay từ lúc ứng dụng khởi động, nhờ thuộc tính [RuntimeInitializeOnLoadMethod], nên nó luôn sẵn sàng trước mọi lần chuyển scene. Scene môi trường nạp xong, thành phần này kiểm tra xem người dùng có đi vào từ menu hay không. Có, thì nó tìm các tác nhân trong scene và cấu hình lại từng tác nhân theo chế độ đã chọn. Không - tức là ai đó mở scene trực tiếp để huấn luyện - thì nó đứng ngoài, scene giữ nguyên cấu hình gốc. Chính cách tiếp cận dựa trên sự kiện này cho phép thêm đầy đủ chức năng chơi game mà không phải sửa bất kỳ scene môi trường nào.



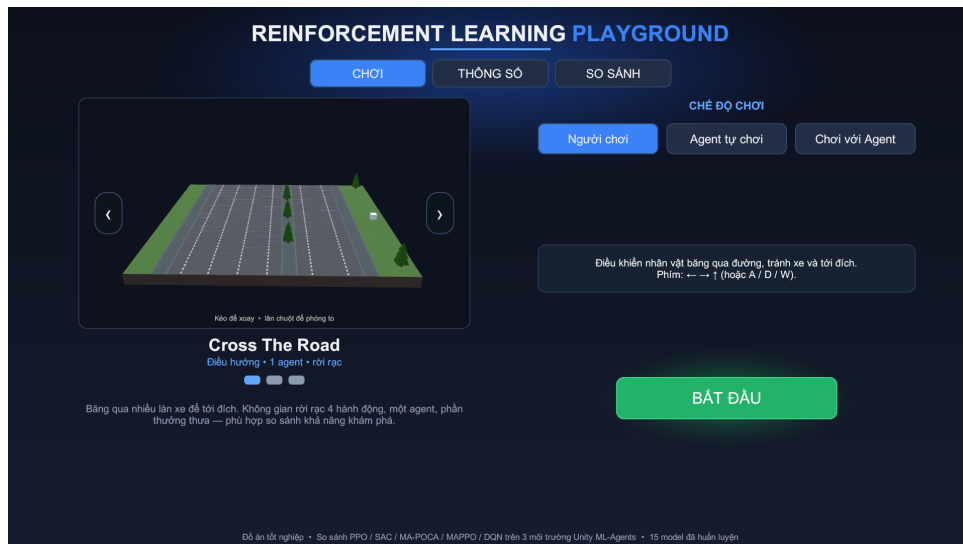
Hình 5.1: Luồng hoạt động của ứng dụng trình diễn

Mấu chốt của khâu cấu hình tác nhân nằm ở việc chuyển đổi giữa các loại hành vi của ML-Agents. Mỗi tác nhân mang một thành phần BehaviorParameters quy định nguồn ra quyết định của nó. Phe do người chơi điều khiển thì loại hành vi đặt thành HeuristicOnly, để hành động lấy từ hàm Heuristic. Chế độ tác nhân máy thì đặt thành InferenceOnly, và mô hình ONNX được nạp vào cho mạng nơ-ron suy luận thẳng trong engine. Hạ tầng suy luận tích hợp sẵn của Unity lo phần còn

lại: mọi thứ chạy cục bộ trên máy người chơi với chi phí tính toán không đáng kể, chẳng cần tiến trình Python nào như hồi huấn luyện.

5.3 Giao diện menu và luồng sử dụng

Khởi động ứng dụng là người dùng vào ngay màn hình menu chính. Toàn bộ giao diện dựng bằng mã lệnh, không dùng prefab giao diện, và chia thành ba tab: *CHƠI*, *THÔNG SỐ*, *SO SÁNH*. Tab *CHƠI* (Hình 5.2) chia màn hình làm hai cột. Cột trái là khung xem trước môi trường ở dạng ba chiều, kèm tên, khẩu hiệu phân loại bài toán và mô tả ngắn. Cột phải gồm lưới nút chọn chế độ chơi, khu vực chọn mô hình, khối mô tả và nút bắt đầu. Khối mô tả đổi động theo tổ hợp môi trường - chế độ đang chọn, hiển thị tóm tắt luật chơi cùng sơ đồ phím điều khiển, để người dùng nắm được cách chơi trước khi vào game.

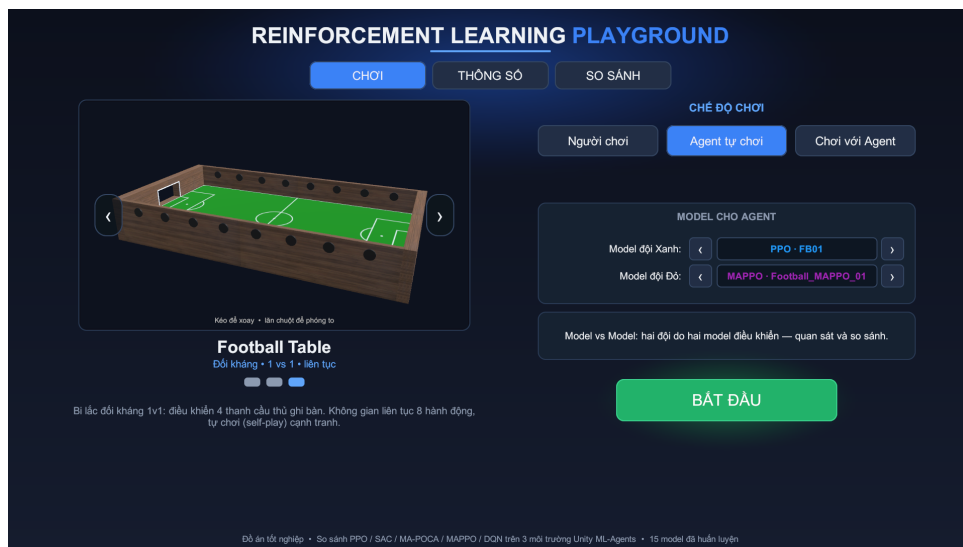


Hình 5.2: Tab *CHƠI* của menu chính: khung xem trước ba chiều (trái) và các lựa chọn chế độ chơi (phải)

Khung xem trước ba chiều là việc của thành phần *Preview3D*. Nó không dùng ảnh tĩnh, mà nạp prefab môi trường từ thư mục *Resources/EnvPrefabs* rồi chỉ sao chép các cặp lưới - vật liệu (*MeshFilter* và *MeshRenderer*) sang một “sân khấu” đặt xa khu vực menu. Giữ lại mỗi phần hình học nghĩa là bản xem trước không mang theo mã điều khiển hay thành phần vật lý nào; chẳng có mô phỏng nào chạy ngầm, cũng không ảnh hưởng chéo tới scene menu. Một camera riêng render sân khấu ấy ra *RenderTexture* rồi gắn vào giao diện, cho phép kéo chuột để xoay,

lăn chuột để phóng to; không thao tác gì thì mô hình tự xoay chậm. Khoảng cách camera ban đầu tính từ hộp bao của toàn bộ hình khối, nên môi trường nào cũng vừa khung mà khối chỉnh tay. Hai nút mũi tên hai bên khung cho phép nhảy nhanh giữa ba môi trường theo kiểu băng chuyền.

Khu vực chọn mô hình chỉ hiện ra khi chế độ đang chọn có tác nhân máy tham gia. Số ô chọn cũng đổi theo ngữ cảnh: tổ hợp nào chỉ cần một mô hình thì hiện một ô duy nhất, riêng Football Table ở chế độ *Agent tự chơi* hiện hai ô riêng cho đội Xanh và đội Đỏ (Hình 5.3) để hai mô hình khác nhau đối đầu trực tiếp. Nhấn từng ô cũng sinh theo ngữ cảnh - “Model cho cả 2 agent”, “Model đội Xanh”, “Model agent đồng đội”... - nên người dùng biết chính xác mô hình mình chọn sẽ điều khiển ai. Danh sách mô hình trong mỗi ô liệt kê tự động từ thư mục tài nguyên, thành ra thêm một mô hình mới chỉ là chuyện sao chép tệp ONNX vào đúng chỗ, không phải sửa dòng mã nào. Mỗi ô hiển thị tên thuật toán ghép mã lần chạy - chẳng hạn “MA-POCA · ctf_poca_v2” - và tô đúng màu quy ước của thuật toán đó. Duyệt lần lượt năm mô hình của một môi trường, người dùng biết ngay đang xem chính sách của thuật toán nào, khỏi phải thuộc lòng quy ước đặt tên các lần chạy.



Hình 5.3: Khu vực chọn mô hình khi Football Table ở chế độ *Agent tự chơi*: hai ô chọn riêng cho đội Xanh và đội Đỏ

Nút bắt đầu chỉ sáng lên khi mọi điều kiện đã đủ. Thư mục mô hình của môi trường đang chọn mà rỗng, hoặc hai ô mô hình của Football Table rơi vào hai loại không gian hành động khác nhau, thì ứng dụng khóa nút bắt đầu và hiện một dòng

cảnh báo đỏ chỉ rõ nguyên nhân. Nhờ vậy người dùng không rơi vào cảnh bấm chơi rồi mới gặp lỗi giữa chừng.

Mọi phiên chơi đều có một thanh thông tin mỏng phủ ở mép trên màn hình: tên môi trường, chế độ chơi, mô hình đang dùng, và nút quay về menu (hoặc phím Esc). Mép dưới là dòng nhắc phím điều khiển. Người dùng vì thế luôn biết mình đang ở tổ hợp thực nghiệm nào, và quay lại menu đổi lựa chọn lúc nào cũng được mà không phải khởi động lại ứng dụng.

5.4 Nạp mô hình ONNX tại thời gian chạy

Mỗi lần huấn luyện ở Chương 4 kết thúc, ML-Agents xuất chính sách đã học ra một tệp ONNX. Muốn dùng được chúng trong ứng dụng thì phải sao chép vào cây thư mục tài nguyên của Unity, theo quy ước mỗi môi trường một thư mục con:

```
Assets/GameHub/Resources/AgentModels/{CrossTheRoad |
CaptureTheFlag | Football}/
```

Tên tệp giữ đúng bằng mã của lần chạy huấn luyện - ctr01, ctf_mappo_v2 chẳng hạn - và cũng chính tên đó là nhãn hiển thị trong menu. Mô hình trong ứng dụng vì vậy luôn truy vết ngược được về đúng thư mục kết quả và đúng biểu đồ đã phân tích ở Chương 4. Unity nạp mỗi tệp ONNX thành một tài sản `ModelAsset` của gói suy luận tích hợp; ứng dụng nhờ đó liệt kê được danh sách mô hình bằng lời gọi `Resources.LoadAll`, rồi gán mô hình đã chọn cho tác nhân qua giao diện `SetModel` của ML-Agents, như đoạn mã dưới đây.

```
1 ModelAsset[] models =
2     Resources.LoadAll<ModelAsset>("AgentModels/" + envName);
3
4 agent.SetModel(behaviorName, selectedModel);
5 behaviorParameters.BehaviorType = BehaviorType.InferenceOnly;
```

Listing 1: Liệt kê và gán mô hình ONNX tại thời gian chạy

Cách tổ chức này tách hẳn dữ liệu mô hình khỏi mã nguồn. Danh mục mô hình trong menu phản ánh trực tiếp nội dung thư mục tài nguyên, nên thêm mô hình mới chỉ là chuyện sao chép tệp. Mà thao tác sao chép ấy giờ do chính kịch bản xuất dữ liệu huấn luyện lo (Mục 5.5): sau mỗi đợt huấn luyện, kịch bản lấy tệp ONNX cuối cùng của từng lần chạy, đặt tên theo mã lần chạy, ghi vào đúng thư mục con của

môi trường. Ứng dụng nhờ đó luôn đóng gói **toàn bộ mười lăm mô hình** ứng với mười lăm lần chạy của Chương 4, liệt kê ở Bảng 5.2. Mỗi môi trường có đủ năm chính sách của năm thuật toán đặt cạnh nhau, so sánh hành vi bằng mắt ngay trên cùng một nhiệm vụ.

Bảng 5.2: Mười lăm mô hình ONNX được đóng gói sẵn trong ứng dụng

Môi trường	Tập mô hình	Thuật toán	Số bước
Cross The Road	ctr01	PPO	2,00M
	CrossTheRoad_SAC_01	SAC	2,00M
	ctr_poca_v1	MA-POCA	2,00M
	ctr_mappo_v1	MAPPO	2,00M
	ctr_dqn_v1	DQN	2,00M
Capture The Flag	ctf_ppo_v1	PPO	1,70M
	ctf_sac_v1	SAC	1,64M
	ctf_poca_v2	MA-POCA	1,70M
	ctf_mappo_v2	MAPPO	1,70M
	ctf_dqn_v1	DQN	1,70M
Football Table	FB01	PPO (self-play)	1,60M
	Football_SAC_01	SAC (self-play)	1,60M
	Football_POCA_01	MA-POCA (self-play)	1,60M
	Football_MAPPO_01	MAPPO (self-play)	1,60M
	football_dqn_v1	DQN (rời rạc)	1,60M

Một chi tiết kỹ thuật phải xử lý riêng khi đóng gói đủ mười lăm mô hình là *không gian hành động*. Trên Cross The Road và Capture The Flag, cả năm thuật toán đều dùng hành động rời rạc nên mọi tập ONNX đều nạp được vào scene gốc. Riêng Football Table, bốn thuật toán PPO, SAC, MA-POCA và MAPPO sinh chính sách hành động liên tục tám kênh, còn DQN chỉ định nghĩa được trên hành động rời rạc nên lần chạy football_dqn_v1 dùng phiên bản tám nhánh ba mức của cùng bài toán (Mục 3.3). Không gian hành động được khai báo trong thành phần BehaviorParameters của scene và ML-Agents dựng bộ truyền động của tác nhân ngay trong pha OnEnable, tức trước khi ứng dụng kịp can thiệp sau khi scene nạp xong; vì thế không thể đổi kiểu hành động tại thời gian chạy như cách đổi mô hình. Giải pháp là giữ song song hai bản scene Football: bản liên tục dùng cho bốn thuật

toán kia và bản rời rạc `FootballDiscrete` sinh tự động từ bản gốc bằng một kịch bản trong trình soạn thảo. Khi người dùng chọn mô hình DQN, menu nạp bản rời rạc thay cho bản gốc; phần còn lại của ứng dụng không thay đổi vì lớp tác nhân của `Football` vốn đã chấp nhận cả hai kiểu hành động và tự quy đổi ba mức $\{0, 1, 2\}$ về $\{-1, 0, +1\}$. Hệ quả duy nhất người dùng nhìn thấy là ở chế độ *Agent tự chơi* của `Football Table`, hai ô chọn mô hình phải cùng loại hành động: menu khóa nút bắt đầu kèm dòng cảnh báo nếu ghép DQN với một thuật toán liên tục, vì một trận đấu chỉ nạp được một scene.

5.5 Trực quan hóa kết quả huấn luyện trong ứng dụng

Thứ tách ứng dụng này khỏi một demo game thông thường là hai tab số liệu nhúng thẳng vào menu. Số liệu hiển thị ở đó không do ai gõ tay, mà trích tự động từ chính các tệp nhật ký huấn luyện. Mỗi lần huấn luyện lại, chỉ cần một lệnh là ứng dụng có số liệu mới.

5.5.1 Đường ống dữ liệu từ *TensorBoard* vào game

Khi huấn luyện, `ML-Agents` ghi các chỉ số ra tệp `tfevents` trong thư mục `results/<run_id>/`. Kịch bản Python `export_training_data.py` đọc những tệp này bằng `EventAccumulator` của `TensorBoard` rồi chạy bốn bước xử lý nối tiếp. Bước một, gộp chuỗi `Environment/Cumulative Reward` của mọi tệp nhật ký thuộc cùng một lần chạy và sắp theo số bước. Bước hai, làm mượt chuỗi bằng trung bình trượt để bớt nhiễu giữa các tệp. Bước ba, rút gọn chuỗi đã làm mượt xuống tối đa một trăm hai mươi điểm cách đều, giữ kích thước tệp đủ nhỏ để nạp trong game. Bước bốn, tính các thống kê tóm tắt - phần thưởng cuối, phần thưởng tốt nhất, trung bình mười phần trăm cuối, bước hội tụ ước lượng, tổng số bước.

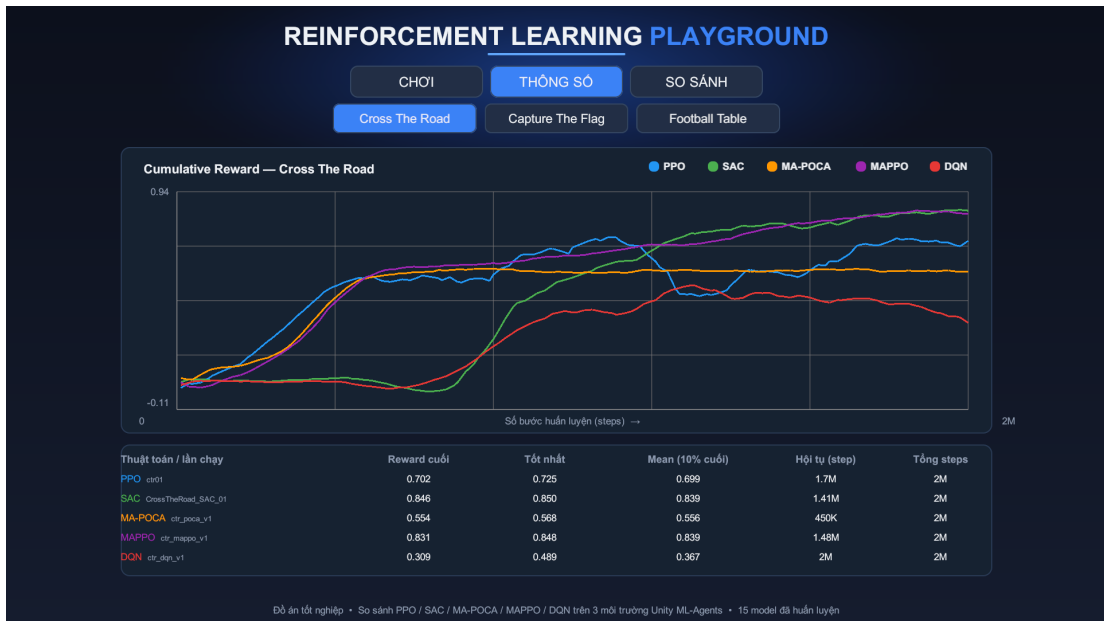
Cũng trong lượt chạy đó, kịch bản làm thêm hai việc để dữ liệu và mô hình không bao giờ lệch pha. Việc thứ nhất: sao chép tệp ONNX cuối cùng của từng lần chạy vào thư mục tài nguyên của ứng dụng dưới đúng tên mã lần chạy, bỏ qua tệp đã trùng kích thước để Unity khởi nạp lại. Việc thứ hai: xác định không gian hành động của từng mô hình bằng cách đọc thẳng tên tenxơ đầu ra trong tệp ONNX, rồi ghi kết quả vào JSON để ứng dụng biết trước mô hình nào cần bản scene rời rạc.

Có hai bước đó, một lệnh duy nhất sau mỗi đợt huấn luyện là đủ để biểu đồ, bảng thống kê và danh mục mô hình trong menu cùng được cập nhật.

Kết quả cuối cùng gom vào đúng một tệp, `Resources/TrainingData/training_data.json`. Cấu trúc của nó phân cấp: danh sách môi trường, mỗi môi trường một danh sách thuật toán, mỗi thuật toán gồm mã lần chạy, tên tệp mô hình, không gian hành động, các thống kê tóm tắt, cùng hai mảng song song `cs` (số bước) và `cv` (giá trị phần thưởng đã làm mượt). Tệp hiện hành chứa đủ mười lăm lần chạy của Chương 4. Bên phía Unity, lớp `TrainingDataStore` nạp tệp một lần duy nhất bằng `JsonUtility` rồi lưu đệm. Năm thuật toán đều có màu cố định - PPO xanh dương, SAC xanh lá, MA-POCA cam, MAPPO tím, DQN đỏ - dùng thống nhất cho biểu đồ, chú giải lẫn bảng, và trùng với bảng màu của các đồ thị ở Chương 4 để người xem đối chiếu giữa hai nơi mà khỏi phải đọc lại chú giải. Mã lần chạy được lưu kèm, nên mọi con số hiển thị trong ứng dụng đều truy ngược được về đúng thư mục kết quả đã dùng phân tích ở Chương 4. Mã đó cũng chính là tên tệp mô hình trong tab *CHƠI*: từ một dòng trong bảng thống kê, người dùng chọn ngay được mô hình tương ứng để xem nó chơi thật.

5.5.2 Tab THÔNG SỐ

Tab *THÔNG SỐ* (Hình 5.4) hiện đường cong phần thưởng tích lũy của mọi thuật toán đã huấn luyện trên môi trường đang chọn, kèm bảng thống kê phía dưới. Biểu đồ do thành phần `UILineChart` vẽ. Đây là một `MaskableGraphic` tự sinh lưới tam giác trong hàm `OnPopulateMesh`: mỗi đoạn của đường cong dựng thành một hình chữ nhật độ dày cố định, kèm lưới nền và trục. Vẽ trực tiếp bằng mesh như vậy khiến biểu đồ không lệ thuộc thư viện đồ thị ngoài nào mà vẫn co giãn đúng theo kích thước màn hình. Miền giá trị hai trục thì tính lại mỗi lần đổi môi trường, từ chính dữ liệu đang vẽ, có đệm thêm mười phần trăm để đường cong không chạm mép khung.



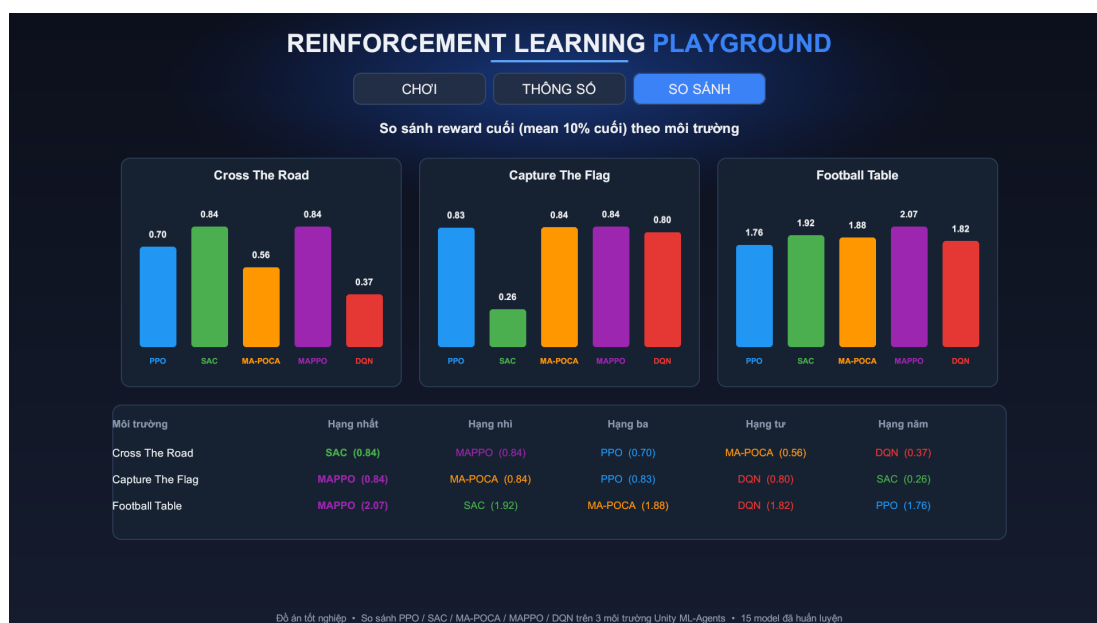
Hình 5.4: Tab THÔNG SỐ: đường cong phần thưởng tích lũy và bảng thống kê của năm thuật toán trên môi trường Cross The Road

Bảng thống kê nằm dưới biểu đồ, liệt kê theo từng thuật toán các cột: phần thưởng cuối, phần thưởng tốt nhất, trung bình mười phần trăm cuối, bước hội tụ ước lượng, tổng số bước. Cột đầu ghi tên thuật toán kèm mã lần chạy ở cỡ chữ nhỏ, để mỗi dòng chỉ đích danh mô hình đã sinh ra con số đó. Thuật toán nào chưa huấn luyện trên môi trường đang xem thì hiện “N/A” chứ không bị ẩn đi - người xem nhờ vậy thấy rõ phạm vi thực nghiệm của đề tài tới đâu. Đại lượng vẽ ở đây là phần thưởng tích lũy môi trường (Environment/Cumulative Reward) lấy nguyên từ nhật ký huấn luyện, nên thang giá trị có thể lệch so với các đường cong phần thưởng ngoại sinh ở Chương 4. Hai tab số liệu trong ứng dụng phục vụ tra cứu nhanh, không thay thế phần phân tích chi tiết của báo cáo.

5.5.3 Tab SO SÁNH

Tab SO SÁNH (Hình 5.5) đưa ba môi trường lên cùng một màn hình. Mỗi môi trường là một thẻ chứa biểu đồ cột, mỗi cột là một thuật toán với chiều cao tỉ lệ theo phần thưởng trung bình mười phần trăm cuối và được tô đúng màu quy ước; giá trị số được ghi ngay trên đỉnh cột. Do thang phần thưởng của ba môi trường rất khác nhau (Football Table dùng thang lớn hơn hẳn hai môi trường còn lại), chiều cao cột được chuẩn hóa theo giá trị lớn nhất trong từng thẻ chứ không dùng chung

một trục; nhờ vậy việc so sánh luôn diễn ra trong nội bộ một môi trường, đúng với kết luận xuyên suốt của đề tài rằng thứ hạng thuật toán chỉ có ý nghĩa khi gắn với một dạng bài toán cụ thể. Bảng xếp hạng phía dưới liệt kê đủ cả năm vị trí của từng môi trường chứ không cắt ở ba vị trí đầu, cho phép đọc ngay kết luận so sánh - kể cả thuật toán xếp cuối - mà không cần nhìn biểu đồ. Có một chỗ vênh cần nói rõ: bảng xếp hạng trong ứng dụng sắp thứ tự thuận theo phần thưởng, nên với Football Table nó không trùng với thứ hạng ở Bảng 4.5, nơi môi trường đối kháng được xếp theo mức tăng chỉ số ELO như đã lập luận ở Mục 4.



Hình 5.5: Tab SO SÁNH: biểu đồ cột theo từng môi trường và bảng xếp hạng thuật toán

5.6 Ba chế độ chơi trong từng môi trường

Ba môi trường khác nhau về bản chất - đơn tác nhân, hợp tác, đối kháng - nên ba chế độ chơi phải ánh xạ vào mỗi nơi theo một cách riêng. Bảng 5.3 tổng hợp lại.

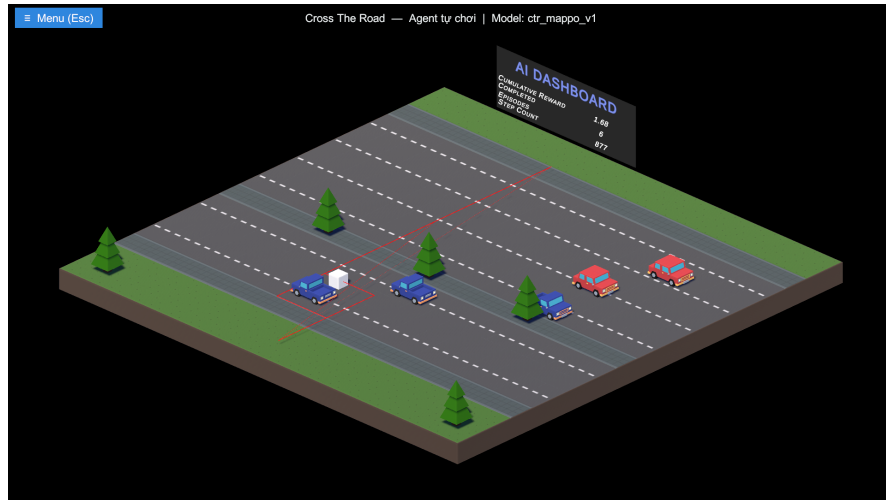
Bảng 5.3: Ảnh xạ ba chế độ chơi vào ba môi trường

Môi trường	Người chơi	Agent tự chơi	Chơi với Agent
Cross The Road	Người điều khiển nhân vật trong khu vực	Mô hình điều khiển nhân vật trong khu vực	Người và tác nhân đua song song ở hai khu vực liền kề
Capture The Flag	Hai người chơi trên cùng bàn phím	Hai tác nhân cùng dùng một mô hình	Người phối hợp cùng tác nhân đồng đội
Football Table	Người đấu với bot luật đơn giản	Mô hình đấu với mô hình (cùng loại hành động)*	Người đấu với mô hình
* Hai ô mô hình phải cùng dùng hành động liên tục hoặc cùng rời rạc, vì mỗi loại chạy trên một bản scene riêng; xem Mục 5.4.			

5.6.1 Cross The Road: trải nghiệm và quan sát chính sách

Chế độ *Người chơi* cho người dùng điều khiển nhân vật bằng phím mũi tên (hoặc A/D/W), đúng theo bốn hành động rời rạc mà tác nhân được phép dùng khi huấn luyện: đứng yên, sang trái, sang phải, tiến lên. Ràng buộc người chơi vào đúng không gian hành động của tác nhân là chủ ý - có thể người dùng mới đánh giá đúng độ khó của bài toán, chứ được điều khiển tự do hơn tác nhân thì rất dễ đi tới kết luận sai lệch.

Chế độ *Agent tự chơi* chuyển mọi tác nhân trong scene sang suy luận bằng mô hình đã chọn, đồng thời camera tự đóng khung vào vùng chơi (Hình 5.6). Thanh thông tin phía trên hiện tên mô hình đang chạy; bảng chỉ số dựng sẵn trong môi trường hiện phần thưởng tích lũy, số tập đã hoàn thành và số bước của tập hiện tại. Người xem nhờ vậy đối chiếu được ngay hành vi quan sát được với chỉ số mà mô hình đạt. Đây cũng là cách trực quan nhất để thấy khác biệt giữa các thuật toán: mô hình SAC băng qua đường với ít va chạm hơn, còn mô hình MAPPO hay dừng lâu hơn ở những làn mật độ phương tiện cao - khớp với thứ hạng phần thưởng đã trình bày ở Chương 4.

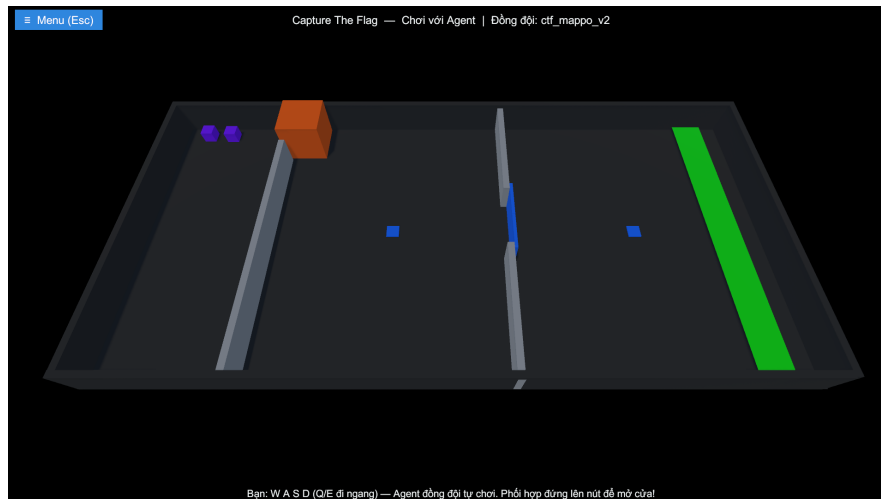


Hình 5.6: Chế độ Agent tự chơi trong Cross The Road: mô hình *ctr_mappo_v1* điều khiển tác nhân, chỉ số hiển thị trực tiếp trong cảnh

Chế độ còn lại, *Chơi với Agent*, thiết kế theo kiểu đua song song. Ứng dụng giữ lại hai khu vực huấn luyện cạnh nhau và tắt số còn lại, giao khu vực bên trái cho người chơi, khu vực bên phải cho tác nhân máy, rồi tính lại khung nhìn sao cho cả hai cùng nằm gọn trong màn hình. Scene Cross The Road hiện có tám khu vực huấn luyện nên chế độ này dùng được thoải mái; số khu vực dư chẳng ảnh hưởng gì vì chúng bị tắt ngay khi scene nạp xong.

5.6.2 Capture The Flag: phối hợp người - máy

Môi trường hợp tác mở ra một kịch bản khá thú vị: con người phối hợp với tác nhân máy trong cùng một đội. Ở chế độ *Chơi với Agent*, người chơi cầm một nhân vật bằng phím W A S D (kèm Q/E để đi ngang), nhân vật thứ hai do mô hình đã chọn điều khiển (Hình 5.7). Nhiệm vụ đòi hỏi đúng chuỗi phối hợp đã phân tích ở Mục 3.4 - một bên đứng lên bàn đạp giữ cửa, bên kia đi qua, rồi đổi vai. Người chơi vì thế phải đoán được ý định của đồng đội máy và hành động sao cho ăn khớp, minh họa trực tiếp cho khái niệm phối hợp đa tác nhân.



Hình 5.7: Chế độ Chơi với Agent trong Capture The Flag: người chơi phối hợp cùng tác nhân đồng đội dùng mô hình *ctf_mappo_v2*

Chế độ *Người chơi* thì cả hai nhân vật đều do con người cầm, kiểu hai người chơi chung một bàn phím: một người dùng cụm W A S D (Q/E đi ngang), người kia dùng cụm phím mũi tên (dấu ngoặc vuông để đi ngang). Còn chế độ *Agent tự chơi* giao cả hai nhân vật cho cùng một mô hình - tái hiện đúng cấu hình lúc huấn luyện, vì hai tác nhân vốn dùng chung một chính sách.

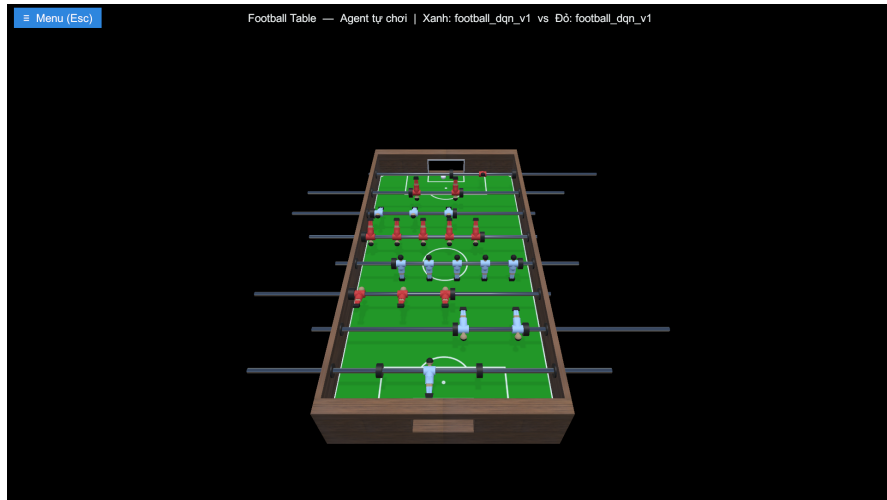
5.6.3 Football Table: đối đầu người - máy và máy - máy

Ở môi trường đối kháng, chế độ *Chơi với Agent* cho người chơi cầm đội Xanh đấu trực tiếp với mô hình bên đội Đỏ. Mỗi đội có bốn thanh gạt với tám kênh điều khiển liên tục, nên sơ đồ phím phải theo nguyên tắc “một thanh tại một thời điểm”: phím Q/E hoặc các phím số 1-4 chọn thanh đang điều khiển, W/S trượt thanh, A/D xoay thanh để sút (Hình 5.8). Tên thanh đang chọn - thủ môn, hậu vệ, tiền vệ, tiền đạo - hiện ở góc trên trái màn hình. Giá trị hành động do người chơi sinh ra còn được nhân với dấu của đội, y như cách chuẩn hóa hệ tọa độ khi huấn luyện, nên cùng một sơ đồ phím dùng được cho cả hai phía sân. Sang chế độ *Người chơi*, đối thủ là bot luật cố định bám theo vị trí bóng - đúng đối thủ mốc đã dùng khi huấn luyện ở Mục 3.3. Độ khó vừa phải, hợp để người dùng làm quen sơ đồ điều khiển trước khi đối đầu các mô hình đã huấn luyện.



Hình 5.8: Chế độ Người chơi trong Football Table: người chơi cầm đội Xanh, góc trên trái hiển thị thanh gạt đang được chọn

Thú vị nhất là chế độ *Agent tự chơi* với hai ô chọn mô hình độc lập. Người dùng có thể cho mô hình PPO huấn luyện bằng tự chơi đấu với MA-POCA, MAPPO hay SAC, hoặc cho hai bản sao của cùng một mô hình gặp nhau; thanh thông tin phía trên hiển thị tên mô hình của cả hai đội cùng lúc. Riêng DQN chỉ đấu được với chính nó, vì đây là chính sách rời rạc duy nhất trên môi trường này, và trận đấu ấy chạy trên bản scene rời rạc (Hình 5.9). Về bản chất, đây là một công cụ đánh giá đối kháng trực tiếp thu nhỏ, bổ sung kênh so sánh định tính cho các kết quả định lượng ở Chương 4 - bởi hai mô hình có phần thưởng trung bình sát nhau vẫn có thể cho ra thế trận rất khác khi gặp trực tiếp.



Hình 5.9: Chế độ Agent tự chơi trong Football Table với mô hình *football_dqn_v1*: ứng dụng nạp bản scene rời rạc và chỉ giữ lại một bàn trong số tám bàn của cấu hình huấn luyện

5.7 Đóng gói ba môi trường thành gói dùng lại được

Ba môi trường của đề tài sinh ra cho một mục đích cụ thể là so sánh thuật toán, nhưng bản thân chúng có giá trị dùng lại. Một lập trình viên Unity muốn thử học tăng cường thường tiêu phần lớn thời gian ở khâu dựng môi trường, chứ đâu phải ở khâu chọn thuật toán. Vì thế đề tài gói cả ba môi trường thành một gói UPM (*Unity Package Manager*) mang tên `com.vanhuyen.rl-game-envs`, cài được vào bất kỳ dự án Unity nào như mọi gói chính thức khác.

5.7.1 Cấu trúc gói

Gói theo đúng bố cục chuẩn của UPM, tóm tắt ở Bảng 5.4. Nhưng điểm đáng nói không phải danh sách thư mục, mà là ranh giới giữa chúng. Toàn bộ mã điều khiển môi trường nằm trong Runtime và biên dịch thành một assembly riêng, tách khỏi Assembly-CSharp của dự án. Nhờ vậy chính trình biên dịch cạnh giữ tính độc lập của gói: một lớp môi trường lỡ tham chiếu tới mã của ứng dụng trình diễn là dự án hỏng biên dịch ngay, chứ không phải đợi tới khi ai đó cài gói mới lòi ra.

Bảng 5.4: Bố cục gói *com.vanhuyen.rl-game-envs*

Thư mục	Nội dung
Runtime/	36 lớp điều khiển của ba môi trường (tác nhân, quan sát, hàm phần thưởng, sinh vật cản, trọng tài) cùng giao diện <code>IManualActionSource</code> ; một assembly riêng.
Editor/	Trình hướng dẫn cài đặt và bốn công cụ: nhân khu vực huấn luyện, build headless, sinh bản scene hành động rời rạc, cài cấu hình huấn luyện.
Configs/	Mười lăm tệp YAML cho năm thuật toán trên ba môi trường.
Samples~/	Bốn mẫu import được từ Package Manager: ba môi trường (scene, prefab, vật liệu, thiết lập vật lý) và bộ cấu hình huấn luyện.
Documentation~/	Tài liệu hướng dẫn, kèm README, CHANGELOG và LICENSE ở gốc gói.

5.7.2 Gỡ phụ thuộc ngược từ môi trường vào ứng dụng

Trước khi đóng gói được, phải gỡ một phụ thuộc sai chiều vốn đã nằm sẵn ở đó. Hàm *Heuristic* của cả ba tác nhân từng gọi thẳng *GetComponent* tới các lớp cụ thể của ứng dụng trình diễn để lấy hành động người chơi nhập vào. Quan hệ đó ngược với thứ bậc đúng: môi trường là tầng dưới, lẽ ra chẳng biết gì về ứng dụng nhúng nó. Trong một dự án đơn lẻ thì vô hại thôi, nhưng tách gói ra là thành lỗi biên dịch thật sự, vì một gói không được phép tham chiếu tới assembly mặc định của dự án.

Đề tài đảo chiều phụ thuộc bằng một giao diện đặt trong chính gói:

```

1 public interface IManualActionSource
2 {
3     void WriteActions(in ActionBuffers actionsOut);
4 }
```

Listing 2: Giao diện cho nguồn điều khiển thủ công

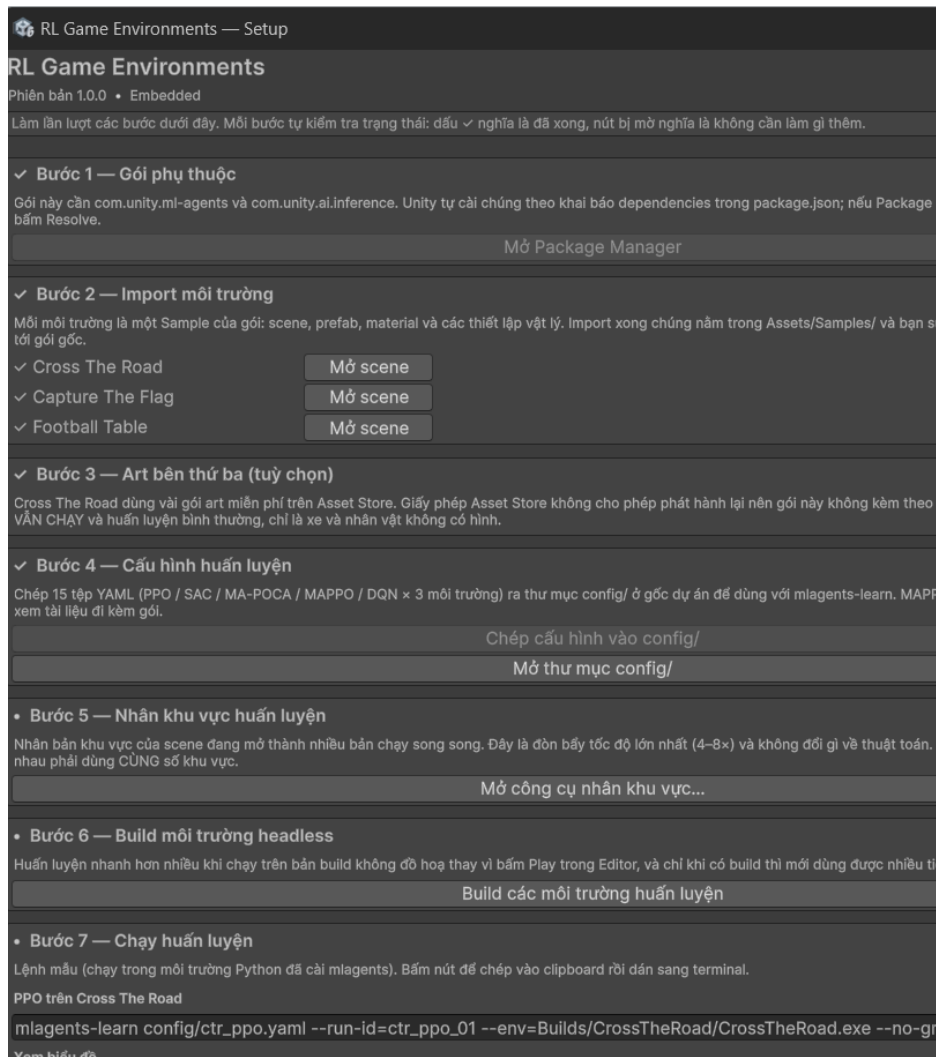
Hàm *Heuristic* của môi trường giờ chỉ hỏi đúng một câu: “trên *GameObject* này có thành phần nào hiện thực giao diện đó không?”. Có thì giao quyền quyết định, không thì quay về sơ đồ phím mặc định. Ứng dụng trình diễn trở thành một bên hiện thực giao diện như mọi bên khác. Còn người dùng gói, muốn thêm cách điều khiển mới - gamepad, kịch bản kiểm thử tự động, hay chính một mô hình ngôn

ngữ lớn - chỉ việc viết một `MonoBehaviour` rồi gắn vào, khởi động tới dòng nào trong mã môi trường.

Kèm theo đó còn một chi tiết phải xử lý: hai không gian hành động của Football Table đã nói ở Mục 5.4. Để mỗi nguồn điều khiển tự lo việc ghi ra hành động liên tục hay rời rạc thì đoạn quy đổi ba mức sẽ bị chép đi chép lại khắp nơi. Gói vì vậy cung cấp sẵn một hàm tiện ích: nguồn điều khiển chỉ cần sinh ý định trong đoạn $[-1, 1]$, còn brain nào đang chạy là việc của gói.

5.7.3 *Trình hướng dẫn cài đặt*

Rào cản lớn nhất với người mới không phải thiếu tài liệu, mà là không biết mình đang đứng ở đâu trong tài liệu đó. Đề tài vì thế viết một cửa sổ Editor (Hình 5.10) chia toàn bộ quy trình thành bảy bước. Mấu chốt nằm ở chỗ mỗi bước *tự kiểm tra trạng thái hiện tại* rồi hiện dấu tích hoặc nút hành động tương ứng, chứ không chỉ liệt kê việc cần làm.



Hình 5.10: Trình hướng dẫn cài đặt của gói: bảy bước, mỗi bước tự kiểm tra trạng thái và chỉ bật nút khi thực sự còn thiếu

Bảy bước lần lượt gồm: kiểm tra gói phụ thuộc; import môi trường từ mục Samples của Package Manager; kiểm tra các gói art bên thứ ba; chép cấu hình huấn luyện ra thư mục `config/`; nhân khu vực huấn luyện; build môi trường headless; và cuối cùng là lệnh mẫu để chạy huấn luyện, kèm nút chép sang clipboard. Ba bước giữa gọi lại đúng những công cụ mà đề tài đã dùng cho phần thực nghiệm của chính mình - nghĩa là người dùng gói đi theo một quy trình đã được kiểm chứng, chứ không phải quy trình viết riêng cho tài liệu.

Bước kiểm tra art bên thứ ba tồn tại vì một ràng buộc pháp lý. Môi trường Cross The Road hiển thị bằng vài gói art miễn phí trên Unity Asset Store. Giấy phép Asset Store cho phép dùng trong dự án của người đã tải, nhưng cấm đóng gói lại để phát

hành cho người khác - kể cả khi gói art đó miễn phí. Gói của đề tài vì vậy không kèm chúng. Đây là hạn chế thật chứ chẳng phải chi tiết thủ tục, nên trình hướng dẫn xử lý rất rõ ràng: quét dự án xem thiếu gói nào, nêu đích danh, mở thẳng trang tải, và nói luôn rằng thiếu art thì môi trường vẫn huấn luyện bình thường - chỉ là xe với nhân vật không có hình. Riêng phần mã nguồn và cấu hình do đề tài viết thì phát hành theo giấy phép MIT.

5.7.4 *Dự án tự dùng chính gói của mình*

Gói nằm ở dạng nhúng ngay trong thư mục Packages/ của dự án, và ứng dụng trình diễn ở các mục trước tiêu thụ nó đúng theo cách một dự án bên ngoài sẽ làm: qua ranh giới assembly, qua giao diện công khai, không đường tắt nào. Cách bố trí ấy biến toàn bộ phần thực nghiệm lẫn phần trình diễn của đồ án thành một phép kiểm thử thường trực cho gói. Mỗi lần chạy lại các luồng ở Bảng 5.5 là một lần xác nhận gói còn dùng được - thay vì phải đặt niềm tin vào một bản đóng gói chỉ được thử đúng một lần lúc phát hành.

5.8 Các vấn đề kỹ thuật và giải pháp

Ghép vòng lặp quyết định của ML-Agents với tương tác thời gian thực của người chơi làm nảy sinh một loạt vấn đề kỹ thuật khá đặc thù. Dưới đây là những vấn đề tiêu biểu cùng cách giải quyết.

Độ trễ điều khiển do chu kỳ quyết định: Khi huấn luyện, các tác nhân chỉ yêu cầu quyết định sau mỗi vài bước vật lý (thành phần `DecisionRequester`) nhằm giảm chi phí tính toán. Tần suất này khiến nhân vật phản hồi phím rất chậm ở chế độ người chơi. Giải pháp là khi cấu hình tác nhân cho người điều khiển, chu kỳ quyết định được hạ xuống 1 - tác nhân yêu cầu quyết định ở mọi bước vật lý - trong khi các tác nhân máy giữ nguyên chu kỳ gốc để tái hiện đúng điều kiện huấn luyện.

Mất phím bấm giữa hai lần quyết định: Hàm `Heuristic` của `Cross The Road` đọc sự kiện nhấn phím tức thời. Người chơi bấm vào một khung hình không trùng thời điểm lấy quyết định là thao tác đó rơi mất. Đề tài thêm một thành phần đệm đầu vào: phím được ghi nhận trong vòng lặp `Update`, tức mỗi khung hình, rồi giữ lại cho tới khi hàm `Heuristic` tiêu thụ ở lần quyết định kế tiếp. Không thao tác

nào bị mất nữa.

Đóng khung camera với máy quay trực giao: Scene Cross The Road dùng một máy quay trực giao bao quát toàn bộ khu vực huấn luyện; khi số khu vực còn hoạt động thay đổi theo chế độ chơi, khung hình gốc không còn phù hợp. Ứng dụng tính hộp bao của các khu vực còn hoạt động, dời máy quay về trục nhìn đi qua tâm hộp và đặt lại kích thước trực giao bằng cách chiếu tám đỉnh hộp lên hai trục ngang - dọc của máy quay. Phép đóng khung này được hoãn đến cuối khung hình đầu tiên, vì tại thời điểm scene vừa nạp xong, các phương tiện chưa được đặt về đúng vị trí xuất phát nên hộp bao tính sớm sẽ bị sai lệch.

Xung đột thứ tự khởi tạo trong Football Table: Thành phần Agent của ML-Agents có thể gọi `OnEpisodeBegin` - kéo theo việc đặt lại bàn và bóng - ngay trong pha `OnEnable`, tức trước khi bộ khởi tạo của scene kịp xong. Kết quả là lỗi tham chiếu rỗng xuất hiện ngắt quãng. Cách sửa là làm cho hai lớp `Team` và `Ball` tự khởi tạo an toàn: thao tác đặt lại sẽ tự gọi khởi tạo nếu phát hiện chưa khởi tạo, còn khởi tạo lặp thì chặn bằng cờ trạng thái.

Tác nhân thừa ở chế độ người chơi: Ở Football Table chế độ *Người chơi*, đội Đỏ do bot luật cầm trực tiếp, nhưng tác nhân ML-Agents của đội này vẫn nằm trong scene và có thể sinh ra hành động nhiễu. Giải pháp gồm hai phần: vô hiệu hóa bộ yêu cầu quyết định của tác nhân đó để nó không bao giờ đòi quyết định mới, và đặt giới hạn bước của tập về không. Nhờ vậy các sự kiện bàn thắng cùng việc đặt lại trận đấu vẫn chạy bình thường.

Nhiều khu vực huấn luyện trong một scene: Để thu kinh nghiệm nhanh hơn, ba scene môi trường đã nhân khu vực huấn luyện lên tám bản chạy song song, các bản sao nằm gọn trong một đối tượng chứa chung. Nhưng ứng dụng ban đầu viết cho scene một khu vực, nên nó xác định khu vực bằng đối tượng gốc của tác nhân - mà sau khi nhân bản thì mọi bản sao chung một đối tượng gốc, và cách xác định đó sụp đổ. Football Table lãnh hậu quả nặng nhất: quy tắc “đội Đỏ là tác nhân đầu tiên không phải đội Xanh” chọn trúng một tác nhân Xanh của bản khác, nên trận đấu chẳng bao giờ hình thành còn mười bốn tác nhân kia vẫn chạy ở cấu hình huấn luyện. Cách sửa là xác định khu vực theo cấu trúc cây: đi ngược lên cha cho tới ngay dưới đối tượng chứa chung, nhờ đó cả bản gốc lẫn bản sao đều trả về đúng

khu vực của mình. Trên nền đó, mỗi chế độ chơi giữ lại đúng số khu vực cần - một bàn cho Football Table và Capture The Flag, hai khu vực cho chế độ đua song song của Cross The Road - tắt phần còn lại, rồi mới chọn tác nhân trong phạm vi khu vực đã giữ.

Đóng khung camera cho vùng chơi dài và dẹt: Việc chỉ giữ lại một khu vực khiến khung nhìn gốc của scene không còn phù hợp, nhưng cách đóng khung ban đầu - đặt camera cách tâm một khoảng tỉ lệ với bán kính hình cầu bao vùng chơi - lại đẩy camera ra quá xa với những vùng chơi dài và dẹt như bàn bi lắc, khiến bàn chỉ chiếm một phần nhỏ màn hình. Đề tài thay bằng phép chiếu tám đỉnh hộp bao lên ba trục của camera: với máy quay trực giao thì tính kích thước trục giao vừa khít, còn với máy quay phối cảnh thì tính khoảng cách nhỏ nhất để nửa chiều cao và nửa chiều rộng vẫn nằm trong góc nhìn. Cách này dùng chung cho cả ba môi trường mà không cần tham số riêng.

Không gian hành động cố định từ lúc nạp scene: Ứng dụng đòi được mô hình của một tác nhân tại thời gian chạy, nhưng không đòi được *kiểu* hành động của nó: ML-Agents dựng bộ truyền động từ BehaviorParameters ngay trong pha OnEnable, tức trước khi sự kiện nạp scene kịp báo về cho ứng dụng, và việc nạp lại chính sách sau đó vẫn dùng đặc tả hành động đã dựng. Hệ quả là mô hình DQN rời rạc của Football Table không thể chạy trên scene liên tục dù lớp tác nhân đã hỗ trợ cả hai kiểu. Giải pháp là sinh sẵn một bản sao rời rạc của scene bằng kịch bản trong trình soạn thảo và để menu chọn bản scene phù hợp với mô hình người dùng vừa chọn - đổi scene thay vì đổi đặc tả hành động, nhờ đó không phải dựng tới scene gốc dùng cho huấn luyện.

Phạt thời gian chia cho không: Trớ trêu là chính việc đặt giới hạn bước về không ở trên lại làm lộ một lỗi tiềm ẩn trong hàm phần thưởng của Football Table. Thành phần phạt thời gian tính bằng nghịch đảo của giới hạn bước; giới hạn bằng không thì giá trị phần thưởng hóa vô cực và ML-Agents ném ngoại lệ ở mỗi bước. Sửa bằng cách chỉ áp dụng phạt thời gian khi giới hạn bước lớn hơn không - đúng ngữ nghĩa “tập không giới hạn bước thì không có áp lực thời gian”. Đây là ví dụ điển hình cho chuyện chuyển một môi trường từ chế độ huấn luyện sang chế độ chơi có thể đánh thức những nhánh mã chưa từng chạy trong suốt quá trình huấn

luyện.

5.9 Kiểm thử ứng dụng và hạn chế

Ứng dụng được kiểm thử ngay trong Unity Editor, theo từng luồng chức năng. Với mỗi tổ hợp môi trường - chế độ, quá trình kiểm thử xác nhận bốn điểm. Một, scene nạp đúng và từng tác nhân được cấu hình đúng vai trò: loại hành vi, mô hình, chu kỳ quyết định. Hai, mô hình ONNX nạp thành công và tác nhân hành xử đúng như kết quả huấn luyện cho phép kỳ vọng. Ba, giao diện phủ hiện đúng thông tin và nút quay về menu chạy được. Bốn, bảng điều khiển của Unity sạch lỗi. Bảng 5.5 tổng hợp kết quả.

Bảng 5.5: Kết quả kiểm thử các luồng chức năng của ứng dụng

Luồng kiểm thử	Nội dung xác nhận	Kết quả
Menu: chọn môi trường, chế độ, mô hình	Giao diện cập nhật đúng; ô mô hình hiện/ẩn theo ngữ cảnh	Đạt
Menu: xem trước 3D, tab THÔNG SỐ, SO SÁNH	Xoay/phóng to mượt; biểu đồ, chú giải và bảng hiện đủ năm thuật toán, khớp training_data.json	Đạt
Menu: duyệt danh mục mô hình	Mỗi môi trường liệt kê đủ 5 mô hình kèm tên thuật toán và màu quy ước	Đạt
Cross The Road - Agent tự chơi	Cả tám khu vực cùng chạy mô hình đã chọn	Đạt
Cross The Road - Chơi với Agent	Giữ lại hai khu vực liền kề, tắt sáu khu vực còn lại; người bên trái, mô hình bên phải	Đạt
Capture The Flag - Agent tự chơi	Giữ một khu vực, hai tác nhân cùng mô hình, camera đóng khung đúng	Đạt
Football Table - Người chơi	Giữ một bàn; Xanh do người, Đỏ do bot luật điều khiển	Đạt
Football Table - Agent tự chơi	Giữ một bàn; hai đội đúng hai mô hình đã chọn	Đạt
Football Table - mô hình DQN	Nạp bản scene rời rạc; cảnh báo và khóa nút bắt đầu khi ghép DQN với mô hình liên tục	Đạt
Quay về menu (nút và phím Esc)	Trở về menu, đổi lựa chọn, vào lại bình thường	Đạt
Mở scene trực tiếp (không qua menu)	Cấu hình huấn luyện gốc được giữ nguyên	Đạt
Gói môi trường: nhận diện và biên dịch	Package Manager nhận gói nhúng; assembly riêng biên dịch được, ba scene không đứt tham chiếu script	Đạt
Gói môi trường: trình hướng dẫn	Bảy bước hiển thị đúng trạng thái đã/chưa xong; các nút gọi đúng công cụ	Đạt

Về hiệu năng, việc suy luận mạng nơ-ron cho các tác nhân đồng thời, cộng thêm giao diện, không làm tốc độ khung hình sụt đáng kể trên cấu hình thực nghiệm ở

Mục 3.8.1. Điều đó khẳng định việc nhúng thẳng mô hình đã huấn luyện vào một sản phẩm game là khả thi. Cả mười lăm mô hình ONNX đi kèm ứng dụng đều nạp và vận hành chính xác, đúng phổ quan sát - hành động đã khai báo khi huấn luyện.

Ứng dụng vẫn còn hai hạn chế đã biết. Thứ nhất là dung lượng: đóng gói đủ mười lăm mô hình khiến thư mục tài nguyên phình mạnh, riêng năm mô hình của Capture The Flag đã chiếm khoảng 325 MB - gấp nhiều lần tổng dung lượng hai môi trường còn lại. Thủ phạm không phải số lượng mô hình mà là kiến trúc mạng. Quan sát tín hiệu mục tiêu của môi trường này khiến ML-Agents dựng bộ mã hóa theo kiểu siêu mạng (*hypernetwork*), trong đó một nhánh phụ sinh trọng số cho lớp ẩn chính, đẩy số tham số của một chính sách nhỏ lên gần mười bảy triệu. Muốn giảm dung lượng bản phát hành thì phải huấn luyện lại Capture The Flag với kiểu điều kiện hóa mục tiêu khác, chứ bớt mô hình đi không giải quyết được gì. Thứ hai, hai tab số liệu phản ánh nội dung tệp `training_data.json` tại thời điểm xuất, nên sau mỗi đợt huấn luyện mới phải chạy lại kịch bản xuất dữ liệu thì ứng dụng mới hiện số liệu cập nhật. Bù lại, cùng lệnh đó cũng cập nhật luôn danh mục mô hình, nên hai phần không thể lệch nhau.

6. KẾT LUẬN

6.1 Kết quả đạt được

Đồ án đã hoàn thành các mục tiêu đặt ra ban đầu. Kết quả chính là ba môi trường huấn luyện game 3D mang tính chất khác nhau trên nền tảng Unity ML-Agents. Football Table lo phần đối kháng một đối một, hành động liên tục, tự chơi. Capture The Flag lo phần hợp tác hai tác nhân với phần thưởng nhóm và chương trình học theo bậc. Cross The Road lo phần điều hướng đơn tác nhân với phần thưởng thưa. Với từng môi trường, đề tài thuật lại trọn vẹn quá trình thiết kế không gian quan sát, không gian hành động và hàm phần thưởng - kể cả những phương án đã thử rồi phải bỏ.

Phần tri thức thiết kế thu được từ quá trình đó có giá trị không kém bản thân ba môi trường. Capture The Flag là trường hợp điển hình. Đặc tả ban đầu của nó trùng phất đúng cái trạng thái phối hợp mà nhiệm vụ đòi hỏi - hai tác nhân đứng tách nhau ở hai phía cửa - nên tác nhân mắc kẹt ở đáy phần thưởng nhóm bất kể

chạy thuật toán nào. Lỗi đó không hiện ra thành lỗi chương trình, nó chỉ hiện thành một đường cong phẳng, và chỉ lộ diện khi ta ngồi xem hành vi thật. Bài học rút ra - phải kiểm tra xem hàm phần thưởng có vô tình phạt chính hành vi đích hay không - áp dụng được cho bất kỳ môi trường hợp tác nào, chẳng riêng gì môi trường này.

Ba môi trường đã được đóng thành một gói Unity Package Manager, kèm mười lăm tệp cấu hình huấn luyện, bộ công cụ Editor và trình hướng dẫn cài đặt bảy bước. Nhờ vậy chúng tồn tại như một sản phẩm dùng lại được, chứ không chỉ là mã nguồn của một đồ án. Đề tài cũng bổ sung vào ML-Agents hai thuật toán vốn không có sẵn - MAPPO và DQN - dưới dạng trainer mở rộng, đồng thời hiện thực hóa quy trình nối môi trường Unity với một thư viện học tăng cường bên ngoài qua Python Low-Level API.

Mười lăm lần huấn luyện đã xác nhận cả ba môi trường đạt yêu cầu theo ba tiêu chí đặt ra ở Chương 4. Chúng *học được*: ở cả ba môi trường đều có ít nhất bốn trong năm thuật toán tiến bộ rõ rệt so với lúc khởi tạo. Chúng *phân biệt được*: trên Cross The Road, phần thưởng cuối trải từ 0,846 của SAC xuống 0,336 của DQN; còn trên Capture The Flag, bốn thuật toán về đích trong dải 0,800–0,840 trong khi SAC sập đổ ở 0,183 - và thứ hạng ở cả hai nơi đều giải thích được bằng cơ chế của bài toán. Chúng cũng *không có lối tắt*: không lần chạy nào đạt phần thưởng sát trần mà lại không giải được nhiệm vụ khi ta quan sát trực tiếp trong ứng dụng trình diễn.

Riêng Football Table bộc lộ một giới hạn cần thừa nhận thẳng thắn. Cả bốn thuật toán ghép được với tự chơi đều tăng ELO từ ba mươi sáu tới sáu mươi bốn điểm so với phiên bản cũ của chính mình - cơ chế tự chơi hoạt động đúng. Nhưng biên độ ấy quá hẹp để môi trường phân định được các thuật toán với nhau. Ở cấu hình hiện tại, Football Table chỉ đạt hai trong ba tiêu chí. Muốn dùng nó làm thước đo so sánh thì cần ngân sách bước lớn hơn, hoặc một đối thủ mốc cố định thay cho tự chơi thuần túy.

Có một quan sát xuyên suốt cả ba môi trường ủng hộ đúng luận điểm trung tâm của đề tài: chênh lệch của cùng một thuật toán giữa hai môi trường luôn lớn hơn chênh lệch giữa các thuật toán trong cùng một môi trường. Khoảng cách giữa thành công của SAC trên Cross The Road và sự sập đổ của chính nó trên Capture The Flag lớn hơn mọi khoảng cách nội bộ. Nói cách khác, đặc tính bài toán - thứ do

thiết kế môi trường quyết định - chi phối kết quả mạnh hơn lựa chọn thuật toán. Và đó chính là lý do khâu xây dựng môi trường đáng được đầu tư nhiều hơn mức thường thấy.

Bên cạnh phần thực nghiệm so sánh, đề tài còn dựng hoàn chỉnh một ứng dụng trình diễn tương tác, trong đó cả mười lăm mô hình đã huấn luyện được triển khai thẳng vào game dưới định dạng ONNX. Người dùng ở đó chọn môi trường, chọn một trong ba chế độ chơi - tự điều khiển, tác nhân tự chơi, chơi cùng tác nhân - chọn mô hình cụ thể cho tác nhân, và tra cứu ngay đường cong huấn luyện của cả năm thuật toán trên giao diện menu. Danh mục mô hình cùng dữ liệu biểu đồ đều sinh ra từ một kịch bản duy nhất chạy trên thư mục kết quả, nên phần trình diễn và phần số liệu của Chương 4 không thể lệch nhau. Các luồng chức năng đã được kiểm thử trong Unity Editor và đều chạy đúng (Mục 5.9). Kết quả này chứng minh việc nhúng mô hình học tăng cường vào một sản phẩm game hoàn chỉnh là khả thi, với chi phí suy luận không đáng kể.

6.2 Đóng góp của đề tài

Nếu phải xếp các đóng góp theo mức hữu ích, tôi đặt bộ môi trường lên đầu. Ba môi trường ba chiều phủ ba dạng bài toán học tăng cường phổ biến, đóng thành một gói Unity Package Manager cài được vào dự án bất kỳ, kèm mười lăm cấu hình huấn luyện đã chạy thật, bộ công cụ Editor và một trình hướng dẫn cài đặt bảy bước biết tự kiểm tra trạng thái. Điều khiến nó khác việc chỉ công bố mã nguồn là ba môi trường này đã qua kiểm chứng: mỗi môi trường đã chịu năm thuật toán khác hẳn nhau về nguyên lý, nên người dùng lại biết trước rằng nó học được, nó phân biệt được, và nó không có lỗi tắt - những điều mà một môi trường mới dựng chẳng bao giờ bảo đảm nổi. Chính đồ án này tiêu thụ gói đó qua ranh giới assembly y như một dự án bên ngoài, nên phần trình diễn hóa thành phép kiểm thử thường trực cho gói.

Kế đến là phần tri thức thiết kế môi trường, thứ mà tài liệu về học tăng cường thường lướt qua. Đề tài ghi lại đầy đủ quá trình đặc tả cho ba dạng bài toán khác nhau, gồm cả những phương án thất bại và dấu hiệu nhận ra chúng: một hàm phần thưởng trừng phạt chính hành vi đích; một tác nhân thiếu quan sát về đồng đội nên không định được thời điểm phối hợp; một khoản phạt thời gian chia cho không

khi giới hạn bước bằng không. Đi cùng là các quyết định vận hành ảnh hưởng lớn nhưng ít được bàn - nhân khu vực huấn luyện để tăng thông lượng, dựng biến thể không gian hành động rời rạc cho thuật toán dựa trên giá trị - và hai con đường đưa một thuật toán không có sẵn vào Unity: gói trainer mở rộng đăng ký MAPPO cùng DQN vào chính chương trình huấn luyện của ML-Agents, và lớp bọc môi trường qua `mlagents_envs` để nối với Stable-Baselines3.

Kết quả so sánh năm thuật toán tuy chỉ là sản phẩm phụ của khâu kiểm chứng nhưng vẫn tham khảo trực tiếp được cho người làm game. Đáng chú ý nhất là một kết quả phủ định: trên bài toán hợp tác có tác nhân đồng nhất, PPO thường bám sát cả MAPPO lẫn MA-POCA trong dải chênh lệch chưa tới một phần trăm - nghĩa là chi phí tích hợp một thuật toán đa tác nhân chuyên biệt chưa chắc được đền đáp. Đi kèm nó là một kết quả gây ngạc nhiên theo chiều ngược lại: MAPPO vượt PPO một khoảng lớn trên chính môi trường đơn tác nhân, nơi lý thuyết bảo rằng phê bình tập trung của nó lẽ ra vô hiệu.

Đóng góp cuối cùng là ứng dụng trình diễn với ba chế độ chơi, cơ chế chọn mô hình cho phép đặt cạnh nhau cả mười lăm chính sách đã huấn luyện, và hai tab số liệu huấn luyện nhúng thẳng trong game. Giá trị của nó không nằm ở phần game, mà ở chỗ nó là công cụ soi lỗi đặc tả. Quan sát hành vi thật là cách duy nhất phát hiện những sai sót mà đường cong phần thưởng giấu đi, và cũng là cách khép kín quãng đường từ một tệp cấu hình YAML tới một hành vi mà người chơi cảm nhận được bằng tay: chính bảng xếp hạng ở Chương 4 có thể kiểm chứng lại bằng mắt, bằng cách cho hai mô hình khác thuật toán chạy trên cùng một nhiệm vụ. Toàn bộ mã nguồn môi trường, cấu hình huấn luyện và dữ liệu kết quả đều được công bố dưới dạng mở.

6.3 Hạn chế

Hạn chế nặng nhất của đề tài: mỗi tổ hợp môi trường - thuật toán chỉ chạy với đúng một seed. Nghĩa là mọi chênh lệch báo cáo ở Chương 4 đều chưa tách được phần do bản chất thuật toán khỏi phần do may rủi của một lần khởi tạo. Những khoảng cách lớn - DQN xếp cuối trên Cross The Road, SAC sụp đổ trên Capture The Flag - nhiều khả năng vẫn đứng vững; nhưng các thứ hạng sát nhau thì đừng đọc như

một kết luận thống kê. Đây là hệ quả của quỹ thời gian tính toán chứ không phải của một lựa chọn phương pháp luận, và nếu chỉ được sửa đúng một điều trong đề án này thì tôi sẽ sửa điều đó.

Hạn chế thứ hai nằm ở độ phân giải của phép đo. Ở Football Table, bốn thuật toán tự chơi kết thúc trong dải ELO 1237–1260. Ở Capture The Flag, bốn thuật toán dẫn đầu nằm trong dải phần thưởng 0,800–0,840. Cả hai khoảng cách đều nhỏ hơn dao động của chính các đường cong, nên ở cấu hình hiện tại hai môi trường này không phân biệt nổi các thuật toán trong nhóm dẫn đầu. Chỉ Cross The Road cho một phổ đủ rộng để việc xếp hạng có nghĩa. Muốn hai môi trường kia nói được nhiều hơn thì phải làm chúng khó lên, chứ không phải chạy chúng lâu hơn.

Còn một ô của lưới thực nghiệm nằm ngoài phép so sánh: DQN trên Football Table chạy không có tự chơi, vì ML-Agents báo lỗi khi ghép trainer off-policy với bộ điều phối đối kháng. Lần chạy đó vì thế không có ELO và học trong điều kiện khác hẳn bốn thuật toán kia, nên mọi con số của nó ở môi trường này chỉ đọc được như một tham chiếu rời.

Sau cùng, đề tài không giải thích được hai hiện tượng mà chính số liệu của mình phơi ra. Hiện tượng thứ nhất là MAPPO vượt PPO trên môi trường đơn tác nhân, nơi hai thuật toán chỉ khác nhau ở phần phê bình tập trung - thứ lẽ ra vô hiệu khi nhóm có mỗi một thành viên. Hiện tượng thứ hai là kiểu thất bại của SAC trên Capture The Flag: entropy sụp xuống gần không rồi bật ngược lên đúng mức cực đại và nằm đó tới cuối. Cả hai đều cần thêm thực nghiệm mới kết luận được; mà với một hạt giống cho mỗi ô thì cũng chưa loại được khả năng hiện tượng thứ nhất chỉ là dao động ngẫu nhiên.

6.4 Hướng phát triển

Việc phải làm trước tiên là chạy lại toàn bộ mười lăm tổ hợp với nhiều hạt giống, bởi chừng nào chưa có phương sai giữa các lần chạy thì mọi so sánh trong đề án vẫn treo lơ lửng. Song song, cần làm Football Table và Capture The Flag khó lên, vì ở cấu hình hiện tại chúng thiếu độ phân giải để tách các thuật toán dẫn đầu. Với Capture The Flag, hướng cụ thể là thêm bài trong chương trình học hoặc buộc hai tác nhân phối hợp chặt hơn, chứ không phải kéo dài thời lượng huấn luyện.

Ngay sau đó là hai câu hỏi mà đề tài đặt ra nhưng chưa trả lời nổi. Thứ nhất, vì sao MAPPO vượt PPO trên môi trường đơn tác nhân? Một thực nghiệm tách riêng phần phê bình tập trung, chạy trên nhiều hạt giống, sẽ cho biết đây là hiệu ứng thật hay chỉ là may rủi của một lần khởi tạo. Thứ hai, cơ chế nào đứng sau kiểu thất bại của SAC trên bài toán hợp tác? Giả thuyết khớp với dữ liệu là bộ đệm phát lại lưu lẫn lộn kinh nghiệm của hai bài học trong chương trình học, và cách kiểm chứng trực tiếp là chạy lại với bộ đệm tách riêng theo từng bài. Ranh giới giữa chia sẻ tham số và attention cũng đáng khảo sát, bằng một môi trường hợp tác có hai tác nhân vai trò bất đối xứng, hoặc có tác nhân rời nhóm giữa lượt chơi.

Ở tầm xa hơn, bộ so sánh có thể mở rộng bằng những thuật toán như DreamerV3 hay QMIX - việc bổ sung giờ đã dễ hơn nhiều nhờ cơ chế trainer mở rộng đã dựng sẵn - còn ba môi trường thì có thể nâng độ phức tạp lên gần với game thương mại. Riêng ứng dụng trình diễn cần một bản chạy độc lập, WebGL hoặc desktop, kèm bảng xếp hạng người chơi đấu với tác nhân và một khảo sát trải nghiệm người dùng. Đó là cách duy nhất trả lời được câu hỏi mà không chỉ số huấn luyện nào chạm tới: hành vi của tác nhân có thực sự “tự nhiên” trong mắt người chơi hay không.

Tài liệu tham khảo

- [1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [2] Richard Bellman. A markovian decision process. *Journal of Mathematics and Mechanics*, 6(5):679–684, 1957.
- [3] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 41–48, 2009.
- [4] Andrew Cohen, Ervin Teng, Vincent-Pierre Berges, Ruo-Ping Dong, Hunter Henry, Marwan Mattar, Alexander Zook, and Sujoy Ganguly. On the use and misuse of absorbing states in multi-agent reinforcement learning. *arXiv preprint arXiv:2111.05992*, 2021.
- [5] Sam Devlin and Daniel Kudenko. Dynamic potential-based reward shaping. *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 433–440, 2012.
- [6] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, pages 2974–2982, 2018.
- [7] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018.
- [8] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

- [9] Arthur Juliani, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, Yuan Gao, Hunter Henry, Marwan Mattar, and Danny Lange. Unity: A general platform for intelligent agents. *arXiv preprint arXiv:1809.02627*, 2020.
- [10] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [11] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 2017.
- [12] Ian Millington and John Funge. *Artificial Intelligence for Games*. CRC Press, Boca Raton, FL, 3 edition, 2019.
- [13] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016.
- [14] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [15] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [16] OpenAI, Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębniak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.

- [17] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International Conference on Machine Learning (ICML)*, pages 2778–2787. PMLR, 2017.
- [18] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, New York, 1994.
- [19] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- [20] Jette Randløv and Preben Alstrøm. Learning to drive a bicycle using reinforcement learning and shaping. In *Proceedings of the Fifteenth International Conference on Machine Learning (ICML)*, pages 463–471, 1998.
- [21] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1889–1897, 2015.
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.
- [23] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [24] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

- [25] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [26] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [27] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [28] Georgios N. Yannakakis and Julian Togelius. *Artificial intelligence and games*. Springer, 2018.
- [29] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Baez, Siddharth Bhatt, Jianyu Peng, Minghao Shen, Xionghao Wang, et al. The surprising effectiveness of ppo in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, volume 35, pages 24611–24624, 2022.

PHỤ LỤC

Phụ lục A. Cấu hình huấn luyện PPO

Dưới đây là tệp cấu hình YAML của PPO cho môi trường Football Table có kết hợp self-play, đúng bản đã dùng cho lần chạy FB01. Những môi trường khác giữ cấu trúc tương tự, chỉ chỉnh siêu tham số theo Bảng 3.7. Cấu hình MAPPO hoàn toàn tương tự PPO và MA-POCA trên cùng một môi trường, chỉ khác ở việc khai báo `trainer_type: mappo`; vì vậy đề tài không liệt kê riêng, và mọi chênh lệch kết quả giữa ba thuật toán này đều quy được về bản thân thuật toán.

```

1 behaviors:
2   Football:
3     trainer_type: ppo
4     hyperparameters:
5       batch_size: 2048
6       buffer_size: 40960
7       learning_rate: 3.0e-4
8       beta: 0.005
9       epsilon: 0.2
10      lambda: 0.95
11      num_epoch: 5
12      learning_rate_schedule: linear
13    network_settings:
14      normalize: true
15      hidden_units: 256
16      num_layers: 2
17    reward_signals:
18      extrinsic:
19        gamma: 0.99
20        strength: 1.0
21    self_play:
22      save_steps: 50000
23      team_change: 100000
24      swap_steps: 200000
25      window: 10
26      play_against_latest_model_ratio: 0.5
27      initial_elo: 1200.0
28    keep_checkpoints: 10
29    max_steps: 1600000
30    time_horizon: 1000
31    summary_freq: 10000

```

Listing 3: football_ppo.yaml

Phụ lục B. Cấu hình huấn luyện SAC

Tệp cấu hình YAML của SAC cho môi trường Cross The Road, đúng như đã dùng cho lần chạy CrossTheRoad_SAC_01. Tham số `steps_per_update` bằng hai mươi là điểm cần chú ý nhất: nó quyết định tốc độ hội tụ đo được của SAC, như phân tích ở Chương 4.

```

1 behaviors:
2   CrossTheRoad:
3     trainer_type: sac
4     hyperparameters:
5       batch_size: 256
6       buffer_size: 100000

```

```

7     learning_rate: 3.0e-4
8     learning_rate_schedule: constant
9     buffer_init_steps: 1000
10    tau: 0.005
11    steps_per_update: 20
12    save_replay_buffer: false
13    init_entcoef: 0.5
14    reward_signal_steps_per_update: 20
15    network_settings:
16      normalize: false
17      hidden_units: 128
18      num_layers: 2
19    reward_signals:
20      extrinsic:
21        gamma: 0.99
22        strength: 1.0
23      curiosity:
24        gamma: 0.99
25        strength: 0.02
26        learning_rate: 3.0e-4
27    keep_checkpoints: 5
28    max_steps: 2000000
29    time_horizon: 64
30    summary_freq: 10000

```

Listing 4: ctr_sac.yaml

Trên môi trường đối kháng, SAC vẫn dùng chính trainer sac của ML-Agents kết hợp khối self_play giống hệt Phụ lục A, nên lần chạy Football_SAC_01 có đầy đủ chỉ số ELO. Bên cạnh đó, đề tài cũng dựng sẵn con đường kết nối môi trường Unity với một thư viện học tăng cường bên ngoài qua Python Low-Level API (xem Mục 3.7.2). Đoạn mã dưới đây minh họa lớp bọc môi trường; cần nhấn mạnh rằng đây là phương án dự phòng đã kiểm chứng, không phải nguồn của bất kỳ số liệu nào trong Chương 4.

```

1 from mlagents_envs.environment import UnityEnvironment
2 from mlagents_envs.envs.unity_gym_env import UnityToGymWrapper
3 from stable_baselines3 import SAC
4
5 # Ket noi truc tiep voi moi truong Unity dang chay
6 unity_env = UnityEnvironment(file_name=None, base_port=5005)
7 env = UnityToGymWrapper(unity_env, allow_multiple_obs=False)
8
9 # Khoi tao tac nhan SAC voi mang 3 lop x 512 don vi
10 model = SAC(
11     "MlpPolicy", env,
12     learning_rate=3e-4,
13     buffer_size=500000,
14     batch_size=1024,
15     tau=0.005,
16     gamma=0.99,
17     ent_coef="auto",
18     policy_kwargs=dict(net_arch=[512, 512, 512]),
19     tensorboard_log="./results/football_sac_v1",
20     verbose=1,
21 )
22 model.learn(total_timesteps=10_000_000)
23 model.save("results/football_sac_v1/Football_sac")

```

```
24 env.close()
```

Listing 5: Kết nối môi trường Unity với mlagents_envs + Stable-Baselines3

Phụ lục C. Cấu hình huấn luyện MA-POCA

Tập cấu hình YAML của MA-POCA cho môi trường Capture The Flag, đúng như đã dùng cho lần chạy `ctf_poca_v2`. Cấu hình MAPPO của lần chạy `ctf_mappo_v2` trùng khớp từng tham số với tập này, chỉ khác `trainer_type: mappo`.

```
1 behaviors:
2   PuzzleBehavior:
3     trainer_type: poca
4     hyperparameters:
5       batch_size: 1024
6       buffer_size: 20480
7       learning_rate: 3.0e-4
8       beta: 0.01
9       epsilon: 0.2
10      lambda: 0.95
11      num_epoch: 3
12      learning_rate_schedule: linear
13    network_settings:
14      normalize: false
15      hidden_units: 256
16      num_layers: 3
17      vis_encode_type: simple
18    reward_signals:
19      extrinsic:
20        gamma: 0.99
21        strength: 1.0
22      curiosity:
23        gamma: 0.99
24        strength: 0.02
25        learning_rate: 3.0e-4
26    keep_checkpoints: 10
27    max_steps: 1700000
28    time_horizon: 128
29    summary_freq: 20000
```

Listing 6: ctf_poca.yaml

Phụ lục D. Cấu hình huấn luyện DQN

Tập cấu hình YAML của DQN cho môi trường Cross The Road, đúng như đã dùng cho lần chạy `ctr_dqn_v1`. DQN được cài đặt dưới dạng trainer mở rộng (Mục 3.7.2) nên khai báo `trainer_type: dqn` chỉ hoạt động khi gói mở rộng đã được cài đặt. Nhóm tham số `buffer_size`, `tau` và `steps_per_update` được đặt trùng với cấu hình SAC ở Phụ lục B, để hai thuật toán off-policy tiêu thụ dữ liệu theo cùng một nhịp. Nhóm tham số `exploration_*` là phần riêng của DQN, mô tả lịch giảm tuyến tính của hệ số ϵ trong quy tắc chọn hành động tham lam có nhiễu.

```
1 behaviors:
2   CrossTheRoad:
3     trainer_type: dqn
4     hyperparameters:
```

```

5     batch_size: 256
6     buffer_size: 100000
7     buffer_init_steps: 1000
8     learning_rate: 3.0e-4
9     learning_rate_schedule: constant
10    tau: 0.005
11    steps_per_update: 20
12    save_replay_buffer: false
13    exploration_schedule: linear
14    exploration_initial_eps: 1.0
15    exploration_final_eps: 0.05
16    exploration_decay_steps: 500000
17    reward_signal_steps_per_update: 20
18    network_settings:
19      normalize: false
20      hidden_units: 128
21      num_layers: 2
22    reward_signals:
23      extrinsic:
24        gamma: 0.99
25        strength: 1.0
26      curiosity:
27        gamma: 0.99
28        strength: 0.02
29        learning_rate: 3.0e-4
30    keep_checkpoints: 5
31    max_steps: 2000000
32    time_horizon: 64
33    summary_freq: 10000

```

Listing 7: ctr_dqn.yaml

Trên môi trường Football Table, tệp cấu hình của DQN giữ nguyên cấu trúc trên nhưng **không có** khối `self_play`: khi khai báo khối này, ML-Agents phát sinh lỗi tra cứu chính sách trong bộ điều phối đối kháng và dừng ngay khi khởi tạo. Lần chạy `football_dqn_v1` vì vậy dùng bản dựng `FootballDiscrete` với tham số hành vi chuyển từ tám kênh liên tục sang tám nhánh ba mức, và không sinh ra chỉ số ELO.

Phụ lục E. Mã nguồn minh họa

E.1. Không gian quan sát (Cross The Road):

```

1 public override void CollectObservations(VectorSensor sensor)
2 {
3     // 3 giá trị: toạ độ x, y, z của agent
4     sensor.AddObservation(transform.localPosition);
5     // 3 giá trị: toạ độ x, y, z của địch
6     sensor.AddObservation(goal.transform.localPosition);
7 }

```

Listing 8: CollectObservations - CrossTheRoadAgent.cs

E.2. Hàm phần thưởng (Cross The Road):

```

1 public void GivePoints() // Khi den dich
2 {
3     AddReward(1.0f);
4     EndEpisode();

```

```

5 }
6
7 public void TakeAwayPoints()           // Khi va chạm phương tiện
8 {
9     AddReward(-0.025f);
10    EndEpisode();
11 }

```

Listing 9: GivePoints / TakeAwayPoints - CrossTheRoadAgent.cs

E.3. Phần thưởng nhóm (môi trường Capture The Flag):

```

1 // Ca hai agent cùng roi phong -> thưởng nhóm
2 if (agents[0].agent.ThisAgentLeft && agents[1].agent.ThisAgentLeft)
3     agentGroup.AddGroupReward(0.5f / MaxEnvironmentSteps);
4
5 // Phạt thời gian (hurry-up penalty)
6 agentGroup.AddGroupReward(-0.25f / MaxEnvironmentSteps);
7
8 // Ca hai đến checkpoint -> thưởng lớn + kết thúc episode
9 if (allFound) {
10    agentGroup.AddGroupReward(reward);
11    agentGroup.EndGroupEpisode();
12 }

```

Listing 10: Group reward - EnvController.cs (trích đoạn)

E.4. Phần thưởng định hướng (Football Table):

```

1 void AddShotReward()
2 {
3     if (Stats.HasBall) {
4         Vector3 delta = opponentTeam.Goal.transform.position
5                         - ball.transform.position;
6         float reward = shotRewardMultiplier *
7                       Vector3.Dot(delta.normalized, ball.Velocity);
8         AddReward(reward);
9     }
10 }

```

Listing 11: AddShotReward - FootballAgent.cs

Phụ lục F. Kết quả TensorBoard

Toàn bộ số liệu huấn luyện (Cumulative Reward, Episode Length, Policy Loss, Entropy, ELO...) đều được ghi lại tự động qua TensorBoard cho từng lần chạy và lưu trong thư mục `results/` của dự án. Bảng 6.1 tóm tắt các chỉ số cuối cùng thực đo được của những lần chạy đã huấn luyện đầy đủ.

Bảng 6.1: Kết quả thực đo của 15 lần huấn luyện

Lần chạy	Thuật toán	Số bước	Reward TB (10% cuối)	Thời gian
Cross The Road				
ctr01	PPO	2M	0,690	~48 phút
CrossTheRoad_SAC_01	SAC	2M	0,846	~139 phút
ctr_poca_v1	MA-POCA	2M	0,556	~59 phút
ctr_mappo_v1	MAPPO	2M	0,835	~58 phút
ctr_dqn_v1	DQN	2M	0,336	~148 phút
Capture The Flag				
ctf_ppo_v1	PPO	1,70M	0,834	~50 phút
ctf_sac_v1 [†]	SAC	1,70M	0,183	~152 phút
ctf_poca_v2	MA-POCA	1,70M	0,838	~53 phút
ctf_mappo_v2	MAPPO	1,70M	0,840	~52 phút
ctf_dqn_v1	DQN	1,70M	0,800	~258 phút
Football Table				
FB01	PPO	1,60M	1,691 (ELO 1259)	~47 phút
Football_SAC_01	SAC	1,60M	1,842 (ELO 1254)	~165 phút
Football_POCA_01	MA-POCA	1,60M	1,752 (ELO 1237)	~50 phút
Football_MAPPO_01	MAPPO	1,60M	2,117 (ELO 1260)	~48 phút
football_dqn_v1 [‡]	DQN	1,60M	1,825 (không có ELO)	~147 phút

[†] Thời gian của ctf_sac_v1 chỉ tính đoạn 1,0M–1,7M sau khi khôi phục từ checkpoint; đoạn 0–1,0M chạy trong một phiên trước đó bị ngắt nên không đo được liên mạch.

[‡] Riêng DQN trên Football phải tắt self-play, do ML-Agents phát sinh lỗi khi kết hợp trainer off-policy với ghost trainer. Vì vậy lần chạy này không có chỉ số ELO và **không so sánh trực tiếp được** với bốn thuật toán còn lại trên cùng môi trường.

Ghi chú chung: mọi lần chạy đều dùng 8 training area song song trong mỗi scene. Reward là phần thưởng trung bình mười phần trăm cuối, cùng định nghĩa dùng trong Chương 4.

Mã nguồn và mô hình: Toàn bộ mã nguồn dự án Unity, các tệp cấu hình, script

huấn luyện và các mô hình đã huấn luyện (định dạng .onnx) được lưu trữ tại kho mã nguồn của đề tài.